

Содержание

1. Объектно-ориентированное программирование (ООП)	5
1.1. Инкапсуляция.....	8
1.2. Наследование	19
1.3. Полиморфизм.....	29
2. Поля класса и экземпляра.....	32
3. Статические методы.....	35
4. Специальные методы	36
4.1. Конструктор класса, метод <code>__init__</code>	37
4.2. Строковое представление объекта.....	38
4.3. Перегрузка операторов	40
5. Композиция классов	49
6. Объектно-ориентированная декомпозиция	50
6.1. Применение декомпозиции при решении задач	51
7. Проект: Применение композиции классов	53
7.1. Варианты средней сложности	53
<i>Вариант 1. «Игра и вопросы»</i>	<i>53</i>
<i>Вариант 2. «Библиотека и книги»</i>	<i>56</i>
<i>Вариант 3. «Музыкальный проигрыватель»</i>	<i>59</i>
<i>Вариант 4. «Управление школой».....</i>	<i>62</i>
7.2. Варианты повышенной сложности.....	64
<i>Вариант 5. «Предмет и Собиратель»</i>	<i>64</i>
<i>Вариант 6. «Железный дровосек»</i>	<i>70</i>
<i>Вариант 7. «Продвинутая оранжерея».....</i>	<i>76</i>
8. Составление формальных грамматик	84
8.1. Формальные и естественные языки.....	84
8.2. Синтаксический разбор.....	86
8.3. Синтаксис, семантика и лексика.....	91
8.4. Порождающая и распознающая грамматики	93
8.5. Модель распознавателя.....	94
8.5.1. Построение таблицы переходов распознавателя на примере	96
8.5.2. Создание распознавателя на языке Python	99

9.	Проект: Моделирование распознавателя.....	103
	<i>Вариант 1.</i> «Двоичная запись».....	103
	<i>Вариант 2.</i> «Восьмеричная запись».....	104
	<i>Вариант 3.</i> «Шестнадцатеричная запись»	104
	<i>Вариант 4.</i> «Путь к файлу».....	105
	<i>Вариант 5.</i> «Язык гласных и согласных»	105
	<i>Вариант 6.</i> «Цепочки из латиницы»	106
	<i>Вариант 7.</i> «Десятеричная запись».....	106
	<i>Вариант 8.</i> «Запись дат, вер. 1».....	107
	<i>Вариант 9.</i> «Запись дат, вер. 2».....	107
	<i>Вариант 10.</i> «Язык гласных и согласных, вер. 2».....	108
	<i>Вариант 11.</i> «Цепочки из латиницы, вер. 2»	108
	<i>Вариант 12.</i> «Русская нотация шахматных фигур»	109
	<i>Вариант 13.</i> «Английская нотация шахматных фигур»	110
	<i>Вариант 14.</i> «Оператор присваивания»	111
	<i>Вариант 15.</i> «Пара ключ-значение»	111
10.	Работа с <code>pip</code>	112
11.	Создание графического интерфейса с помощью PyQT5.....	113
	11.1. Используемые инструменты	113
	11.2. Создание первого проекта с графическим интерфейсом	115
	11.2.1. Создание проекта и подготовка макета окна	115
	11.2.2. Перевод файла макета окна в код на Python	118
	11.2.3. Подключение файла GUI к проекту на Python	119
	11.2.4. Добавление кнопки	120
	11.3. Создание приложения-библиотеки.....	121
	11.3.1. Настройка общего шрифта.....	121
	11.3.2. Планирование пользовательского интерфейса	121
	11.3.3. Определение классов-наследников макетов окон	127
	11.3.4. Запуск и тестирование работы проекта	129
	11.3.5. Доработка проекта – очистка текстовых полей	132
	11.3.6. Работа с таблицей.....	132
	11.3.7. Поиск книг по ключевому слову	134

11.3.8.	Удаление элемента из таблицы	137
11.3.9.	Самостоятельная доработка приложения.....	140
12.	Массивы и списки.....	141
12.1.	Пример работы с массивом	141
12.2.	Пример работы со связанным списком.....	143
12.3.	Оценка работы операций над структурами данных	145
12.4.	Сущность списков в Python.....	145
12.5.	Резюме	150
13.	Научное программирование. Библиотека Matplotlib	150
13.1.	Точечные диаграммы и линейные графики.....	151
a.	Настройка параметров графика.....	155
i.	Описание графика функции	155
ii.	Расположение описания графика функции	155
iii.	Название графика и метки осей	156
iv.	Расширенная настройка графиков	157
14.	Библиотека NumPy	161
14.1.	Массивы в NumPy	162
14.1.1.	Создание простого массива	163
14.1.2.	Типизация массивов	164
14.1.3.	Создание формы массива	165
14.1.4.	Создание массива из последовательности	167
14.1.5.	Создание массива на основе функции	169
14.1.6.	Полезные поля класса ndarray	171
14.2.	Основные типы данных dtype	171
14.3.	Универсальные функции (unifunc)	173
14.4.	Разница между вектором и матрицей.....	176
14.5.	Операции над матрицами	176
14.5.1.	Транспонирование матрицы	176
14.5.2.	Умножение матриц	177
15.	Регулярные выражения	179
15.1.	Поиск первого совпадения. Функции match и search	181
15.1.1.	Функция match.....	181

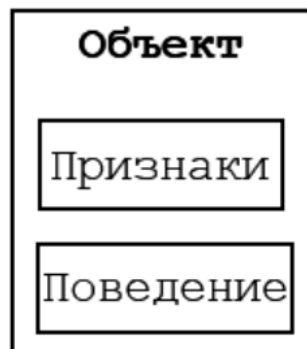
15.1.2.	Функция search	182
15.2.	Поиск всех совпадений. Функция findall	185
15.3.	Комбинация нескольких регулярных выражений в одно	189
15.4.	Получение подстрок по шаблону. Функция split	190
15.5.	Задачи для самостоятельного выполнения	192
15.5.1.	Легкие задачи	192
15.5.2.	Определение регистрационных знаков ТС	192
15.5.3.	Проверка формата даты.....	194

1. Объектно-ориентированное программирование (ООП)

Объектно-ориентированное программирование (ООП) – парадигма программирования, в которой основной концепцией является взаимодействие объектов между собой. Каждый объект является экземпляром класса, т.е. создается по его образу и подобию.

Класс – это тип, описывающий устройство объекта. По сути, чертеж, на основе которого создаются объекты.

Объект – это сущность, которая имеет свои признаки (поля) и поведение (методы), т.е. объединяет в себе данные и функции, производимые над этими данными.



В совокупности признаки (поля) объекта можно охарактеризовать таким словом, как «состояние».

Опишем чертеж создания объекта – создадим класс на Python. Создадим класс «калькулятор», признаками которого будет два хранимых числа в памяти, а поведением будет сложение двух чисел, запоминаемых в полях:

```
class Calculator:
    x = 5
    y = 10

    def summary(self):
        z = self.x + self.y
        return z
```

Для создания класса должно использоваться ключевое слово «class», затем его имя и двоеточие. Затем могут задаваться значения полей класса по умолчанию, т.е. при создании любого объекта этого класса изначально «x» будет равно 5, а «y» всегда 10.

Затем идет описание метода сложения, для этого ставится ключевое слово `def`, такое же как и у функций. По сути, методы классов – это функции, которые могут быть вызваны только у этого класса, а также могут напрямую обращаться к состоянию этого класса через ключевое слово «`self`», что означает собственный, как это сделано для расчета `z`.

У метода класса всегда первым параметром должно идти ключевое слово «`self`».

Теперь используем экземпляр класса в коде, т.е. создадим объект «калькулятора» и вызовем метод сложения:

```
Calc = Calculator() # Создание экземпляра класса
# Записываем результат сложения в переменную
answer = Calc.summary()
print(answer)
```

На экран будет выведено:

15

Абстрагирование – один из главных способов решения сложных задач, это процесс выделения существенных для наблюдателя характеристик некоторого объекта, которые отличает его от других объектов, а также исключения незначительных деталей.

Например, если бы мы сравнивали кошек и собак только по звукам, которые они издают, то оставили бы только абстракции, в которых бы хранился издаваемый звук и ничего лишнего (цвет шерсти, длина хвоста и пр.).

Поведение (методы) – объекты не существуют в вакууме - они воздействуют на другие объекты или же сами подвергаются воздействию. В ходе этих воздействий могут изменяться состояния воздействующих друг с другом объектов.

Например, создадим класс коробка, которая будет хранить в себе апельсины и класс человека, поведением которого может быть поедание одного апельсина из коробки:

```
# Создаем класс коробки
class Box:
    # У него есть одного поле - количество апельсинов
    oranges = 10

# Класс человека
class Human:
    # Одно из возможных поведений - поедание апельсина
    def eat_orange(self, fruit_box):
        fruit_box.oranges -= 1

orange_box = Box()
eater = Human()
eater.eat_orange(orange_box)
print(orange_box.oranges)
```

В основе ООП лежит объектно-ориентированная декомпозиция. Разработка объектно-ориентированных программ заключается в построении иерархии классов, описывающих отношения между объектами, и в определении классов. Вычисления в объектно-ориентированной программе задаются сообщениями, передаваемыми от одного объекта к другому. Объектно-ориентированная парадигма программирования — это попытка объединить лучшие свойства других вычислительных моделей (императивной, функциональной и декларативной).

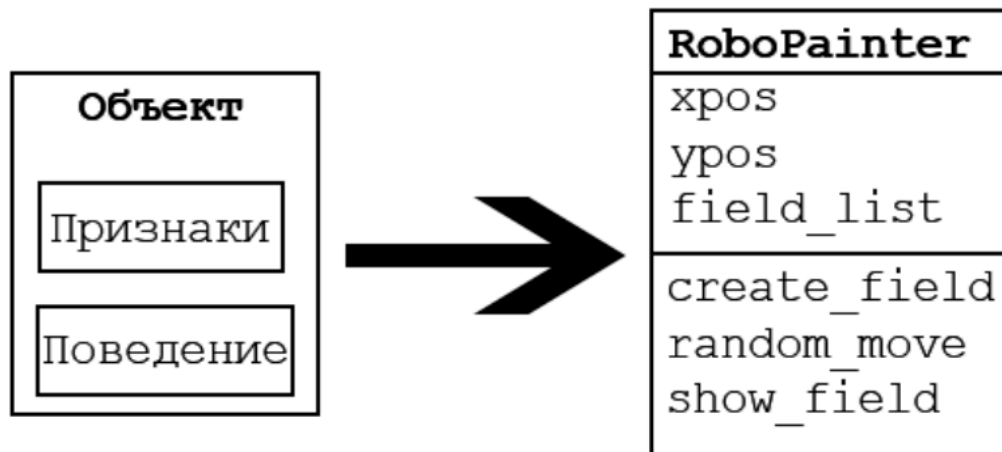
1.1. Инкапсуляция

Инкапсуляция является одной из основных ключевых возможностей ООП, она позволяет объединять множество полей и методов внутри одного класса.

Такая группировка данных позволяет ранее разрозненное описание, например, робота-красильщика из задачи раздела «Функции» не объединять логически в голове программиста следующие переменные и функции:

```
def create_field(field_list, field_size):  
...  
def random_move(field_list, xpos, ypos):  
...  
def show_field(field_list):  
...  
xpos = 0 # Положение робота по x  
ypos = 0 # Положение робота по y  
field_list = [] # Матрица закрашиваемого поля по x
```

Применим инкапсуляцию и сгруппируем имена переменных и функции в один класс, который будет отвечать за всю работу робота-красильщика:



На Python это будет выглядеть примерно так:

```
class Robopainter:  
    xpos = 0  
    ypos = 0  
    field_list = []  
  
    def create_field(self, field_size):  
        ...  
    def random_move(self):  
        ...
```

```
def show_field(self):  
    ...
```

Теперь не требуется задавать параметрами положение красильщика и матрицу поля, так как они уже принадлежат данному классу.

Инкапсуляция позволяет ограничивать данные и функции от их изменения и вызова извне класса, а также помогает достигнуть сокрытия данных.

В Python можно делать поля приватными, т.е. извне такие поля изменять нельзя, используя одно или два знака подчеркивания перед именем поля.

Создадим класс «Касса» (CashRegister) и попробуем изменить приватное поле общей стоимости продажи (selling_price):

```
class CashRegister:  
    __selling_price = 500  
    def change_price(self, new_price):  
        self.__selling_price = new_price  
    def sell_product(self):  
        print(f'Покупка на сумму {self.__selling_price }')  
  
# Создание объекта класса «Касса»  
CR = CashRegister()  
CR.sell_product()  
  
# Попытка изменить стоимость напрямую:  
CR.__selling_price = 650  
CR.sell_product()  
  
# Изменение стоимости с помощью метода  
CR.change_price(650)  
CR.sell_product()
```

Результат работы программы:

```
Покупка на сумму 500  
Покупка на сумму 500  
Покупка на сумму 650
```

Действительно, напрямую поле объекта кассы изменить не удалось, только с помощью специального метода. Можно сказать, что приватные поля сокрыты от внешнего воздействия, что защищает их от случайного изменения.

Попрактикуемся:

Взаимодействие с объектами указанных ниже классов происходит посредством вызова команд, например – если будет выбрана опция «1», то это означает вызов первого метода по списку. Ввод команд заканчивается, когда будет встречен ноль.

Описанные в задачах поля и методы требуется реализовать, но не воспрещается добавлять свои, которые могут упростить работу класса.

Задание 1. Создайте класс «Лягушка» (Frog). У неё обязательно должно быть поле – счетчик пойманных мух. У неё есть два метода – издать звук (make_sound) и поймать муху (catch_fly).

Лягушка может издавать разные звуки – привычное «Ква», «Фьюу» и «*писк*» в случайном порядке.

При нажатии на единицу будет вызываться метод **make_sound**, при 2 – **catch_fly**.

Только после того, как она выдаст «Ква», она сможет поймать муху (только одну). Повторная попытка ловли мухи за один «Ква» ни к чему не приведет.

Ввод пользователя:	Результат работы программы:
1 2 1 2 1 2 0	- *писк* Лягушке не удалось никого поймать - Ква Лягушка поймала муху! - Фьюу Лягушке не удалось никого поймать Пойманных мух: 1
1 2 2 0	- Ква Лягушка поймала муху! Лягушке не удалось никого поймать Пойманных мух: 1

Задание 2. Железный дровосек занимается вырубкой деревьев, каждая такая вырубка расходует масло железного дровосека в зависимости от типа дерева. Следующие данные должен помнить дровосек во время работы:

Тип дерева	Кол-во ударов топором для сруба	Расход масла за удар, %	Приносит дерева после сруба

Молодая сосна	1	10	2
Ель	2	7.5	3
Кедр	3	6	4
Дуб	5	8	10

Дровосек носит с собой волшебную нескончаемую масленку, которой он может себя смазать, если у него запас масла больше 0%. Задача дровосека срубить как можно больше деревьев, не забывая смазывать себя время-от-времени.

Создайте класс «Железный дровосек» (TinWoodman), который имеет память из таблицы выше (wood_data), а также показатели масла (oil), количества собранного дерева (overall_wood) и количества срубленных деревьев (trees_cutted).

Реализуйте следующие методы:

- работа дровосека (main_work) – основной метод, от которого начинается работа дровосека;
- поиск дерева (search_tree) – дровосек находит дерево случайное из списка выше. Любой тип деревьев можно найти с одинаковой вероятностью;
- срубить дерево (cut_tree) – дровосек замахивается и делает удар топором по найденному дереву. Большинству деревьев для сруба нужен не один удар, деревянные доски дровосек получает только после полного сруба;
- расход масла (lower_oil) – при ударах расходуется масло, реализуйте это через метод. Если показатель масла ≤ 0 , то дровосек должен заржаветь;
- пополнить масло (fill_oil) – дровосек поливает себя из масленки и восстанавливает показатель масла на 100%;
- статистика (stat) – вывести данные сколько масла осталось, сколько всего масла израсходовал дровосек, сколько получил деревянных досок и сколько деревьев он срубил;
- вернуться домой (end_work) – дровосек заканчивает на сегодня работу и возвращается домой. Также выводится статистика.

Пример работы программы – 1

Обычный сценарий работы дровосека

Дровосек начинает работу в лесу

Выберите действие:

- 1 – поиск дерева;
- 2 – пополнить масло;
- 3 – статистика;
- 4 – вернуться домой.

Ответ: 1

Дровосек находит Молодая сосна!

Выберите действие:

- 1 – срубить дерево;
- 2 – пополнить масло;
- 3 – статистика;
- 4 – вернуться домой.

Ответ: 1

Дровосек делает удар по Молодая сосна (1/1)!

Масло (90/100)

Дерево упало, дровосек получает 2 доски.

Выберите действие:

- 1 – поиск дерева;
- 2 – пополнить масло;
- 3 – статистика;
- 4 – вернуться домой.

Ответ: 2

Дровосек достает масленку

Масло (100/100)

Выберите действие:

- 1 – поиск дерева;
- 2 – пополнить масло;
- 3 – статистика;
- 4 – вернуться домой.

Ответ: 3

Статистика:

Масло: 100/100

Израсходовано всего масла: 10

Доски: 2

Срублено деревьев: 1

Выберите действие:

- 1 – поиск дерева;
- 2 – пополнить масло;

3 – статистика;
4 – вернуться домой.

Ответ: 4

После тяжелого трудового дня дровосек возвращается домой.

Статистика:

Масло: 100/100

Израсходовано всего масла: 10

Доски: 2

Срублено деревьев: 1

Пример работы программы – 2

Сценарий расхода масла до нуля

Дровосек начинает работу в лесу

...

(проходит время, у дровосека осталось только 15% масла)

...

Выберите действие:

- 1 – поиск дерева;
- 2 – пополнить масло;
- 3 – статистика;
- 4 – вернуться домой.

Ответ: 1

Дровосек находит Дуб!

Выберите действие:

- 1 – срубить дерево;
- 2 – пополнить масло;
- 3 – статистика;
- 4 – вернуться домой.

Ответ: 1

Дровосек делает удар по Дуб (1/5)!

Масло (7/100)

Выберите действие:

- 1 – срубить дерево;
- 2 – пополнить масло;
- 3 – статистика;
- 4 – вернуться домой.

Ответ: 1

Дровосек делает удар по Дуб (2/5)!

Масло (-1/100)

О нет, масло закончилось!

Дровосек заржавел, игра окончена.

Статистика:

Масло: -1/100

Израсходовано всего масла: 1453

Доски: 501

Срублено деревьев: 50

Пример работы программы – 3

Сценарий расхода масла до нуля при последнем ударе по дереву

Дровосек начинает работу в лесу

...

(проходит время, у дровосека осталось только 15% масла.

Показатель срубленных деревьев - 5)

Выберите действие:

- 1 - поиск дерева;
- 2 - пополнить масло;
- 3 - статистика;
- 4 - вернуться домой.

Ответ: 1

Дровосек находит Ель!

Выберите действие:

- 1 - срубить дерево;
- 2 - пополнить масло;
- 3 - статистика;
- 4 - вернуться домой.

Ответ: 1

Дровосек делает удар по Ель (1/2)!

Масло (7.5/100)

Выберите действие:

- 1 - срубить дерево;
- 2 - пополнить масло;
- 3 - статистика;
- 4 - вернуться домой.

Ответ: 1

Дровосек делает удар по Ель (2/2)!

Масло (0/100)

Дерево упало, дровосек получает 3 доски.

О нет, масло закончилось!

Дровосек заржавел, игра окончена.

Статистика:

Масло: 0/100
Израсходовано всего масла: 100
Доски: 20
Срублено деревьев: 6

Задание 3. Создайте класс «будильник», который должен иметь такие поля как часы и минуты (по умолчанию время 00:00), а также реализует методы:

- поставить срабатывание будильника на заданное время (set_timer);
- сообщить сколько осталось до срабатывания ближайшего будильника (time_rest);
- смена времени (change_time).

Вы можете добавлять другие поля и методы, если таковые вам потребуются для решения задачи.

Допускается, что пользователь вместо «07:05» может ввести «7:5» и программа его поймет.

Ввод пользователя:	Результат работы программы
3 6:15 1 7:30 3 7:30 2 0	Время установлено на 06:15 Установлен будильник на 07:30 Время установлено на 07:30 Сработал будильник! Нет заведенных будильников
1 07:40 1 07:50 1 07:55 3 07:49 2 0	Установлен будильник на 07:40 Установлен будильник на 07:50 Установлен будильник на 07:55 Время установлено на 07:49 Вы пропустили будильник на 07:40! Ближайший будильник сработает через 1 минуту.
3 23:50 1 0:2 2 0	Время установлено на 23:50 Установлен будильник на 00:02

	Ближайший будильник сработает через 12 минут
3 23:50 1 7:55 2 0	Время установлено на 23:50 Установлен будильник на 07:55 Ближайший будильник сработает через 8 часов и 5 минут

Задание 4. Создайте класс «Касса», в кассе могут храниться следующие номиналы купюр 1, 5, 10, 50, 100, 200, 500, 1000, 2000, 5000 в различных количествах.

Создайте методы, которые реализуют работу:

- задать начальное значение купюр и их количество в кассе (set_data);
- на вход поступает цена товара и оплата за товар – вывести сколько должна быть сдача и какими купюрами (sell_product). Введенная оплата должна быть переведена в купюры и добавиться к данным, сдача же должна уйти из кассы;
- вывести на экран общую сумму, хранящуюся в кассе, а также номиналы оставшихся купюр (show_total_amount).

На сдачу сначала должны выдаваться максимально большие купюры. Также и с оплатой - если поступило в качестве оплаты товара сумма 125 – это значит, что было подано на вход 100 рублей - 1 купюра, 10 рублей – 2 монеты, и 5 рублей – 1 монета.

Ввод пользователя	Результат работы программы
1 100-4, 50-4, 10-11, 5-30, 1-100 2 200 500 3	Касса записана Покупка прошла успешно! Сдача: 300 руб. Номиналы: 100 руб. -- 3 шт. Общая сумма в кассе: 1160 руб. Номиналы: 1 руб. -- 100 шт. 5 руб. -- 30 шт. 10 руб. -- 11 шт. 50 руб. -- 4 шт. 100 руб. -- 1 шт. 500 руб. -- 1 шт.
1 5000-3, 2000-30, 1000-5 3 2 38000 40000	Касса записана

3 0	Общая сумма в кассе: 80000 руб. Номиналы: 1000 руб. -- 5 шт. 2000 руб. -- 30 шт. 5000 руб. -- 3 шт. Покупка прошла успешно! Сдача: 2000 руб. Номиналы: 2000 руб. -- 1 шт. Общая сумма в кассе: 118000 руб. Номиналы: 1000 руб. -- 5 шт. 2000 руб. -- 29 шт. 5000 руб. -- 11 шт.
1 5000-3,2000-30,1000-5,500-50,200-30,100-4,50-4,10-11,5-30,1-100 2 939 1000 2 8567 10000 2 50062 50100 3 0	Касса записана Покупка прошла успешно! Сдача: 61 руб. Номиналы: 1 руб. -- 1 шт. 10 руб. -- 1 шт. 50 руб. -- 1 шт. Покупка прошла успешно! Сдача: 1433 руб. Номиналы: 1 руб. -- 3 шт. 10 руб. -- 3 шт. 200 руб. -- 2 шт. 1000 руб. -- 1 шт. Покупка прошла успешно! Сдача: 38 руб. Номиналы: 1 руб. -- 3 шт. 5 руб. -- 1 шт. 10 руб. -- 3 шт. Общая сумма в кассе: 171528 руб. Номиналы: 1 руб. -- 93 шт. 5 руб. -- 29 шт. 10 руб. -- 4 шт. 50 руб. -- 3 шт.

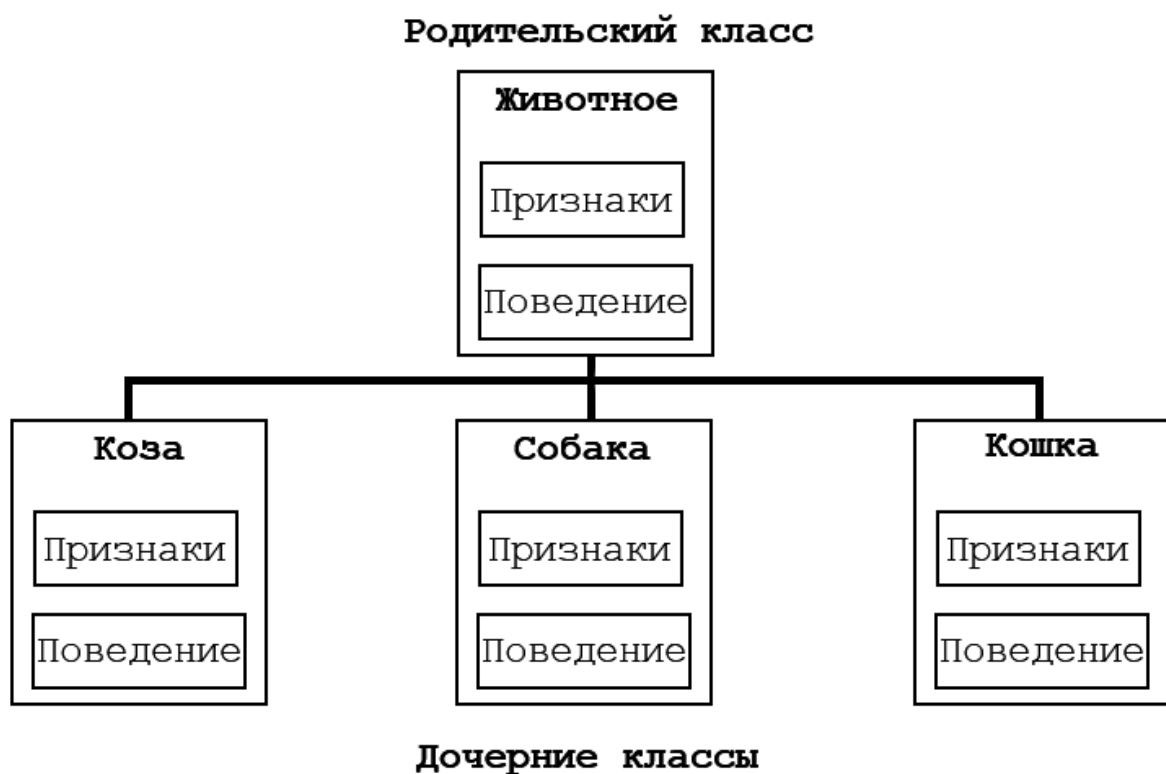
	100 руб. -- 5 шт. 200 руб. -- 28 шт. 500 руб. -- 50 шт. 1000 руб. -- 5 шт. 2000 руб. -- 30 шт. 5000 руб. -- 15 шт.
1 100-4 2 200 1000 3 0	Касса записана Не хватило валюты! Касса не может выдать оставшиеся 400 рублей! Пополните кассу и повторите операцию Общая сумма в кассе: 400 руб. Номиналы: 100 руб. -- 4 шт.

1.2. Наследование

Наследование – это способ создания новых классов на основе существующих, дочерний класс наследует признаки и поведение родительского класса. Дочерний класс может иметь новые свойства или же измененные прежние. Наследование позволяет создавать иерархии классов и является эффективным средством внесения изменений и дополнений в программы.

«Родительский» и «дочерний» классы также имеют такие названия как «базовый» и «производный» соответственно.

Например, создадим класс «Животное», признаки и поведение которых будут наследовать дочерние классы «Коза», «Собака» и «Кошка».



Напишем на Python родительский «Животное» и два дочерних класса – «Коза» и «Собака»:

```
class Animal:
    def eat(self):
        print('Я могу есть')
    def sleep(self):
        print('Я могу спать')
    def say(self):
        print('Я могу общаться')

class Goat(Animal):
```

```

def eat(self):
    print('Я могу есть траву')
def say(self):
    print('Беее!')

class Dog(Animal):
    def eat(self):
        print('Я ем всё что даст хозяин')
    def say(self):
        print('Буф!')
    def guard(self):
        print('Я могу охранять дом')

# Создание объекта класса "Коза"
goat_1 = Goat()
# Вызов метода, наследуемого от родительского класса
goat_1.sleep()
# Вызов переопределенных методов родительского класса
goat_1.eat()
goat_1.say()

# Создание объекта класса "Собака"
dog_1 = Dog()
# Вызов переопределенных методов родительского класса
dog_1.eat()
dog_1.say()
# Вызов метода, который был объявлен только в дочернем классе
dog_1.guard()

```

Результат работы программы:

```

Я могу спать
Я могу есть траву
Беее!
Я ем всё что даст хозяин
Буф!
Я могу охранять дом

```

Наследовать можно сразу свойства нескольких классов, т.е. можно было бы написать класс C, наследующий свойства классов A и B:

```
class C(A,B):
```

Применение наследования позволяет создать иерархию классов, т.е. упорядоченную систему абстракций, что значительно упрощает понимание сложных систем.

Разобрать super()

```
def to_grow(self, time_change: Time):  
    iters_of_growing = time_change // self.minutes_to_grow  
    print(self.curr_grow + iters_of_growing * self.grow_change)  
    if self.curr_grow + iters_of_growing * self.grow_change < 90:  
        super().to_grow(time_change)
```

Попрактикуемся:

Описанные в задачах поля и методы требуется реализовать, но не воспрещается добавлять свои, которые могут упростить работу класса.

Задание 1. Создать родительский класс «Человек» (Human) и дочерний класс «Студент» (Student).

«Человек» обладает полями «имя» (name), «пол» (sex), «рост» (height), «вес» (weight). Методами класса будут:

- «задать имя» (set_name);
- «задать пол» (set_sex);
- «задать рост» (set_height);
- «задать вес» (set_weight).

«Студент» дополнительно обладает полями «год поступления» (year_entered) и «сколько лет обучаться» (graduate). Методами класса будут:

- «задать год поступления» (set_year_entered);
- «задать сколько лет обучаться» (set_graduate);
- «студент отучился год» (increase_year) – студент отучился один год, как только он закончит обучение, нужно будет сообщить об этом.

Следующие команды определены в программе:

- 1 <Имя> <Пол> <Рост> <Вес> - задать данные объекта родительского класса;
- 2 <Имя> - задать имя объекта родительского класса;
- 3 <Имя> <Пол> <Рост> <Вес> - задать основные данные объекта производного класса;
- 4 <год поступления> <сколько лет обучаться> - задать дополнительные данные объекту производного класса;
- 5 <Имя> - изменить имя объекта производного класса;
- 6 – вызов метода «студент отучился год»;
- 7 – вывод данных полей объекта родительского класса;
- 8 – вывод данных полей объекта дочернего класса;

- 0 – завершение работы программы.

Ввод пользователя	Результат работы программы
1 Валентина Ж 170 60 7 3 Анатолий М 180 70 4 2018 4 8 2 Роза 5 Александр 6 6 8 6 6 0	Имя: Валентина Пол: Ж Рост: 170 Вес: 60 Имя: Анатолий Пол: М Рост: 180 Вес: 70 Год поступления: 2018 Лет обучаться: 4 Имя: Александр Пол: М Рост: 180 Вес: 70 Год поступления: 2018 Лет обучаться: 2 Александр закончил обучение – 2022 год!

Задание 2. В качестве родительского класса создайте класс «Инструмент», а также два дочерних класса – «Кирка» и «Лопата».

Каждый из инструментов обладает такими свойствами как «Редкость» (rarity) и «Прочность» (durability).

При обнаружении металлов герой будет добывать их с помощью кирки – mine_metal, при обнаружении завалов будет их расчищать с помощью лопаты - clear_dirt.

Редкость влияет на множитель снижения прочности инструмента. Сложность добычи металлов – 25 прочности, сложность расчистки завалов – 15 прочности. При добыче металлов герой получает 1 металл, при расчистке завалов 1 мешок земли.

При поломке одного из инструментов герой достает новый из волшебной сумки со случайной редкостью:

Редкость	Множитель сложности добычи	Вероятность выпадения
----------	----------------------------	-----------------------

Обычная	2	50
Редкая	1.25	35
Легендарная	0.65	15

Вероятность нахождения металлов – 20%, завала – 80%.

В качестве способа организации игры напишите функцию start, которая будет запускать игру, создавать объекты классов, выводить на экран различные события и считывать пользовательский ввод с клавиатуры.

Пример работы программы:

Вы начинаете игру

Инструменты:

- «Обычная» кирка 100/100
- «Обычная» лопата 100/100

Герою попадаетесь жила!

Выберите действие:

1. добывать киркой;
2. копать лопатой;
3. статистика;
4. завершить игру.

Ответ: 2

Металлы нельзя добывать лопатой!

Выберите действие:

1. добывать киркой;
2. копать лопатой;
3. статистика;
4. завершить игру.

Ответ: 1

Герой добывает 1 металл, Кирка 50/100

Герою попадаетесь жила!

Выберите действие:

1. добывать киркой;
2. копать лопатой;
3. статистика;
4. завершить игру.

Ответ: 1

Герой добывает 1 металл, Кирка 0/100

«Обычная» кирка разрушается!

Герой достает из сумки «Легендарная» кирка с вероятностью 15%

Герою попадаетесь жила!

Выберите действие:

1. добывать киркой;
2. копать лопатой;
3. статистика;
4. завершить игру.

Ответ: 1

Герой добывает 1 металл, Кирка 84/100

Герой натывается на завал!

Выберите действие:

1. добывать киркой;
2. копать лопатой;
3. статистика;
4. завершить игру.

Ответ: 4

Задание 3. Родительским классом будет «Растение», которые обладает следующими полями:

- текущие вода и свет (curr_water и curr_sun);
- максимально возможное количество воды и света, при котором растение не погибнет (max_water и max_sun);
- **используемый грунт (curr_soil);**
- подходящий для цветка грунт (fav_soil) (по умолчанию любой);
- прогресс роста (curr_progress).

И методами:

- **рост (grow)** – увеличивается прогресс роста растения. **На это влияет подходящий ли выбран грунт (в неподходящем растет в 25% от нормального).** На рост расходуется вода и накопленный свет;
- **полить водой (get_water)** – повышается уровень влаги растения на заданное значение;
- **фотосинтез (get_sun)** – повышается уровень света на заданное значение.

Цветок погибает, если будут превышены или слишком низки показатели света и воды три раза за жизнь цветка.

Дочерними классами будут «Роза» и «Кактус»:

Роза имеет также поле «Цвет лепестков», когда роза достигает прогресса в 80%, то распускается бутон. Лепестки могут быть голубого, красного, желтого, белого и черного цвета.

Когда «Кактус» достигает прогресса роста в 20%, то у него появляются колючки.

Признаки	Роза	Кактус
fav_soil	Земля	Песок
Рост за ход	30%	20%
Вода за рост	10	5
Свет за рост	15	20
max_water	80	50
max_sun	50	80

Требуется разработать программу, которая работает с объектами классов «Роза» и «Кактус».

Следующие команды определены в программе:

1 <название класса> <уникальное имя объекта> <вода> <свет> <если роза, то цвет лепестка> - создание объекта определенного класса с уникальным именем и начальными значениями воды и света;

2 <уникальное имя объекта> <кол-во воды> - полить цветок с определенным именем;

3 <уникальное имя объекта> <кол-во света> - цветок освещен на некоторое значение;

4 <уникальное имя объекта> - цветок растет;

5 – вывести данные по каждому цветку;

0 – завершить работу программы.

Ввод пользователя	Результат работы программы
1 Rose Роза_1 5 5 Жёлтый	В горшок посажена роза «Роза_1»
1 Cactus Кактус_1 5 5	В горшок посажен кактус «Кактус_1»
2 Кактус_1 20	«Кактус_1» полит (25/50)
3 Кактус_1 50	«Кактус_1» освещен (55/80)
4 Кактус_1	«Кактус_1» растет (20/100) – вода (20/50), свет (35/80)
2 Роза_1 70	У «Кактус_1» появились иголки!
3 Роза_1 40	«Роза_1» полит (75/80)
4 Роза_1	«Роза_1» освещен (45/50)
4 Роза_1	

4 Роза_1 5 0	«Роза_1» растет (30/100) – вода (65/80), свет (30/100) «Роза_1» растет (60/100) – вода (55/80), свет (15/100) «Роза_1» растет (90/100) – вода (45/80), свет (0/100) У «Роза_1» распустился бутон с желтыми лепестками! Критическое значение света у «Роза_1» (1/3)! На подоконнике находится 2 горшка с цветами: - Роза «Роза_1» (90/100): Вода (45/80); Свет (0/100); Угроза жизни (1/3); Имеет желтые лепестки. - Кактус «Кактус_1» (20/100): Вода (20/50); Свет (35/80); Угроза жизни (0/3); Имеет иголки.
1 Cactus Цветок 50 50 2 Цветок 70 3 Цветок 40 5 0	В горшок посажен кактус «Цветок» Критическое значение воды у «Цветок» (1/3)! «Цветок» полит (120/50) Критическое значение воды у «Цветок» (2/3)! «Цветок» освещен (90/80) Критическое значение воды у «Цветок» (3/3)! Критическое значение света у «Цветок» (4/3)! «Цветок» погиб На подоконнике находится 1 горшок с цветами: - Кактус «Цветок» погиб.

Задание 4. Написать родительский класс «Событие» (Event) и дочерний класс «Встреча» (Meeting).

«Событие» обладает полями «время» (time), «место» (place), «содержание события» (meaning) и «статус события» (status).

Методами же являются «назначить время» (set_time), «назначить место» (set_place), «задать содержание» (set_meaning) и «проверить актуальность» (actualize), которое в зависимости от передаваемого как параметр времени сообщает:

- предстоит ли ещё событию произойти - текущее время меньше назначенного. Переводит статус в состояние «ещё не началось»;
- действует ли событие - текущее время в пределах (начало события, начало события+30 минут). Переводит статус в состояние «проводится»;
- событие уже - прошло текущее время больше пределов действия события «завершено».

Невозможно изменить параметры события, если оно уже «проводится» или «завершено».

«Встреча» также обладает полем «с кем встреча» (partner) и «важность встречи» (importance), также существуют методы, которые задают эти поля «set_partner» и «set_importance».

«Важность встречи» может быть задана только в пределах от 1 до 5, где 1 – самая важная встреча, а 5 – самая неважная.

Программа реализует следующие команды:

- 1 – задать текущее время;
- 2 – задать событие;
- 3 – задать встречу;
- 4 – проверить состояние события;
- 5 – проверить состояние встречи;
- 6 – изменить время события;
- 7 – изменить время и важность встречи;
- 8 – вывод данных о событии;
- 9 – вывод данных о встрече;
- 0 – завершение работы программы.

Ввод пользователя	Результат работы программы
1 11:50	Событие «Поломка ПК н.10» создано
2 10:50 «Ауд. 350» «Поломка ПК н.10»	Событие «Поломка ПК н.10» завершено
4	
3 12:33 «Конференц-зал»	Встреча «Переговоры» создана
«Переговоры» «Мушкетов М.П.» 5	
5	

7 13:02 1 9 6 00:02 1 13:10 9 1 14:00 5 0	Встреча «Переговоры» с «Мушкетов М.П.» ещё не началась (важность 5)! Встреча «Переговоры»: С: «Мушкетов М.П.» Место: «Конференц-зал» Время: 13:02 Важность: 1 Статус: ещё не началось Невозможно изменить событие «Поломка ПК н.10» = событие завершено. Встреча «Переговоры»: С: «Мушкетов М.П.» Место: «Конференц-зал» Время: 13:02 Важность: 1 Статус: проводится Встреча «Переговоры» с «Мушкетов М.П.» завершено
1 12:21 2 13:00 «Ауд. 301» «Пара программирование на Python» 3 14:17 «Коридор» «Переговоры» «Груздева Д. В.» 3 4 5 7 15:30 1 9 6 14:00 8 1 14:15 6 14:30 4 5 1 15:30 8 9 1 17:56 4 5 0	Событие «Пара программирование на Python» создано Встреча «Переговоры» создана Событие «Пара программирование на Python» ещё не началось Встреча «Переговоры» с «Груздева» ещё не началось (важность 3)! Встреча «Переговоры»: С: «Груздева» Место: «Коридор» Время: 15:30 Важность: 1 Статус: ещё не началось Событие «Пара программирование на Python»: Место: «Ауд. 301» Время: 14:00 Статус: ещё не началось Невозможно изменить событие «Пара программирование на Python» = событие проводится.

	Событие «Пара программирование на Python» проводится Встреча «Переговоры» с «Груздева» ещё не началось (важность 1)! Событие «Пара программирование на Python»: Место: «Ауд. 301» Время: 14:00 Статус: завершено Встреча «Переговоры»: С: «Груздева» Место: «Коридор» Время: 15:30 Важность: 1 Статус: проводится Событие «Пара программирование на Python» завершено Встреча «Переговоры» с «Груздева» завершено (важность 1)!
--	---

1.3. Полиморфизм

Разберем слово «полиморфизм» - «поли» означает «множество», а «морф» означает «форма», т.е. полиморфизм должен означать способность принимать множество форм.

Полиморфизм в языках программирования может проявляться двумя способами:

- возможность одной сущности (метода, оператора или объекта) совершать различные операции в зависимости от ситуации. Иными словами, у какого объекта класса вызывается метод;
- возможность одной сущности вести себя одинаково при входных данных различных типов.

Рассмотрим первый способ проявления полиморфизма, пусть некий метод в зависимости от разных классов будет совершать различные действия:

Создадим родительский класс «Растение» и дочерние «Роза» и «Дерево», каждый из которых будет иметь метод «рост».

```

class Plant:
    def grow(self):
        print('Растение растёт N лет')

class Rose(Plant):
    def grow(self):
        print('Роза растёт 45 дней')

class Oak(Plant):
    def grow(self):
        print('Дуб растёт 200 лет')

rose_1 = Rose()
rose_1.grow()

oak_1 = Oak()
oak_1.grow()

```

Результат работы программы:

```

Роза растёт 45 дней
Дуб растёт 200 лет

```

Теперь рассмотрим вторую ситуацию – пусть метод класса ведет себя одинаково при входных данных, различных по типу:

Создадим класс «Сумматор», который складывает все значения в переданной структуре данных и возвращает их сумму.

```

class Adder:
    def sum_struct(self, data):
        sum = 0
        for bit in data:
            sum += bit
        return sum

add1 = Adder()

tuple_data = (1, 2, 15, 20)
list_data = [30, 66, 11, 0.5]
set_data = {77, 88, 30, 20}

print(f'Сумма кортежа: {add1.sum_struct(tuple_data)}')
print(f'Сумма списка: {add1.sum_struct(list_data)}')
print(f'Сумма множества: {add1.sum_struct(set_data)}')

```

Полиморфизм позволяет использовать один интерфейс для различных объектов, что даёт возможность писать программистам более эффективный код.

2. Поля класса и экземпляра

Рассмотрим следующий код:

```
class DB:
    data = []
    def add_new_num(self, number):
        self.data.append(number)

numbers_1 = DB()
numbers_2 = DB()

numbers_1.add_new_num(23)

print(f'numbers_1: {numbers_1.data}')
print(f'numbers_2: {numbers_2.data}')
```

Как видно, у класса DB (DataBase) есть поле (или же *атрибут*) «данные» (data), а также метод «добавить новое число» (add_new_num). Затем мы создаем два экземпляра класса DB – numbers_1 и numbers_2 и вызываем метод «добавь новое число» у numbers_1 с аргументом 23. Вопрос: Что будет выведено на экран?

Логично было бы предположить, что число записалось в данные у numbers_1, а у numbers_2 ничего нет. Выполним код:

```
numbers_1: [23]
numbers_2: [23]
```

Как же так? Всё дело в поле класса - поле data было объявлено для класса DB, а не для конкретного экземпляра класса, например, numbers_1 или numbers_2.

Можем это проверить – обратимся к этому полю не у объектов, а у самого класса и даже после удаления объектов (ключевое слово del) данные будут храниться в классе. Доработаем код:

```
class DB:
    data = []
    def add_new_num(self, number):
        self.data.append(number)

numbers_1 = DB()
numbers_2 = DB()

numbers_1.add_new_num(23)
```

```

print(f'numbers_1: {numbers_1.data}')
print(f'numbers_2: {numbers_2.data}')

# Выводим значение поля класса
print(f'Поле класса data: {DB.data}')

# Удаляем объекты класса из памяти
del numbers_1
del numbers_2

print(f'Поле класса data: {DB.data}')

# Можете удостовериться - объектов больше нет
print(f'numbers_1: {numbers_1.data}')
```

Результатом работы программы будет:

```

numbers_1: [23]
numbers_2: [23]
Поле класса data: [23]
Поле класса data: [23]
Traceback (most recent call last):
  File "Q:\test.py", line 23, in <module>
    print(f'numbers_1: {numbers_1.data}')
NameError: name 'numbers_1' is not defined
```

Действительно, данные находятся у класса, а не у его объектов, как мы ожидали ранее. Объекты были удалены, это видно по ошибке на строчке 23 (последней строчке кода) – попытка обратиться к удаленному объекту вернула ошибку.

С этим разобрались, но как же разделить данные для отдельных объектов, вместо общей памяти? Вместо полей класса нужно создать **поля экземпляра**. Сделаем это с помощью метода «задать данные», который будет задавать пустой список у объекта:

```

class DB:
    def set_data(self):
        self.data = []
    def add_new_num(self, number):
        self.data.append(number)

# Создаем объект 1 и задаём пустой список
numbers_1 = DB()
numbers_1.set_data()

# Создаем объект 2 и задаём пустой список
numbers_2 = DB()
numbers_2.set_data()
```

```
numbers_1.add_new_num(23)

print(f'numbers_1: {numbers_1.data}')
print(f'numbers_2: {numbers_2.data}')
```

При выполнении такого кода будет выведено:

```
numbers_1: [23]
numbers_2: []
```

Как видно из результата работы программы, теперь данные разделены между собой.

Зачем же нужны поля класса, ведь они, кажется, вносят неразбериху в создаваемый код? При подходе, как мы писали первый код в этой главе, если будет больше, чем один экземпляр класса, то тогда они начнут перезаписывать друг друга общие данные. У полей класса есть более интересное назначение. Например, с помощью общей памяти можно задавать каждому из создаваемых объектов уникальные имена. Для наглядности доработаем предыдущий класс:

```
class DB:
    counter = 1
    def set_data(self):
        self.name = f'Список чисел - {DB.counter}'
        self.data = []

        DB.counter += 1

    def add_new_num(self, number):
        self.data.append(number)

    def show_data(self):
        print(f'Имя: {self.name}\nЧисла: {self.data}\n')

# Создаем объект 1 и задаём пустой список
numbers_1 = DB()
numbers_1.set_data()

# Создаем объект 2 и задаём пустой список
numbers_2 = DB()
numbers_2.set_data()

# Добавим несколько чисел в списки
numbers_1.add_new_num(23)
numbers_1.add_new_num(-1)
numbers_2.add_new_num(66)

# Выведем данные о списках на экран
```

```
numbers_1.show_data()
numbers_2.show_data()
```

Теперь в методе `set_data` имеется новая строчка `DB.counter+=1`, что означает «обратиться к полю класса `DB` и увеличить значение на единицу». Для удобства работы вывода данных был создан метод `show_data`.

В первом примере мы не обращались к `data` через `DB.data`, но код всё равно работал – это связано с тем, что Python ссылался на одну область данных у `data`. При создании переменной типов `int`, `float`, `str` и `bool` такого не происходит, без явного обращения.

Т.е. если сейчас мы добавим поле класса в `DB`:

```
class DB:
    data = []
    counter = 1
    def set_data(self):
        self.name = f'Список чисел - {DB.counter}'
        self.data = []

        DB.counter += 1
```

Это ни на что не повлияет, так как при вызове `set_data` создается поле объекта, а значит при добавлении элементов в объекте они будут добавляться в поле объекта (`self.data`, а не `DB.data`).

Резюме

Поля класса – это такие поля, которые хранят общую память для всех объектов этого класса. В общем случае, для того чтобы обратиться к полю класса нужно написать «<имя класса>.<наименование поля класса>». В других языках программирования поля класса называются *статическими* полями.

Поля экземпляра – это поля, которые хранят память для конкретного экземпляра класса. Для обращения к полю экземпляра используется ключевое слово `self`.

Старайтесь не прописывать полями класса переменные, которые предполагаются только как поля экземпляра, иначе невнимательность может повлечь большие проблемы.

3. Статические методы

Скорее всего пропускаем эту тему, в дальнейшем раскрою

```
@staticmethod
def from_file(fname: str, verbose : bool = False)->List['CareerTransition']:
    #Do some stuff
    pass
```

4. Специальные методы

Согласованность между сущностями в языке важна. Поработав с некоторыми объектами в Python вы сможете предположить как работают другие. Так, например, мы узнали, что с помощью цикла while можно получать элементы строки, в т.ч. используя функцию len для определения количества элементов, ровно также можно работать и со списками. Удобно, когда объекты работают по схожим правилам, в качестве второго примера вспомните - чтобы вывести на экран состояние любого из объектов стандартных типов мы использовали print, почему же тогда для пользовательских классов, которые мы создавали ранее нельзя сделать также?

Специальными или же «**магическими**» методами называются методы, которые реализуют работу стандартных операторов и операций для объектов определенного класса.

Специальный метод, такой как «__init__», можно прочесть как «подчеркивание- подчеркивание-инит-подчеркивание-подчеркивание» или же «under-under-init-under-under», но так как это довольно затруднительно, программисты иногда говорят просто «under-under-init» (несмотря на то, что обычное двойное подчеркивание спереди имеет другой смысл) или же «**double under init**» -> «**dunder init**». Опытные программисты на Python поймут вас, если вы будете говорить о dunder-методе.

Старайтесь не создавать собственные атрибуты с именами вида __foo__, потому что в будущем подобные имена могут получить специальный смысл, даже если в текущей версии Python это не так.

4.1. Конструктор класса, метод `__init__`

Конструктор класса – специальный метод, который создает экземпляр класса, т.е. объект. В Python для этого используется метод `__init__`. Название метода «init» является сокращением от «initialization», т.е. «инициализация».

В конструкторе указывается какие начальные значения должны быть у полей создаваемого объекта.

Например, ранее в одном из примеров был написан класс коробки с апельсинами, чтобы изменить начальное значение апельсинов в коробке можно было бы написать метод «set_number»:

```
class Box:
    def set_number(self, new_number):
        self.oranges = new_number

new_box = Box()
# Задание начального значения с помощью метода
new_box.set_number(5)
print(new_box.oranges)
```

Т.е. нужно объявить создание объекта класса Box, а затем вызвать метод, который будет задавать значение полю oranges – set_number. Вместо таких сложностей воспользуемся конструктором, который сразу задаст нужное нам начальное значение при создании объекта:

```
class Box:
    def __init__(self, new_oranges):
        self.oranges = new_oranges

new_box = Box(5)
print(new_box.oranges)

new_box_1 = Box(10)
print(new_box_1.oranges)
```

Пустое значение при создании коробки, например:

```
class Box:
    def __init__(self, new_oranges=0):
        self.oranges = new_oranges

new_box = Box()
```

```
print(new_box.oranges)

new_box = Box(5)
print(new_box.oranges)

new_box_1 = Box(10)
print(new_box_1.oranges)
```

Попрактикуемся:

Задание 1. Создайте список `boxes` и с помощью цикла `for` добавьте в него 10 коробок с апельсинами, в первой коробке должен быть один апельсин, во второй два и так далее. Выведите значение каждого элемента списка на экран с помощью метода `show_data`.

Задание 2. Создайте класс «Карта», которая может иметь два поля – масть (`suite`) и ранг (`rank`). Класс должен также обладать методом `show_data`, который бы выводил значение полей объекта. Напишите конструктор, в которой передаются значения масти и ранга создаваемой карты. Затем в цикле создайте все возможные карты и добавьте их в список «колода» (`deck`).

После этого снова пробегитесь по списку и отобразите все созданные карты.

Разобрать подробно конструктор при наследовании

Сделать пример с рабочими различных профессий в наследовании

4.2. Строковое представление объекта

Возвращаясь к основной теме главы, было бы удобно, если бы мы могли выводить данные об объекте с помощью стандартного `print`, вместо создания своих методов по типу `show_data` из практического задания выше. Для этого мы можем реализовать у класса работу со специальными методами `__repr__` и `__str__`.

В чем же между ними разница?

Задача `__repr__` быть однозначным.

Обратите внимание, что есть одно значение по умолчанию, которое является истинным: если `__repr__` определено, а `__str__` нет, объект будет вести себя так, как если бы `__str__ = __repr__`.

Проще говоря, это означает: почти каждый объект, который вы реализуете, должен иметь функциональный `__repr__`, который можно использовать для понимания объекта. Реализация `__str__` необязательна: сделайте это, если вам нужна функция «красивой печати» (например, используемая генератором отчетов).

Задача `__str__` быть читаемым

Так как у `__str__` нет задачи быть однозначным, если вы реализуете абстракцию IP-адреса, то для `dunder-str` оно может выглядеть просто как «192.168.1.1», для класса даты это может быть «07.04.2023».

Как итог, определяйте `dunder-repr` для любого создаваемого класса, а уже `dunder-str` если вы думаете, что было бы удобно для пользователя выводить определенную часть данных.

Как пример рассмотрим поведение `__repr__` и `__str__` класса для определения сущностей в интернете – UUID:

```
obj = uuid.uuid1()
print(obj.__repr__())
print(obj.__str__())
```

Результатом выполнения программы будет:

```
UUID('2d7fc7f0-7706-11e9-94ae-0242ac110002')
2d7fc7f0-7706-11e9-94ae-0242ac110002
```

Разработчикам нужна комбинация «что за сущность» + «значение», в то время как пользователю это не важно, ему нужно только значение.

```

def to_grow(self, time_change: Time):
    # Остаток времени, который ещё не использован для роста
    growth_piece = Time(0, self.growth_time % self.minutes_to_grow)
    iters_of_growth = (growth_piece + time_change) // self.minutes_to_grow
    print(iters_of_growth)

    k = 0
    while k < iters_of_growth and self.health_danger < 3:
        print(self)
        self.curr_water -= self.spend_water
        self.curr_light -= self.spend_light
        self.curr_grow += self.grow_change
        self.check_minmax()

```

4.3. Перегрузка операторов

Благодаря специальным методам также можно переопределить выполнение таких операций, как сложение, индексирование, подсчет количества элементов и многие другие. Для этого нужно **перегрузить оператор**, чтобы он мог работать с этим классом.

Перегрузка операторов является одним из способов реализации полиморфизма, обеспечивает работу стандартных операторов языка с пользовательскими типами данных.

Приведем ниже таблицу с наименованием всех интересующих нас dunder-методов для перегрузки операторов:

Выполняемая функция	Наименование dunder-метода
Специальные для объектов класса методы	
Инициализация объекта (конструктор)	<code>__init__</code>
Удаление объекта (деструктор)	<code>__del__</code>
Преобразование в строку	
Представление для разработчиков	<code>__repr__</code>
Представление для пользователя	<code>__str__</code>
Преобразование в число	

Взятие по модулю	<code>__abs__</code>
Булево значение	<code>__bool__</code>
Целое число	<code>__int__</code>
Вещественное число	<code>__float__</code>
Индекс элемента	<code>__index__</code>
Эмуляция последовательностей	
Взятие длины последовательности	<code>__len__</code>
Получить элемент	<code>__getitem__</code>
Задать значение элементу	<code>__setitem__</code>
Удаление элемента	<code>__delitem__</code>
Содержит ли последовательность элемент	<code>__contains__</code>
Операторы сравнения	
Меньше, <	<code>__lt__</code>
Меньше либо равно, <=	<code>__le__</code>
Равно, ==	<code>__eq__</code>
Не равно, != или <>	<code>__ne__</code>
Больше, >	<code>__gt__</code>
Больше либо равно, >=	<code>__ge__</code>
Арифметические операторы	
Сложение, +	<code>__add__</code>
Вычитание, -	<code>__sub__</code>
Умножение, *	<code>__mul__</code>
Деление, /	<code>__truediv__</code>
Деление без остатка, //	<code>__floordiv__</code>
Взятие остатка от деления, %	<code>__mod__</code>
Возведение в степень, ** или pow()	<code>__pow__</code>
Инверсные арифметические операторы	
Сложение, +	<code>__radd__</code>
Вычитание, -	<code>__rsub__</code>
Умножение, *	<code>__rmul__</code>

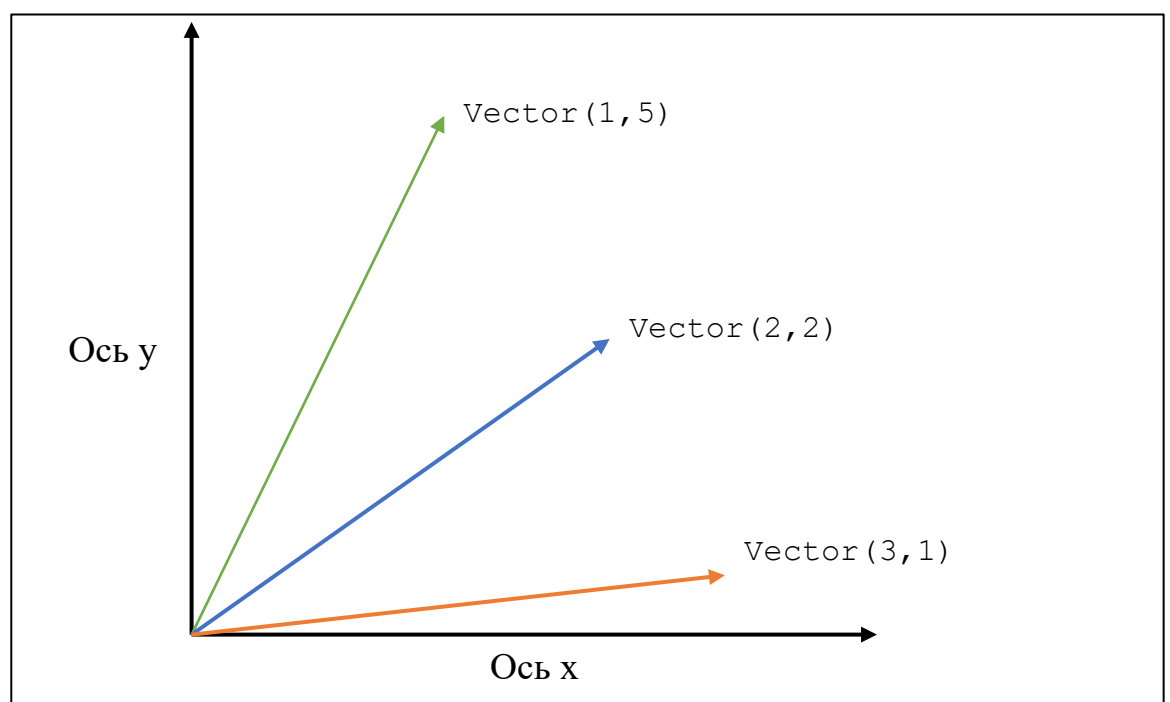
Деление, /	<code>__rtruediv__</code>
Деление без остатка, //	<code>__rfloordiv__</code>
Взятие остатка от деления, %	<code>__rmod__</code>
Возведение в степень, ** или <code>pow()</code>	<code>__rpow__</code>

С полным перечнем можно ознакомиться по ссылке:

<https://docs.python.org/3/reference/datamodel.html#specialnames>

Инверсные операторы нужны для случаев, когда операнды переставлены местами. Например, вместо `class_obj*k` будет написано `k*class_obj`.

Опробуем перегрузку операторов на практике, создадим класс «Вектор», который будет характеризоваться двумя полями – координатой конца по x и y:



Такие вектора можно будет складывать друг с другом, умножать на число, сравнивать операцией «больше чем, >», а также выводить на экран значения вектора для разработчика.

```
class Vector():
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```

# Перегрузка вывода на экран
def __repr__(self):
    return f'Vector({self.x}, {self.y})'

# Перегрузка сложения
def __add__(self, other):
    x = self.x + other.x
    y = self.y + other.y
    return Vector(x, y)

# Перегрузка умножения (умножение на скаляр)
def __mul__(self, number):
    x = self.x * number
    y = self.y * number
    return Vector(x, y)

# Перегрузка сравнения >
def __gt__(self, other):
    # Предположим, что вектор больше другого только
    # если обе его координаты больше
    if self.x > other.x and self.y > other.y:
        return True
    else:
        return False

if __name__ == '__main__':
    v1 = Vector(1,2)
    v2 = Vector(3,4)

    print(f'v1: {v1}')
    print(f'v2: {v2}')

    v3 = v1 + v2
    v4 = v1 * 2
    v5 = v2 * -1

    print(f'{v1} + {v2} = {v3}')
    print(f'{v1} * 2 = {v4}')
    print(f'{v2} * -1 = {v5}')

    comp_1 = v1 > v2
    comp_2 = v2 > v1

    print(f'{v1} > {v2} = {comp_1}')
    print(f'{v2} > {v1} = {comp_2}')

```

Результатом программы будет:

```
v1: Vector(1, 2)
v2: Vector(3, 4)
Vector(1, 2) + Vector(3, 4) = Vector(4, 6)
Vector(1, 2) * 2 = Vector(2, 4)
Vector(3, 4) * -1 = Vector(-3, -4)
Vector(1, 2) > Vector(3, 4) = False
Vector(3, 4) > Vector(1, 2) = True
```

Для того, чтобы было удобнее пользоваться созданным классом, определим тип параметра `number` dunder-метода `__mul__`:

```
def __mul__(self, number: int):
    x = self.x * number
    y = self.y * number
    return Vector(x, y)
```

И тут возникает вопрос – а можно ли обозначить, тип параметра метода `__add__` как класс `Vector`, т.е. чтобы методы класса `Vector` принимали объекты его же самого?

Да, можно, но так как сначала мы определяем сам класс `Vector`, то просто написать код ниже не выйдет:

```
def __add__(self, other: Vector):
    x = self.x + other.x
    y = self.y + other.y
    return Vector(x, y)
```

Можно воспользоваться модулем `__future__`, который позволяет использовать в аннотации метода ещё не объявленный `Vector` как тип данных:

```
from __future__ import annotations

class Vector():
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return f'Vector({self.x}, {self.y})'

    def __add__(self, other: Vector) -> Vector:
        x = self.x + other.x
        y = self.y + other.y
        return Vector(x, y)

    def __mul__(self, number: int) -> Vector:
```

```
x = self.x * number
y = self.y * number
return Vector(x, y)
```

Попрактикуемся:

При создании классов из данных заданий обязательно должны быть реализованы dunder-методы `__init__` и `__repr__` для инициализации объектов и отображения их значений на экран. Остальные методы реализуются как по конкретному заданию, так и на ваше усмотрение.

Задание 1. Доработайте класс «Вектор» так, чтобы он поддерживал вычитание векторов, а также деление вектора на скаляр. Продемонстрируйте это просто в программе.

Задание 2. Доработайте класс «Вектор» - добавьте поле «имя вектора» (name), чтобы с ними было проще взаимодействовать пользователю в дальнейшем.

Также требуется определить в программе следующие команды:

- `1 <название вектора> <x> <y>` - создать вектор с заданным названием и характеристиками, добавить его в общий список;
- `<вектор 1> + <вектор 2> = <вектор 3>` - сложить два вектора между собой, новый вектор записать и назвать как `<вектор 3>`. Результат сложения вывести на экран;
- `<вектор 1> - <вектор 2> = <вектор 3>` - сложить два вектора между собой, новый вектор записать и назвать как `<вектор 3>`. Результат вычитания вывести на экран;
- `<вектор 1> * <скаляр> = <вектор 2>` - умножить вектор на скаляр, новый вектор записать и назвать как `<вектор 2>`. Результат умножения вывести на экран;
- `<вектор 1> / <скаляр> = <вектор 2>` - разделить вектор на скаляр, новый вектор записать и назвать как `<вектор 2>`. Результат деления вывести на экран;
- `<вектор 1> > <вектор 2> = ?` – сравнить два вектора между собой, возвращать True/False. Результат сравнения вывести на экран;
- `<вектор 1> <= <вектор 2> = ?` – сравнить два вектора между собой, возвращать True/False. Результат сравнения вывести на экран;
- `0` – завершить выполнение программы.

Для того, чтобы обращаться по имени к каждому объекту класса требуется организовать их значение – это можно сделать с помощью списка

или словаря. Если результатом сложения записан вектор, имя которого уже занято – нужно переписать старое значение новым.

Ввод пользователя	Результат работы программы
1 Век1 10 5	Век1(10,5) + Век2(-5,2) = Век3(5,7)
1 Век2 -5 2	Век1(10,5) * 2 = Век2(20,10)
Век1 + Век2 = Век3	Век2(20,10) - Век3(5,7) = Век4(15,3)
Век1 * 2 = Век2	Век4(15,3) / 3 = Век1(5,1)
Век2 - Век3 = Век4	Век1(5,1) > Век2(20,10) = False
Век4 / 3 = Век1	Век1(10,5) <= Век2(20,10) = True
Век1 > Век2 = ?	
Век1 <= Век2 = ?	

Задание 3. Создайте класс «Время», у него есть поля – часы и минуты. Объекты класса время можно складывать между собой и вычитать, также можно сравнивать (определить для них все операции сравнения – «>, >=, <, <=, ==, !=»).

Входные данные	Выходные данные
12:50 + 23:10 = ?	12:50 + 23:10 = 12:00
05:05 + 20:40 - 10:20 = ?	05:05 + 20:40 - 10:20 = 15:25
05:05 < 23:25 = ?	05:05 < 23:25 = True
11:45 >= 23:25 = ?	11:45 >= 23:25 = False
05:05 == 23:25 = ?	05:05 == 23:25 = False
12:00 != 00:00 = ?	12:00 != 00:00 = True
0	

Гарантируется, что на вход будут подаваться строки в формате:
<два числа, разделенные двоеточием I> <знак оператора I> ... <два числа, разделенные двоеточием N-I> <знак оператора N-I> <два числа, разделенные двоеточием N> = ?

Задание 4. Создаваемая программа будет являться справочником для создания различных предметов в некоторой игре. Создайте класс «Предмет» (Item). Предметы можно складывать между собой, а также некоторые предметы можно разбирать и получать часть из предметов, требуемых для их создания ранее.

Требуется определить операцию получения нового предмета, как сложение объектов класса «Предмет», а разбор предметов – вычитание.

Все рецепты, которые используют операцию сложения, обладают свойством **коммутативности**. Коммутативность означает возможность менять местами элементы операции без изменения результата выполнения, т.е. $a+b = b+a$.

Рецепт	Предмет
Сборка предметов	
Трава + Трава	Веревка
Уголь + Палка	Факел
Большой камень + Веревка + Палка	Топор
Топор + Дерево	Палка
Острый камень + Веревка + Палка	Копье
Палка + Палка + Факел	Костер
Курица + Копье	Мясо
Мясо + Костер	Жареное мясо
Вода + Костер	Уголь
Вода + Костер + Мясо	Тушеное мясо
Разбор предметов	
Веревка - Разборщик	Трава
Топор - Разборщик	Палка
Копье - Разборщик	Палка
Одежда - Разборщик	Ткань
Топор - Верстак	Большой камень
Копье - Верстак	Острый камень

Считывание с клавиатуры будет до того момента, пока пользователь не введет ноль.

Ниже представлены тесты работы подобной программы:

Ввод пользователя	Результат работы программы
-------------------	----------------------------

Искомый рецепт: Трава + Трава	Рецепт даст игроку: Веревка
Искомый рецепт: Палка + Уголь	Рецепт даст игроку: Факел
Искомый рецепт: Палка + Палка	Рецепт даст игроку: Такого рецепта не существует
Искомый рецепт: 0	Выход из справочника...
Искомый рецепт: Трава	Рецепт даст игроку: Такого рецепта не существует
Искомый рецепт: Костер + Мясо	Рецепт даст игроку: Жареное мясо
Искомый рецепт: Мясо + Костер + Вода	Рецепт даст игроку: Тушеное мясо
Искомый рецепт: Топор – Верстак	Рецепт даст игроку: Большой камень
Искомый рецепт: Верстак – Топор	Рецепт даст игроку: Такого рецепта не существует
Искомый рецепт: 0	Выход из справочника...

Комментарии к условию задачи:

Несмотря на то, что сложение должно иметь смысл «Все ресурсы будут потрачены для того, чтобы получить что-то новое», а вычитание ($a-b=c$) «Чтобы из $\langle a \rangle$ получить $\langle c \rangle$, был использован $\langle b \rangle$ », при написании рецептов приготовления с костром эта мысль теряется.

Рассмотрим рецепт «Мясо + Костер» - костер ведь теряться не должен, он может дальше гореть, в результате рецепта просто прожарилось мясо. Причина, по которой была допущена такая вольность - хотелось, чтобы вы попробовали сообразить как сделать так, чтобы рецепты «Вода + Костер» и «Вода + Костер + Мясо» возвращали разные значения, при этом обладая свойством коммутативности.

В дальнейшем, при взятии за основу данного задания его рецепты будут изменены, корректнее было бы писать рецепт тушеного мяса не как «Вода + Костер + Мясо», а «(Вода + Мясо) - Костер».

5. Композиция классов

Ранее мы задавали поля экземпляров класса различными типами данных – числами, строкой, списком и словарем, но поля также могут иметь тип пользовательского класса.

Тип связи между классами, когда один является частью другого называется **композицией**. Причем, класс, принадлежащий одному классу, не может одновременно принадлежать другому.

Например, композицией классов может быть связь «человек-рука». Человеку принадлежит объект класса рука. Одна рука двум разным людям принадлежать не может.

Опишем композицию классов «человек-рука». Пусть у руки есть поведение «совершить движение», а у человека есть поведение «подвигать руками», где он будет вызывать у каждой руки вышеназванный метод:

```
class Hand():
    def __init__(self, name):
        self.name = name
    def make_move(self):
        print(f'- {self.name} совершает движение')

class Human():
    def __init__(self, name):
        self.name = name
        self.LeftHand = Hand('левая рука')
        self.RightHand = Hand('правая рука')

    def move_hands(self):
        print(f'{self.name} решает размяться:')
        self.LeftHand.make_move()
        self.RightHand.make_move()

# Создадим объект класса «Человек»
h1 = Human('Саша')
# Выполним метод «движение руками»
h1.move_hands()
```

Результат работы программы:

Саша решает размяться:
- левая рука совершает движение
- правая рука совершает движение

Таким образом, композиция классов – ещё один способ упорядочить код, используя объектно-ориентированную парадигму программирования.

6. Объектно-ориентированная декомпозиция

Способ управления сложными системами был известен еще в древности – «**разделяй и властвуй**». При проектировании сложной программной системы необходимо разделять ее на все меньшие и меньшие подсистемы, каждую из которых можно совершенствовать независимо. В этом случае для понимания любого уровня системы нам необходимо одновременно держать в уме информацию лишь о немногих ее частях. В самом деле, декомпозиция вызвана сложностью программирования системы, поскольку именно эта сложность вынуждает делить пространство состояний этой программной системы.

Алгоритмическая декомпозиция. Большинство специалистов, которые при создании программных систем используют технологию структурного программирования, и проектированию «сверху вниз», в частности, программисты воспринимают декомпозицию как обычное разделение алгоритмов на модули или процедуры, где каждый модуль системы выполняет один из этапов общего процесса.

Объектно-ориентированная декомпозиция. Здесь происходит разделение модели предметной области на элементы, выбрав в качестве критерия декомпозиции принадлежность этих элементов к различным абстракциям данной рассматриваемой предметной области. Прежде чем разделять задачу на шаги, необходимо определить какие объекты предметной области будут входить в разрабатываемую модель.

Хотя обе схемы решают одну и ту же задачу, но они делают это разными способами и при этом используют различные модели. Во второй случае мир представляется совокупностью автономных элементов предметной области, которые взаимодействуют друг с другом, чтобы обеспечить поведение системы, соответствующее более высокому уровню. Каждый элемент обладает своим собственным поведением, и каждый из них моделирует некоторый объект реального мира. С этой точки зрения объект является вполне осязаемой вещью, которая демонстрирует вполне определенное поведение конкретной части предметной области. Объекты что-то делают, и им можно послать сообщение, попросив их выполнить то-то и то-то. Так как данная декомпозиция основана на объектах, а не на алгоритмах, она называется **объектно-ориентированной декомпозицией**.

Разделение по алгоритмам концентрирует внимание на порядке *происходящих событий и вычислительном процессе*, а разделение по объектам придает особое значение элементам, которые являются *объектами*. Однако нельзя сконструировать сложную систему одновременно двумя способами. Разделение системы возможно либо по объектам, либо по алгоритмам, а затем, используя полученную структуру, попытаться рассмотреть проблему с другой точки зрения.

Объектная декомпозиция имеет несколько важных преимуществ перед алгоритмической декомпозицией:

Объектная декомпозиция уменьшает размер программных систем за счет повторного использования общих механизмов, что приводит к существенной экономии выразительных средств.

Объектно-ориентированные системы более гибки и проще эволюционируют со временем, потому что их схемы базируются на устойчивых промежуточных формах.

Действительно, объектная декомпозиция существенно снижает риск при создании сложной программной системы, так как она развивается из меньших систем, в которых мы уже уверены.

Более того, объектная декомпозиция помогает на ранних стадиях проектирования разобраться в сложной программной системе.

6.1. Применение декомпозиции при решении задач

Разберем декомпозицию на примере следующей задачи:

Реализуйте класс «Игра», в котором хранится список из объектов класса «Вопрос» и «Игровая сессия».

В «игру» можно добавлять вопросы с помощью текстового файла или ввода с клавиатуры.

«Игровая сессия» определяется именем игрока, которому задаются вопросы, при выборе верных ответов он получает баллы, они также фиксируются у сессии.

Выделим отдельно классы и опишем каждый в отдельности – какие будут поля и методы у каждого класса. Совершим декомпозицию поставленной задачи на следующие классы: «Игра», «Игровая сессия» и «Вопрос». При обозначении наименования классов, полей и методов также в круглых скобках указывайте написание этих сущностей на английском языке как они будут использоваться в коде.

Класс «Вопрос» (Question) имеет следующие поля:

- содержание вопроса (meaning);
- верный ответ (right_answer);
- сколько очков получает игрок (reward).

Класс «Вопрос» не имеет методов.

Класс «Игровая сессия» (GameSession) имеет следующие поля:

- имя игрока (player_name);
- количество баллов (points);
- количество верно отвеченных вопросов (right_answers);
- количество отвеченных вопросов (all_answers).

Класс «Игровая сессия» имеет следующие методы:

- получить процент верных ответов (get_right_percent).

Класс «Игра» (MainGame) имеет следующие поля:

- полный список вопросов (full_qlist);
- полный список игровых сессий (session_list);
- текущий список вопросов (current_qlist).

Класс «Игра» имеет следующие методы:

- считать файл банка вопросов (read_file);
- добавить вопрос в банк с клавиатуры (kboard_input);
- создать игровую сессию и начать игру (start_game);
- случайный список вопросов (get_random_qlist);

- вывести статистику по игровым сессиям (топ-10 игровых сессий) (show_top_ten).

7. Проект: Применение композиции классов

В ходе решения задачи для индивидуального выполнения требуется использовать различные пройденные главы по объектно-ориентированному программированию. При решении задачи нужно будет применить композицию классов, а также для каждого из классов реализовать конструктор и строковое представление объектов, также может понадобиться наследование, возможно нужно будет перегрузить некоторые операторы.

Студенту самостоятельно необходимо определить какие поля и методы должны быть реализованы у того или иного класса, если о них не сообщено явно в задании. Допускается реализация некоторых функций (если это не нарушает объектно-ориентированный подход), а также создание собственных дополнительных классов, если это упрощает работу программы.

Использование встроенных или сторонних библиотек без предварительной договоренности с преподавателем **запрещено**.

Для вариантов средней сложности (1-4) также прилагаются текстовые файлы, представляющие из себя данные для загрузки в программу, они доступны по ссылке:

https://drive.google.com/drive/folders/1XawM8AukWkBt9yJ682Vjkhsoap8aKuBq?usp=share_link

7.1. Варианты средней сложности

7.1.1. Вариант 1. «Игра и вопросы»

Реализуйте класс «Игра», в котором хранится список из объектов класса «Вопрос» и «Игровая сессия».

Класс «Вопрос» хранит в себе содержание вопроса, верный ответ, сколько баллов игрок получает при верном ответе, а также у некоторых вопросов есть комментарий, который необходимо вывести пользователю после его ответа.

При ответе на вопрос у пользователя будет выбор из 4 ответов. Каждый, кроме верного, берется от верных ответов на другие вопросы.

Класс «Игровая сессия» определяется именем игрока, которому задаются вопросы, при выборе верных ответов он получает баллы, они также фиксируются у сессии. В ходе игровой сессии игроку нужно ответить на 10 случайных вопросов.

Класс «Игра» является основным – он управляет игровыми сессиями, формирует вопросы для каждой сессии и проводит игру. Также можно посмотреть топ-5 лучших результатов игровых сессий, который должен храниться в долгосрочной памяти.

Общая база вопросов может как загружаться из текстовых файлов, так и заполняться с клавиатуры.

Сценарий взаимодействия с программой
Выберите действие: 1. загрузить вопросы из файла; 2. добавить вопрос вручную; 3. сохранить вопросы в отдельный файл; 4. начать игровую сессию; 5. посмотреть топ-5 лучших игроков; 0. завершить работу программы. Ответ: 1 Введите наименование файла: Вар 1, БД 1.txt Вопросы (10) добавлены в библиотеку
Выберите действие: 1. загрузить вопросы из файла; 2. добавить вопрос вручную; 3. сохранить вопросы в отдельный файл; 4. начать игровую сессию; 5. посмотреть топ-5 лучших игроков; 0. завершить работу программы. Ответ: 2 Введите содержание вопроса: Заглавный герой известного романа съел половину конфет в коробке, а вторую разложил так, чтобы казалось, что коробка заполнена вся. Что это за роман? Введите ответ: Защита Лужина Введите количество баллов за ответ:

30

Введите комментарий:

В шахматах доска изначально тоже заполнена лишь наполовину.

Вопрос добавлен в БД

Выберите действие:

1. загрузить вопросы из файла;
2. добавить вопрос вручную;
3. сохранить вопросы в отдельный файл;
4. начать игровую сессию;
5. посмотреть топ-5 лучших игроков;
0. завершить работу программы.

Ответ: **3**

Введите наименование файла:

Вар 1, БД 5.txt

Вопросы (11) выгружены в файл «Вар 1, БД 5.txt»

Выберите действие:

1. загрузить вопросы из файла;
2. добавить вопрос вручную;
3. сохранить вопросы в отдельный файл;
4. начать игровую сессию;
5. посмотреть топ-5 лучших игроков;
0. завершить работу программы.

Ответ: **4**

Введите имя пользователя:

Игрок_1

Начало игры!

Вопрос 1:

В произведении Чарльза Симика об ИКСАХ "едят улыбки, выплевывая зубы". Назовите ИКСЫ одним словом.

Варианты ответа:

1. Соседи
2. Городские трамваи
3. Арбузы
4. Орден Почетного Легиона

Ответ: **3**

Верный ответ!

Комментарий:

Долька арбуза напоминает улыбку. При поедании часто выплевывают семечки.

...

(далее идут остальные 9 вопросов в таком же формате)

...

Игра завершена.

Статистика:

Верно решено: 6/10

Количество баллов: 100

Выберите действие:

1. загрузить вопросы из файла;
2. добавить вопрос вручную;
3. сохранить вопросы в отдельный файл;
4. начать игровую сессию;
5. посмотреть топ-5 лучших игроков;
0. завершить работу программы.

Ответ: **5**

Топ-5 лучших игроков:

1 Место:

Игрок_1, верно решено (6/10), 100 баллов

7.1.2. Вариант 2. «Библиотека и книги»

Реализуйте класс «Библиотека», которая хранит в себе объекты класса «Книга».

Объект класса «Книга» определяется наименованием книги, указанием автора, жанром и количеством страниц.

Класс «Библиотека» хранит в себе книги и помогает пользователю ими распоряжаться:

- книги в библиотеку можно добавлять как с помощью текстового файла, так и с клавиатуры;
- список книг, добавленных в библиотеку, можно сохранять в отдельный файл;
- можно совершать поиск книг по одному из критериев – наименование книги, автор, жанр или количество страниц;
- можно совершить подборку книг по количеству свободных минут у пользователя.

Предполагая, что в среднем человек тратит на прочтение одной страницы книги 1,5 минуты, то у каждой книги можно определить затраты по времени. Алгоритм формирования списка книг по длительности прочтения таков – сначала идут самые большие по времени подходящие книги.

Например, если есть 4 книги с длительностью чтения [200, 235, 440, 22], а количество минут пользователя будет 800, то будут выбраны книги с длительностью 440 и 235, остальные уже не помещаются по длительности в подборку.

Если по критериям поиска не найдено ни одной книжки, программа должна сообщить «Книг, удовлетворяющих критерию <наименование критерия>=<содержание критерия> не было обнаружено».

Например, «Книг, удовлетворяющих критерию "кол-во страниц"=340 не было обнаружено».

При поиске книг по критериям «жанр» и «автор» пользователю будет предложено либо выбрать опцию из выпадающего списка, либо ввести самому. При любом поиске ручного ввода следует проверять содержится ли слово в данном поле, а не на равенство. Например, если бы был поиск содержится книг, у которых в наименовании есть «Python», то выдались бы все книги, содержащие это слово.

Ниже приведен пример тестирования работы программы с обзором всех основных функций:

Сценарий взаимодействия с программой
Выберите действие: 1. загрузить книги из файла; 2. добавить книгу вручную; 3. сохранить книги в отдельный файл; 4. найти книги по критериям; 5. подборка книг по времени; 0. завершить работу программы. Ответ: 1 Введите наименование файла: Вар 2, БД 2.txt Книги (26) добавлены в библиотеку
Выберите действие: 1. загрузить книги из файла; 2. добавить книгу вручную; 3. сохранить книги в отдельный файл; 4. найти книги по критериям; 5. подборка книг по времени; 0. завершить работу программы. Ответ: 2 Ручное добавление книги:

"Мы" - Е. Замятин - роман, антиутопия - 224 стр.

Книга добавлена в библиотеку

Выберите действие:

1. загрузить книги из файла;
2. добавить книгу вручную;
3. сохранить книги в отдельный файл;
4. найти книги по критериям;
5. подборка книг по времени;
0. завершить работу программы.

Ответ: **3**

Введите наименование нового файла:

Вар 2, БД 5

Текущий список книг (27) сохранен в файл "ЛР 2, БД 5.txt"

Выберите действие:

1. загрузить книги из файла;
2. добавить книгу вручную;
3. сохранить книги в отдельный файл;
4. найти книги по критериям;
5. подборка книг по времени;
0. завершить работу программы.

Ответ: **4**

Выберите критерий для поиска:

1. наименование книги;
2. автор;
3. жанр;
4. количество страниц.

Ответ: **3**

Выберите один из существующих жанров или задайте сами:

1. зарубежная компьютерная литература;
2. программирование;
3. отечественная компьютерная литература;
4. роман;
5. антиутопия.

Ответ: **3**

Книги с жанром «отечественная компьютерная литература»:

1. Django 3.0. Практика создания веб-сайтов на Python (Профессиональное программирование);
2. Мультиагентное обучение с подкреплением. Изд.2.

Ответ: **1**

Вывод информации о книге:

Наименование: Django 3.0. Практика создания веб-сайтов на Python (Профессиональное программирование)

Авторы: Дронов В.А.

Жанры: отечественная компьютерная литература, программирование

Кол-во страниц: 497

Выберите действие:

1. загрузить книги из файла;
2. добавить книгу вручную;
3. сохранить книги в отдельный файл;
4. найти книги по критериям;
5. подборка книг по времени;
0. завершить работу программы.

Ответ: 5

Выберите тип подборки:

1. подборка без фильтров;
2. подборка по автору;
3. подборка по жанру.

Ответ: 1

Введите общую длительность подборки в минутах:

1355

Подборка книг без фильтров:

1. Научное программирование на Python (970 мин.);
2. Обработка естественного языка Python и Spacy (384 мин.).

7.1.3. *Вариант 3.* «Музыкальный проигрыватель»

В ходе данной работы требуется смоделировать работу классов «Проигрыватель» (MusicPlayer), «Плейлист» (Playlist) и «Музыка» (Music).

У класса «Музыка» есть данные о наименовании композиции, исполнителе, жанре музыки и длительности проигрывания.

Класс «Плейлист» хранит объекты класса «музыка».

С помощью класса «Проигрыватель» можно составлять различные плейлисты:

- по ключевым словам в названии;
- по исполнителю;
- по жанру музыки.

Также плейлист можно создать по времени проигрывания - сначала будут идти композиции с самым долгим временем проигрывания.

Например, если есть 4 композиции с длительностью проигрывания [5:30, 1:13, 2:30, 4:30], а количество минут пользователя будет 10, то будут выбраны композиции с длительностью 5:30 и 4:30, остальные уже не помещаются по длительности в плейлист.

Сценарий взаимодействия с программой

Выберите действие:

1. загрузить список музыки из файла;
2. добавить музыку вручную;
3. сохранить музыку в отдельный файл;
4. создать плейлист по критериям;
5. создать плейлист по времени;
0. завершить работу программы.

Ответ: **1**

Введите наименование файла:

Вар 3, БД 1.txt

Музыка (34) добавлена в библиотеку

Выберите действие:

1. загрузить список музыки из файла;
2. добавить музыку вручную;
3. сохранить музыку в отдельный файл;
4. создать плейлист по критериям;
5. создать плейлист по времени;
0. завершить работу программы.

Ответ: **2**

Ручное добавление композиции:

"Stay With Me" – Miki Matsubara – сити-поп – 5:12

Композиция добавлена

Выберите действие:

1. загрузить список музыки из файла;
2. добавить музыку вручную;
3. сохранить музыку в отдельный файл;
4. создать плейлист по критериям;
5. создать плейлист по времени;
0. завершить работу программы.

Ответ: **3**

Введите наименование нового файла:

Вар 3, БД 5

Текущий список композиций (35) сохранен в файл "Вар 3, БД 5.txt"

Выберите действие:

1. загрузить список музыки из файла;
2. добавить музыку вручную;
3. сохранить музыку в отдельный файл;
4. создать плейлист по критериям;
5. создать плейлист по времени;

0. завершить работу программы.

Ответ: **4**

Выберите критерий для формирования плейлиста:

1. наименование композиции;
2. автор;
3. жанр.

Ответ: **3**

Выберите «жанр» из списка или введите вручную:

1. рок;
2. поп-рок;
3. хаус;
4. электроника;
5. J-поп;
6. рок;
7. хип-хоп;
8. поп;
9. фанк;
10. инди;
11. панк-рок;
12. фолк-рок;
13. авторская песня;
14. метал;
15. хорус;
16. нью-бит;
17. альт-рок;
18. сити-поп.

Ответ: **4**

Плейлист с жанр=электроника:

1. Night Moves;
2. You make me;
3. Touch;
4. My Way;
5. Forces.

Выберите действие:

1. загрузить список музыки из файла;
2. добавить музыку вручную;
3. сохранить музыку в отдельный файл;
4. создать плейлист по критериям;
5. создать плейлист по времени;
0. завершить работу программы.

Ответ: **5**

Введите количество минут: **10**

Плейлист с время=10 минут:

1. Heartbreaker (5:28);

7.1.4. Вариант 4. «Управление школой»

В ходе данной работы требуется смоделировать классы «Школа» (School), «Класс» (SchoolClass), «Школьник» (Student).

Класс «Школьник» хранит в себе ФИО, год рождения и список оценок по предметам.

Каждый «Класс» хранит в себе идентификатор, ФИО классного руководителя и список учеников.

Класс «Школа» управляет школьными классами и обладает следующим функционалом:

- загрузка данных о школе из файла;
- задать данные о школе вручную;
- сохранение данных о школе в файл;
- вывод лучшего ученика в каждом классе по общему среднему баллу (если высший средний балл одинаков у нескольких учеников, тогда вывести всех);
- вывод лучших учеников в каждом подклассе (класс 7, а подклассами будут 7А, 7Б и т.д..) по среднему баллу одного из предметов.

Сценарий взаимодействия с программой

Выберите действие:

1. загрузить данных о школе из файла;
2. задать данные о школе вручную;
3. сохранить данные о школе в отдельный файл;
4. вывод лучшего ученика в каждом классе по всем предметам;
5. вывод лучшего ученика в каждом подклассе по предмету;
0. завершить работу программы.

Ответ: **1**

Введите наименование файла:

Вар 4, БД 1.txt

Данные о школе загружены:

2 класса;

7 учеников.

Выберите действие:

1. загрузить данных о школе из файла;

2. задать данные о школе вручную;
3. сохранить данные о школе в отдельный файл;
4. вывод лучшего ученика в каждом классе по всем предметам;
5. вывод лучшего ученика в каждом подклассе по предмету;
0. завершить работу программы.

Ответ: **2**

Выберите что необходимо ввести:

1. наименование школы;
2. ФИО директора;
3. создать новый класс;
4. добавить ученика с оценками в класс;
0. вернуться назад.

Ответ: **3**

Введите номер и литеру класса:

8А

Введите ФИО классного руководителя:

Абрикосов А.А.

Выберите что необходимо ввести:

1. наименование школы;
2. ФИО директора;
3. создать новый класс;
4. добавить ученика с оценками в класс;
0. вернуться назад.

Ответ: **4**

Введите класс, ФИО ученика и его оценки:

7А; Виноградова А.А.: Химия: 5, 5, 4, 4, 5; физика: 4, 2, 2, 3, 2, 3, 2, 4, 4; Русский язык: 4, 5, 3, 5, 5;

Выберите действие:

1. загрузить данных о школе из файла;
2. задать данные о школе вручную;
3. сохранить данные о школе в отдельный файл;
4. вывод лучшего ученика в каждом классе по всем предметам;
5. вывод лучшего ученика в каждом подклассе по предмету;
0. завершить работу программы.

Ответ: **5**

Введите число класса:

7

В 7 классе преподаются следующие предметы:

1. Алгебра;
2. Геометрия;
3. Русский язык.

Выберите один из предметов: **3**

Лучший ученик по Русский язык в 7А – Иванов М.М. (4,6);

Лучший ученик по Русский язык в 7Б – Одерышев Н.О. (4,4) .

7.2. Варианты повышенной сложности

7.2.1. Вариант 5. «Предмет и Собиратель»

В ходе данной работы требуется реализовать классы «Предмет» и «Собиратель».

Задача собирателя – находить в лесу некоторые предметы и из них собирать другие. Каждый найденный или созданный предмет является объектом класса «Предмет» (Item). Требуется реализовать систему крафта новых предметов из тех, что имеются у собирателя. Новая вещь должна добавиться в рюкзак.

В лесу собиратель может найти следующие предметы:

Редкость и шанс обнаружения	Наименование предмета
Обычная, 60%	Мята
	Палка
	Голубика
	Трава
	Вода
	Кремень
	Камень
	Рыба
	Дерево
Редкая, 30%	Перо
	Аметист
	Кроличья лапка
	Уголь

Легендарная, 10%	Перо жар-птицы
	Хризолит
	Шар откровений

Процесс создания предметов можно описать с помощью операторов сложения, вычитания, умножения и деления.

Сложение ($a+b=c$) означает, что оба предмета (a , b) будут затрачены на создание чего-то нового (c).

Вычитание ($a-b=c$) означает, что уменьшаемое (a) будет затрачено с помощью вычитаемого (b) для получения нового предмета (c). Вычитаемое (b) при этом затрачено не будет.

Умножение ($a*b=c$), где a - предмет, b – число повторений предмета в рецепте. Например, «Кремень*2+Ветка» равняется по смыслу «Кремень+Кремень+Ветка».

В отличие от задания на перегрузку операторов «19.3 – 4 Верстак с рецептами», данное задание не обязательно реализовывать через перегрузку операторов сложения, вычитания и умножения.

Операции сложения и вычитания обладают свойством **коммутативности**. Коммутативность означает возможность менять местами элементы операции без изменения результата выполнения, т.е. $a+b = b+a$.

Рецепт	Предмет
Кремень+Палка	Нож
Кремень*2+Дерево*3	Верстак
Кремень+Дерево-Верстак	Лопата
Кремень*2+Палка+Трава*3	Костер
Костер+Вода	Уголь*10
Мята*3+Голубика*2+Вода-Костер	Зелье лечения
Кроличья лапка-Нож+Вода-Костер	Зелье удачи
Рыба-Нож-Костер	Жареная рыба
Аметист*3+Хризолит*2+Железо*5+Палка-Верстак	Магический посох

Перо*6+Перо Жар-птицы*2+Трава*10- Верстак	Магическая панамка
--	--------------------

Собранные и созданные вещи собиратель может продавать на рынке, а также покупать новые, некоторые из которых в лесу не обнаружить. В лавке будет 4 случайных предмета из списка в количестве от 1 до 10. Предметы в лавке обновляются, когда игрок выбирает команду «Пойти на рынок». Во время нахождения на рынке повторно случайно поменяться ассортимент не может. Проданные игроком предметы будут появляться в разделе «Купить предметы». Собиратель может продать предметы только за 75% рыночной стоимости. Ниже приведена таблица стоимости предметов:

Предмет	Стоимость
Мяга	10
Палка	
Голубика	
Трава	
Кремень	15
Камень	
Рыба	
Дерево	
Перо	
Уголь	
Жареная рыба	25
Железо	
Аметист	50
Кроличья лапка	75
Перо жар-птицы	100
Хризолит	150
Зелье лечения	200
Зелье удачи	450

Шар откровений	500
Магическая панамка	1500
Магический посох	3000

Взаимодействие пользователя с программой осуществляется с помощью следующих команд:

1. поиск одного предмета;
2. поиск десяти предметов за раз;
3. посмотреть список рецептов;
4. создать предмет. Должны выводиться только доступные для создания рецепты;
5. пойти на рынок;
0. завершение работы программы.

Ниже приведена таблица тестирования одного из возможных сценариев работы программы:

Обычное взаимодействие с программой	
Выберите действие: 1. поиск одного предмета; 2. поиск десяти предметов за раз; 3. посмотреть список рецептов; 4. создать предмет; 5. пойти на рынок; 0. завершить работу программы. Ответ: 1 Собиратель находит предмет – Мята (Обычная)	
Выберите действие: 1. поиск одного предмета; 2. поиск десяти предметов за раз; 3. посмотреть список рецептов; 4. создать предмет; 5. пойти на рынок; 0. завершить работу программы. Ответ: 2 Собиратель находит следующие предметы: 1. Мята (Обычная); 2. Вода (Обычная); 3. Вода (Обычная); 4. Трава (Обычная); 5. Рыба (Обычная);	

6. Дерево (Обычная);
7. Аметист (Редкая);
8. Хризолит (Легендарная);
9. Кремень (Обычная);
10. Палка (Обычная).

Выберите действие:

1. поиск одного предмета;
2. поиск десяти предметов за раз;
3. посмотреть список рецептов;
4. создать предмет;
5. пойти на рынок;
0. завершить работу программы.

Ответ: **3**

Собиратель знает 10 рецептов:

1. Кремень+Палка = Нож;
2. Кремень*2+Дерево*3 = Верстак;
- ... (и так далее)

Выберите действие:

1. поиск одного предмета;
2. поиск десяти предметов за раз;
3. посмотреть список рецептов;
4. создать предмет;
5. пойти на рынок;
0. завершить работу программы.

Ответ: **4**

Выберите способ сборки:

1. создать вручную;
2. выбор из предложенных.

Ответ: **1**

Введите рецепт:

Палка+Кремень

Собранный предмет: Нож

Выберите действие:

1. поиск одного предмета;
2. поиск десяти предметов за раз;
3. посмотреть список рецептов;
4. создать предмет;
5. пойти на рынок;
0. завершить работу программы.

Ответ: **4**

Выберите способ сборки:

1. создать вручную;
2. выбор из предложенных.

Ответ: **2**

Собиратель может собрать следующие предметы (1):

1. Кремень+Палка=Нож.

Выберите рецепт: **1**

Собран Нож, затрачены Кремень (1), Палка (1)

Выберите действие:

1. поиск одного предмета;
2. поиск десяти предметов за раз;
3. посмотреть список рецептов;
4. создать предмет;
5. пойти на рынок;
0. завершить работу программы.

Ответ: **5**

Собиратель приходит на рынок:

1. купить предметы;
2. продать предметы;
0. покинуть рынок.

Ответ: **1**

У собирателя 510 зол.

Продавец показывает следующие товары:

1. Шар откровений (3) – 500 зол.;
2. Кремень (10) – 15 зол.;
3. Мята (7) – 10 зол.;
4. Голубика (5) – 10 зол.;
0. отмена покупки.

Ответ: **1**

Собиратель покупает Шар откровений, теперь у него 10 зол..

Собиратель приходит на рынок:

1. купить предметы;
2. продать предметы;
0. покинуть рынок.

Ответ: **2**

Собиратель хочет продать:

1. Мята (3) – 7.5 зол.;
2. Трава (1) – 7.5 зол.;
3. Рыба (1) – 11.25 зол.;
4. Дерево (1) – 11.25 зол.;
5. Аметист (1) – 37.5 зол.;
6. Хризолит (1) – 112.5 зол.;
7. Шар откровений (1) – 375 зол.;
0. отмена продажи.

Ответ: 1

Собиратель продает 1 Мята, теперь у него 17.5 зол..

Собиратель приходит на рынок:

1. купить предметы;
2. продать предметы;
0. покинуть рынок.

Ответ: 0

Выберите действие:

1. поиск одного предмета;
2. поиск десяти предметов за раз;
3. посмотреть список рецептов;
4. создать предмет;
5. пойти на рынок;
0. завершить работу программы.

Ответ: 0

7.2.2. Вариант 6. «Железный дровосек»

В ходе данной задачи требуется смоделировать четыре класса – «железный дровосек», «волшебная масленка», «дерево» и «лес», которые взаимодействуют между собой.

Реализуйте класс «железный дровосек». Железный дровосек может искать деревья и их срубить. На сруб любого из деревьев он тратит масло, которое он может пополнять из масленки (максимальное значение масла у дровосека 100 ед.). Если масло опустится до 0 значения, дровосек заржавеет, и игра закончится.

Задача дровосека срубить как можно больше деревьев, не забывая смазывать себя время-от-времени.

Ниже приведены характеристики работы дровосека:

Тип дерева	Кол-во ударов топором для сруба	Расход масла за удар, %	Приносит дерева после сруба	Общий расход масла
Молодая сосна	1	10	2	10
Ель	2	7.5	3	15
Кедр	3	6	4	18
Яблоня	3	5	9	15

Тополь	4	4	5	16
Липа желаний	5	6	15	30
Клен	5	7	10	35
Щедрый дуб	6	8	25	48

Класс «волшебная масленка», хранит в себе 300 ед. масла. Масленку можно использовать, чтобы пополнять количество масла дровосека. Для пополнения запаса масла в самой масленке используется добытое дерево. Для пополнения 50 масла нужно затратить 15 дерева.

Класс «дерево» представляет из себя каждое найденное дровосеком дерево. У дерева есть показатель того, сколько осталось ударов до того, как оно упадет (`hits_to_fall`), а также показатель того сколько с него досок будет получено (`reward`).

Класс «лес» является основным для дерева. Дровосек находит деревья в лесу, вызывая у объекта класса «лес» метод «новое дерево» (`new_tree`).

Вероятности нахождения каждого из деревьев делятся на категории – обычное дерево, редкое и уникальное. При выпадении одной из категорий шанс выпадения любого из деревьев одинаково.

Редкость	Шанс выпадения, %	Тип дерева
Обычная	50	Молодая сосна
		Ель
		Кедр
		Тополь
Редкая	35	Яблоня
		Клен
Уникальная	15	Липа желаний
		Щедрый дуб

Ниже представлены примеры работы программы:

Пример работы программы – 1

Обычный сценарий работы дровосека

Дровосек начинает работу в лесу

Выберите действие:

- 1 – поиск дерева;
- 2 – использовать масленку;
- 3 – пополнить масленку;
- 4 – статистика;
- 5 – вернуться домой.

Ответ: 1

Дровосек находит дерево – Молодая сосна!

Выберите действие:

- 1 – срубить дерево;
- 2 – использовать масленку;
- 3 – пополнить масленку;
- 4 – статистика;
- 5 – вернуться домой.

Ответ: 1

Дровосек делает удар по Молодая сосна (1/1)!

Масло дровосека (90/100)

Дерево упало, дровосек получает 2 доски.

Выберите действие:

- 1 – поиск дерева;
- 2 – использовать масленку;
- 3 – пополнить масленку;
- 4 – статистика;
- 5 – вернуться домой.

Ответ: 2

Дровосек достает масленку и смазывает себя

Масло дровосека (100/100)

Масло в масленке (290/300)

Выберите действие:

- 1 – поиск дерева;
- 2 – использовать масленку;
- 3 – пополнить масленку;
- 4 – статистика;
- 5 – вернуться домой.

Ответ: 5

После тяжелого трудового дня дровосек возвращается домой.

Статистика:

Масло дровосека: 100/100

Масло масленки: 290/300

Израсходовано всего масла: 10
Доски в инвентаре: 2
Затрачено досок: 0
Срублено деревьев: 1

Пример работы программы – 2

Сценарий работы масленки

Дровосек начинает работу в лесу

...

(проходит время, у дровосека осталось только 15 ед. масла)

...

Выберите действие:

- 1 – поиск дерева;
- 2 – использовать масленку;
- 3 – пополнить масленку;
- 4 – статистика;
- 5 – вернуться домой.

Ответ: 4

Статистика:

Масло дровосека: 15/100

Масло масленки: 80/300

Израсходовано всего масла: 305

Доски в инвентаре: 82

Затрачено досок: 0

Срублено деревьев: 22

Ответ: 1

Дровосек находит дерево – Щедрый дуб!

Выберите действие:

- 1 – срубить дерево;
- 2 – использовать масленку;
- 3 – пополнить масленку;
- 4 – статистика;
- 5 – вернуться домой.

Ответ: 1

Дровосек делает удар по Щедрый дуб (1/6)!

Масло дровосека (7/100)

Выберите действие:

- 1 – срубить дерево;
- 2 – использовать масленку;
- 3 – пополнить масленку;
- 4 – статистика;

5 – вернуться домой.

Ответ: 2

Дровосек достает масленку и смазывает себя

Масло дровосека (95/100)

Масло в масленке (0/300)

Выберите действие:

- 1 – срубить дерево;
- 2 – использовать масленку;
- 3 – пополнить масленку;
- 4 – статистика;
- 5 – вернуться домой.

Ответ: 3

Дровосек тратит 15 дерева для пополнения масленки (50/300)

Выберите действие:

- 1 – срубить дерево;
- 2 – использовать масленку;
- 3 – пополнить масленку;
- 4 – статистика;
- 5 – вернуться домой.

Ответ: 5

После тяжелого трудового дня дровосек возвращается домой.

Статистика:

Масло дровосека: 95/100

Масло масленки: 50/300

Израсходовано всего масла: 305

Доски в инвентаре: 67

Затрачено досок: 15

Срублено деревьев: 22

Пример работы программы – 3

Сценарий полного расхода масла у дровосека

...

(проходит время, у дровосека осталось только 15 ед. масла)

...

Выберите действие:

- 1 – поиск дерева;
- 2 – использовать масленку;
- 3 – пополнить масленку;
- 4 – статистика;
- 5 – вернуться домой.

Ответ: 4

Статистика:

Масло дровосека: 15/100

Масло масленки: 0/300

Израсходовано всего масла: 305

Доски в инвентаре: 82

Затрачено досок: 75

Срублено деревьев: 40

Ответ: 1

Дровосек находит дерево – Щедрый дуб!

Выберите действие:

1 – срубить дерево;

2 – использовать масленку;

3 – пополнить масленку;

4 – статистика;

5 – вернуться домой.

Ответ: 1

Дровосек делает удар по Щедрый дуб (1/6)!

Масло дровосека (7/100)

Выберите действие:

1 – поиск дерева;

2 – использовать масленку;

3 – пополнить масленку;

4 – статистика;

5 – вернуться домой.

Ответ: 1

Дровосек делает удар по Щедрый дуб (1/6)!

Масло дровосека (0/100)

О нет, масло закончилось!

Дровосек заржавел, игра окончена.

Статистика:

Масло дровосека: 0/100

Масло масленки: 0/300

Израсходовано всего масла: 320

Доски в инвентаре: 82

Затрачено досок: 75

Срублено деревьев: 40

7.2.3. Вариант 7. «Продвинутая оранжерея»

В ходе данной работы требуется смоделировать классы «Оранжерея» (Greenhouse), «Время» (Time), «Растение» (Plant), «Яблоня» (AppleTree) и «Орхидея» (Orchid).

Класс Plant

Класс «Растение» является базовым по отношению к «Яблоне» и «Орхидее». Он имеет показатели воды и освещенности. Растения можно поливать, а также повышать их освещенность.

Plant - метод to_grow:

Раз в N-ное количество минут растение растет, расходуя определенное количество воды и света *одновременно*.

Тип растения	Рост (1 раз во сколько минут)	Изменение роста, %	Тратит влаги	Тратит света
Яблоня	40	2	10	8
Орхидея	20	5	5	10

У каждой реализации класса «Растение» есть свои максимальные и минимальные значения влаги и света. При нарушении этих показателей у растения будет ухудшаться здоровье каждые 30 минут – есть четыре стадии здоровья растения - здоровое, легкая болезнь, тяжелая болезнь, мертво.

Если же за следующие 90 минут растение спокойно растет (показатели света и влаги находятся в диапазоне min-max), то оно оправится и перейдет на предыдущую стадию, но не из стадии «мертво».

Тип растения	Min-max влага	Min-max свет
Яблоня	10-90	10-60
Орхидея	15-55	30-80

Класс AppleTree

Класс «Яблоня» является плодоносящим деревом, а потому по достижении роста в 80% дерево начинает давать плоды раз в день – от 10 до 20 яблок. Яблоки автоматически собираются и начинают храниться на складах в оранжерее. Затем дерево будет плодоносить 10 дней подряд, после этого оно переходит в состояние «Мертво».

Класс Orchid

Класс «Орхидея» при достижении 90% роста раскрывает бутон, лепестки которого могут быть одного из цветов – красные, желтые, сиреневые или же белые лепестки. Цвет бутона определяется случайным образом. Через 2 дня после наступления прогресса роста в 100% растение перейдет в статус «Мертво».

Класс Greenhouse

Класс «Оранжерея» является основным – в нём растут деревья и цветы.

Greenhouse – поле time_exists:

У оранжереи есть показатель времени (поле *time_exists*) – сколько она существует и занимается выращиванием растений.

По умолчанию время в оранжерее равняется 00:00.

Greenhouse - метод tend_to_plants:

Оранжерея является *продвинутой*, а потому сама принимает решения по поливу и освещению каждого растения в отдельности раз в час. Если она замечает, что тот или иной показатель вышел за минимальное значение, то она польет или осветит растение на значение максимально допустимое минус один.

Например, если у Орхидеи в тот или иной час значение света N, такое что $N \leq 30$, то Оранжерея осветит орхидею так, чтобы значение стало $(80-1) = 79$.

Класс Time

Класс «Время» отвечает за управление временем, проходящим в оранжерее. С ходом времени растут растения в оранжерее. Время можно складывать, например $19:30 + 1:30 = 21:00$, чтобы отобразить, что прошло

полтора часа. Также время можно вычитать, чтобы определять между ним разницу, например: 21:00-19:30 = 1:30.

С оранжереей можно взаимодействовать следующими командами:

- 1 <заданное время> - пропустить время;
- 2 <тип растения> <название растения> <значение воды> <значение света> - посадить растение в оранжерею;
- 3 <название растения> <значение для полива> - полить определенное растение;
- 4 <название растения> <значение света> - осветить определенное растение;
- 5 <название растения> - выкопать определенное растение;
- 6 <название растения> - статистика по определенному растению;
- 7 – общая статистика;
- 0 – завершить работу программы.

Ниже представлена таблица тестирования работы программы:

Ввод пользователя	Результат работы программы
Сессия 1 – оранжерея с одной орхидеей	
2 Орхидея Цв1 30 50	Посажена Орхидея с наименованием «Цв1»!
6 Цв1	Статистика по «Цв1»: Тип: Орхидея Вода: 15<30<55 Свет: 30<50<80 Рост: 0% Состояние: Здоровое Посажено в 00:00 Лепестки отсутствуют
1 00:20	Время изменилось с 00:00 на 00:20 – прошло 20 минут. Цв1 – Рост (5/100)
6 Цв1	Статистика по «Цв1»: Тип: Орхидея Вода: 15<25<55 Свет: 30<40<80 Рост: 5% Состояние: Здоровое Посажено в 00:00

	Лепестки отсутствуют
1 00:40	Время изменилось с 00:20 на 01:00 – прошло 40 минут. Умная Оранжерея осветила Цв1, чтобы избежать критического показателя Свет 30 -> Свет 79.
6 Цв1	Цв1 – Рост(15/100) Статистика по «Цв1»: Тип: Орхидея Вода: 15<20<55 Свет: 30<79<80 Рост: 15% Посажено в 00:00 Лепестки отсутствуют
1 04:20	Время изменилось с 01:00 на 05:20 – прошло 4 часа и 20 минут. У Цв1 распускается бутон с красными лепестками!
6 Цв1	Статистика по «Цв1»: Тип: Орхидея Вода: 15<49<55 Свет: 30<69<80 Рост: 80% Состояние: Здоровое Посажено в 00:00 Имеет красные лепестки
1 01:20	Время изменилось с 05:20 на 06:40 – прошло 1 час и 20 минут. Цв1 достигает роста в 100%
1 48:00	Время изменилось с 06:40 на 54:40 – прошло 48 часов. Цв1 – естественная смерть.
6 Цв1	Статистика по «Цв1»: Тип: Орхидея Вода: 15<54<55 Свет: 30<79<80 Рост: 100% Состояние: Мертво Посажено в 00:00 Имеет красные лепестки

5 Цв1	Растение «Цв1» было вынесено из оранжереи
6 Цв1	Растение с именем «Цв1» отсутствует в оранжерее!
0	
Сессия 2 – оранжерея с одной яблоней	
2 Яблоня Дерев01 40 45	Посажена Яблоня с наименованием «Дерев01»!
1 00:40	Время изменилось с 00:00 на 00:40 – прошло 40 минут. Дерев01 – Рост(2/100)
6 Дерев01	Статистика по «Дерев01»: Тип: Яблоня Вода: 10<30<90 Свет: 10<37<60 Рост: 2% Состояние: Здоровое Посажено в 00:00 Не плодоносит
1 00:80	Время изменилось с 00:40 на 02:00 – прошло 80 минут. Умная Оранжерея полила Дерев01, чтобы избежать критического показателя Вода 10 -> Вода 89.
6 Дерев01	Статистика по «Дерев01»: Тип: Яблоня Вода: 10<89<90 Свет: 10<21<60 Рост: 6% Состояние: Здоровое Посажено в 00:00 Не плодоносит

АРХИВ

1. На композицию классов также подошла бы программа для управления магазином – хранит продукты с категорией товара, ценой и прочим.

2. Композиция – «21» - есть игровой стол, 4 игрока. Каждый из них получает по две карты из колоды. Есть на середине карты на середине стола.

3. Одной из более серьезных задач может быть моделирование экосистемы леса. Есть растения – деревья, кусты и трава, т.е. есть наследование. Есть животные – медведи, волки и зайцы. Есть река – в ней водятся пару видов рыб.

У каждого животного есть показатели возраста, голода и жажды. Утолять жажду можно с помощью воды в реке, мясной/рыбной пищи и ягод.

Волк охотится за зайцами.

Зайцы едят траву.

Допиши дальше

Можно сделать командное задание, где каждый будет отвечать за свою часть леса, но потом можно будет это собрать в единое целое.

29. Создать класс `Date` для работы с датами в формате «год.месяц.день». Дата представляется структурой с тремя полями типа `unsigned int`: для года, месяца и дня. Класс должен включать не менее трех функций инициализации: числами, строкой вида «год.месяц.день» (например, «2004.08.31») и датой. Обязательными операциями являются: вычисление даты через заданное количество дней, вычитание заданного количества дней из даты, определение високосности года, присвоение и получение отдельных частей (год, месяц, день), сравнение дат (равно, до, после), вычисление количества дней между датами.

1. Создать класс `BitString` для работы с битовыми строками не более чем из 100 бит. Битовая строка должна быть представлена массивом типа `unsigned char`, каждый элемент которого принимает значение 0 или 1. Реальный размер массива задается как аргумент конструктора инициализации. Должны быть реализованы все традиционные операции для работы с битовыми строками: `and`, `or`, `xor`, `not`. Реализовать сдвиг влево и сдвиг вправо на заданное количество битов.

19. Создать класс `Octal` для работы с беззнаковыми целыми восьмеричными числами, используя для представления числа массив из 100 элементов типа `unsigned char`, каждый элемент которого является восьмеричной цифрой. Младшая цифра имеет меньший индекс (единицы — в нулевом элементе массива). Реальный размер массива задается как аргумент конструктора инициализации. Реализовать арифметические операции, аналогичные встроенным для целых в C++, и операции сравнения.

13. Товарный чек содержит список товаров, купленных покупателем в магазине. Один элемент списка представляет собой пару: товар-сумма. Товар — это класс `Goods` с полями кода и наименования товара, цены за единицу товара, количества покупаемых единиц товара. В классе должны быть реализованы методы доступа к полям для получения и изменения информации, а также метод вычисления суммы оплаты за товар. Для моделирования товарного чека реализовать класс `Receipt`, полями которого являются номер товарного чека, дата и время его создания, список покупаемых товаров. В классе `Receipt` реализовать методы добавления, изменения и удаления записи о покупаемом товаре, метод поиска информации об определенном виде товара по его коду, а также метод подсчета общей суммы, на которую были осуществлены покупки. Методы добавления и изменения принимают в качестве аргумента объект класса `Goods`. Метод поиска возвращает объект класса `Goods` в качестве результата.

39. Создать класс `String` (см. задание 2.20).

20. Создать класс `String` для работы со строками, аналогичными строкам Turbo Pascal (строка представляется как массив 255 байт, длина — в первом байте). Максимальный размер строки должен задаваться. Обязательно должны быть реализованы: определение длины строки, поиск подстроки в строке, удаление подстроки из строки, вставка подстроки в строку, сцепление двух строк.

38. Создать класс WorkTeacher (см. задание 2.18).

18. Нагрузка преподавателя за учебный год представляет собой список дисциплин, преподаваемых им в течение года. Одна дисциплина представляется информационной структурой с полями: название дисциплины, семестр проведения, количество студентов, количество часов аудиторных лекций, количество аудиторных часов практики, вид контроля (зачет или экзамен). Реализовать класс WorkTeacher, моделирующий бланк назначенной преподавателю нагрузки. Класс содержит фамилию преподавателя, дату утверждения, список преподаваемых дисциплин, объем полной нагрузки в часах и в ставках. Дисциплины в списке не должны повторяться. Объем в ставках вычисляется как частное от деления объема в часах на среднюю годовую ставку, одинаковую для всех преподавателей кафедры. Элемент списка преподаваемых дисциплин содержит дисциплину, количество часов, выделяемых на зачет (0,35 ч на одного студента) или экзамен (0,5 ч на студента), сумму часов по дисциплине. Реализовать добавление и удаление дисциплин; вычисление суммарной нагрузки в часах и ставках. Должен осуществляться контроль за превышением нагрузки более полутора ставок.

2 СЕМЕСТР

Тема 10. Регулярные выражения (5/5)

Составление регулярного выражения, запись его в коде программы, поиск и сопоставление с регулярным выражением, другие возможности стандартной библиотеки, использующие регулярные выражения.

Описать Генераторы списков и не только

8. Составление формальных грамматик

8.1. Формальные и естественные языки

На **естественных языках** говорят люди; это, например, английский, испанский и французский языки. Они не были спроектированы людьми (хотя люди пытаются соблюдать в них некий порядок); они развивались естественно.

Формальные языки разрабатываются людьми для определенных целей. Например, математические символы представляют собой формальный язык, который особенно хорош для обозначения отношений между числами и символами. Химики используют формальный язык для представления химической структуры молекул. И самое важное: Языки программирования — это формальные языки, предназначенные для выражения вычислений.

Формальные языки, как правило, имеют строгие **синтаксические правила**, которым подчиняется структура кода. Например, в математике утверждение $3 + 3 = 6$ имеет правильный синтаксис, а $3 + = 3\$6$ — нет. В химии H_2O — синтаксически правильная формула, а $2Zz$ — нет.

Параметр сравнения	Естественные языки	Искусственные языки
Эволюция	Сформировались в результате развития общества	Созданы людьми для достижения определенной цели
Использование	Для общения и в быту	Для обмена информацией в специальных областях знаний
Неточности	Часто встречаются слова с неточным и неясным содержанием	Не допускаются слова с неточным и неясным содержанием

Зависимость от контекста	Значения отдельных слов и предложений зависят не только от них самих, но и от их окружающего контекста	Значения отдельных слов и предложений не зависят от контекста
Синонимы	Встречаются синонимы — разные слова, имеющие одинаковый смысл	Как правило, синонимов нет
Омонимы	Встречаются омонимы — слова, имеющие несколько значений	Омонимов нет
Строгие правила образования предложений	Нет	Используются только строгие правила построения предложений
Исключения	Для многих правил существуют исключения	Нет исключений из правил

Выбор языка может быть очень важен. Пример – римские числа, отсутствие нуля, проблемы с делением.

8.2. Синтаксический разбор

Синтаксические правила бывают двух видов:

- относящиеся к формированию основных элементов языка (лексем или же токенов);
- правила формирования из лексем предложений.

Лексемы (токены) — основные элементы языка, такие как слова, числа и химические элементы. Одна из проблем с « $3 + = 3\$6$ » заключается в том, что это недопустимый в математике токен. Аналогично, « $2Zz$ » недопустимо, так как нет элемента с сокращением « Zz ».

Второй тип правил синтаксиса относится к способу объединения токенов. Операция « $3 + = 3$ » в математике недопустима, поскольку хотя символы $+$ и $=$ являются допустимыми токенами, их нельзя использовать один сразу за другим. Точно так же в химической формуле нижний индекс указывается после имени элемента, а не перед.

Предложение может быть правильно структурировано, но содержать **недопустимые токены**, например:

*Это хорошо структуриров@нное предложение с недопустимым т*кенами.*

И наоборот, предложение может быть **неверно структурировано**, несмотря на допустимые токены:

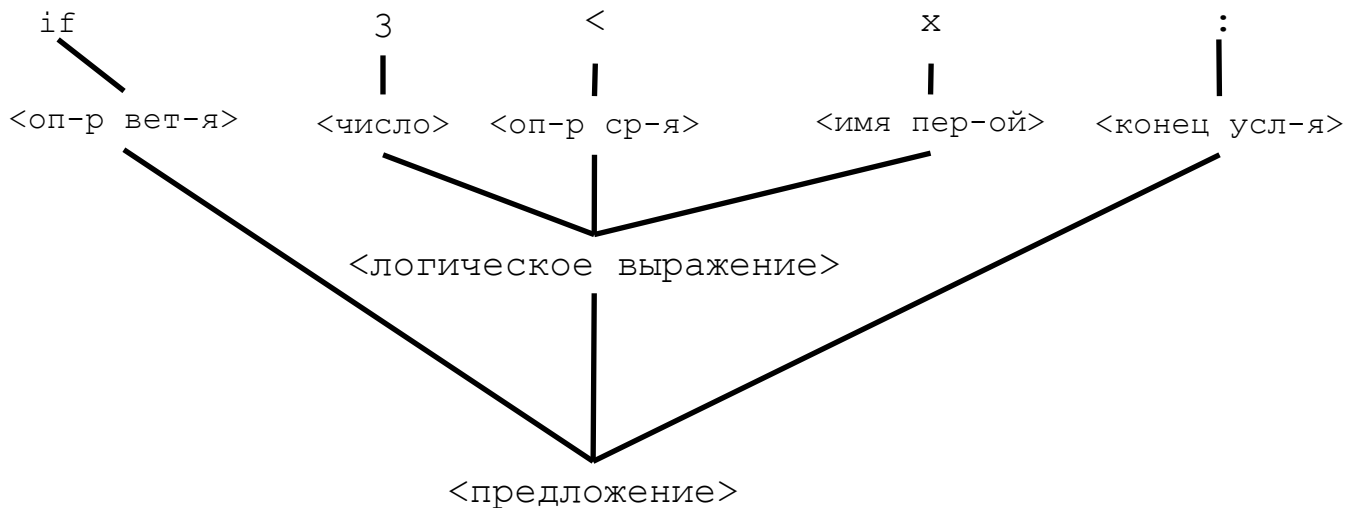
Это токены предложение допустимые содержит, но недопустимую структуру все.

Когда вы читаете предложение на естественном языке или утверждение на формальном языке, вы должны понять структуру. Этот процесс называется **синтаксическим разбором**.

Выполним разбор следующей строчки кода на языке Python с помощью синтаксического дерева:

```
if 3 < x:
```

Для этого для начала нужно выделить лексемы из данного предложения. Например, ключевое слово «if» является оператором ветвления, x – именем переменной, а символ «:» - концом условия. Лексемы можно выделять далее в отдельные группы. Так, совокупность числа, оператора сравнения и имени переменной даёт в данном предложении логическое выражение и уже тогда оператор ветвления, логическое выражение и конец условия дадут синтаксически верную строчку кода, которая задаёт условие для программы.



В настоящий момент ограничимся описанием синтаксиса (структуры) языков программирования, а не их семантики (смысла). Синтаксические деревья не несут цели передать смысла предложения. Это синтаксическое описание назовем грамматикой языка.

Метаязык, который будет использован, впервые был предложен американским лингвистом Хомским для описания естественных языков. По мнению Хомского, этот метаязык полезен лишь для описания небольших подмножеств, состоящих из простых предложений, поэтому лингвисты обращаются к более мощным метаязыкам. Примененная конкретная система записи называется формой Бэкуса-Наура.

Выпишем правила грамматики:

<предложение> -> <оп-р вет-я> <лог. выр-е> <конец усл-я>
 <оп-р вет-я> -> if
 <лог. выр-е> -> <число> <оп-р ср-я> <имя пер-ой>
 <число> -> 3
 <оп-р ср-я> -> <
 <имя пер-ой> -> x
 <конец усл-я> -> :

Символ «->» означает, что выражение слева может быть описано более подробно правилами справа.

Грамматика построения правила «число» может быть усложнена: для того, чтобы сказать, что числом может быть любая цифра следует написать:

<число> -> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Т.е. в правиле сказано, что число – это ноль или 1, или 2 и т.д.. Правила могут быть рекурсивные, т.е. чтобы сказать, что число может быть многозначным набором цифр, нужно написать правила:

<число> -> <цифра> <число> | <цифра>

<цифра> -> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Перечисление цифр или же букв алфавита могут занимать много времени, поэтому допускается запись:

<цифра> -> 0-9

<буква> -> A-Z | a-z | A-z

При записи буквы были указаны сразу три варианта – первый для правила записи в верхнем регистре, второй для записи в нижнем, а последний для любого регистра.

Попрактикуемся:

Задание 1. Составьте порождающую грамматику для предложений, которые состоят из слов, алфавита «a», «b», «c» и разделяются запятыми.

Пример верной строки	Пример неверной строки
abc, cba, aaa, bbb	aa, bb, c
aaaaaaaaaaaaaaaa	abc,asd;bbb
c, b, a	c, , , , , b
c	,

Порождающей грамматикой будет:

<предложение> -> <слово> , <предложение> | <слово>
<слово> -> <символ> <слово> | <символ>
<символ> -> a | b | c

Задание 2. Составьте порождающую грамматику для дат в формате «ДД/ММ/ГГГГ», разделенных точкой с запятой.

Пример даты:

30/10/2000;05/12/1995

Для упрощения задачи обозначим следующие правила:

- значение месяца не может быть больше 12 и меньше 1;
- дней в месяце не больше 31 (т.е. 31 февраля построить этой грамматикой можно);
- не может быть года 0000;
- год состоит из 4 цифр.

Начальное правило для составления грамматики:

<предложение> -> <дата>;<предложение> | <дата>

Чтобы построить дату нужно правило с терминалами:

(0 1-9) | (1-2 0-9) | (3 0|1)

Записали в круглые скобки, чтобы было ясно для чего именно используется ИЛИ.

Задание 3. Составьте порождающую грамматику для цикла на языке C++, переменная цикла, если используется, обязательно будет целым числом. Также гарантируется, что в цикле будет не более одного логического выражения в условии выполнения цикла.

Пример верной строки	Пример неверной строки
<code>for (int i=0; i<23; i++)</code>	<code>for (int i=0; i++)</code>
<code>for (; i >= -1; ++i)</code>	<code>(int i=0; i<23; i++)</code>
<code>for (int i=a; i >= b; i--)</code>	<code>for (i=0; i<23; i++)</code>
<code>for (; i == b;)</code>	<code>for (int i=0; i=23;)</code>
<code>for (; ;)</code>	<code>far (; ; ;)</code>

<цикл> -> for(<объявление>;<лог. вып-е>;<изм.пер.цикла>)
<объявление> -> int <имя переменной>=(<число>|<имя переменной>)
| ∅

<имя переменной> -> <пб><об>
<пб> -> A-z
<об> -> <буква> <об>|<цифра> <об> | _ <об> | ∅
<буква> -> A-z
<цифра> -> 0-9

Одно из назначений грамматики заключается в том, чтобы описать все предложения языка с помощью приемлемого числа правил. Это важно, если учесть тот факт, что обычно количество предложений в языке бесконечно. Для описания синтаксиса и семантики используются разные средства. Так описание синтаксиса языка может быть выполнено с применением формальных грамматик, а описание его семантики — с помощью атрибутивных

грамматик. В данном разделе рассматриваются только вопросы, связанные с описанием синтаксиса.

Пользуясь аналогией с естественным языком, формальный язык можно представить как множество предложений, построенных по определенным правилам. Способ построения предложений формального языка и правила построения могут быть определены с помощью формальных грамматик. При этом множество правил построения называют схемой грамматики, а порядок построения определяется с помощью понятия вывода. Как правило, с помощью правил грамматики можно строить различные выводы, результатом которых являются правильные предложения языка. Поэтому формальные грамматики часто называют порождающими грамматиками, а вывод — процессом порождения.

8.3. Синтаксис, семантика и лексика

Говоря о любом языке, можно выделить его синтаксис, семантику и лексику.

Синтаксис языка — это набор правил, формирующий допустимые конструкции языка, т. е. синтаксис определяет «форму языка» — задает набор цепочек символов, которые принадлежат языку. Чаще всего синтаксис языка можно задать в виде строгого набора правил, но полностью это утверждение справедливо только для чисто формальных языков. Даже для большинства языков программирования набор заданных синтаксических конструкций нуждается в дополнительных пояснениях, а синтаксис естественных языков изобилует исключениями.

Семантика языка — это раздел языка, определяющий значение предложений языка. Семантика языка определяет «содержание языка» — задает смысл для всех допустимых цепочек языка. Семантика для

большинства языков определяется неформальными методами. Отношения между знаками, и тем, что они обозначают, изучаются семиотикой. Чисто формальные языки лишены какого-либо смысла.

Лексика — это совокупность слов, словарный запас языка.

Лексема языка — это конструкция, которая состоит из элементов алфавита и не содержит в себе других конструкций. Таким образом, лексема может содержать только элементарные (терминальные) символы и не может содержать других лексических единиц.

Лексемами русского языка являются слова русского языка, а знаки препинания и пробелы представляют собой разделители, не образующие лексем. Лексемами алгебры являются числа, знаки математических операций, обозначения функций и неизвестных величин. В языках программирования лексическими единицами являются ключевые слова, идентификаторы, константы, метки, знаки операций, при этом в отдельную группу выделяются разделители — запятые, скобки, точки с запятой и т. д.

Процедура — это конечная последовательность инструкций, которые могут быть автоматически выполнены. Примером может служить машинная программа. Процедура, которая всегда заканчивается, называется алгоритмом.

Один из способов представления языка — задать алгоритм, определяющий принадлежность цепочки языку. Более общий способ состоит в том, чтобы определить процедуру, которая останавливается с ответом «да» для цепочек, принадлежащих языку, и либо останавливается с ответом «нет», либо вообще не останавливается для цепочек, не принадлежащих языку. Говорят, что такая процедура или алгоритм распознает язык.

Этот метод представляет язык с точки зрения распознавания. Язык можно также представить методом порождения, а именно, можно задать

процедуру, которая систематически порождает в определенном порядке цепочки языка.

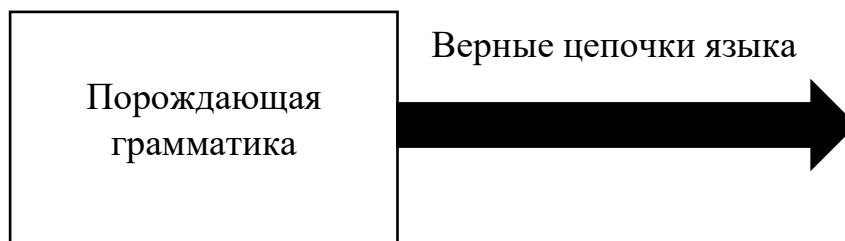
8.4. Порождающая и распознающая грамматики

Существует два способа задать язык:

Первый способ — это определить конечное множество правил порождения за конечное число шагов правильных цепочек, причем эти правила не позволяют построить никакую цепочку, не принадлежащую языку.

Второй способ — это задать механизм распознавания, который, получив в качестве аргумента любую конечную цепочку лексем из словаря, за конечное число шагов дает ответ, принадлежит эта цепочка определяемому языку или нет.

Под *порождающей грамматикой* языка понимается конечный набор правил, позволяющий строить все «правильные» предложения этого языка, применение которых не даст ни одного «неправильного» предложения.



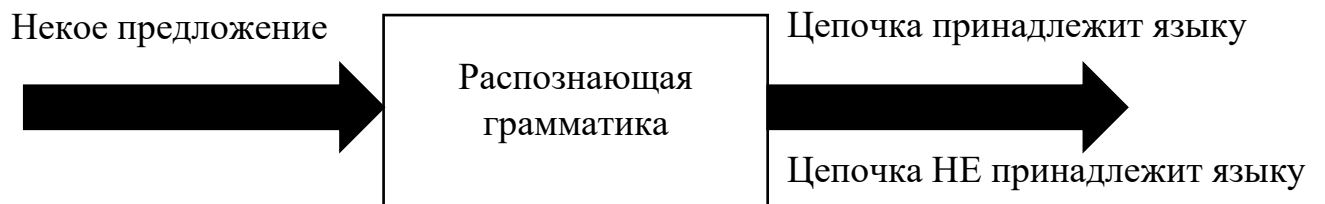
Порождающую грамматику чисел мы уже составляли в форме Бэкуса-Наура:

<число> -> <цифра> <число> | <цифра>

<цифра> -> 0-9

Распознающая грамматика задает критерий принадлежности произвольной цепочки данному языку. Это фактически некоторый алгоритм, принимающий в качестве входа символ за символом произвольную цепочку и дающий на выходе один из двух возможных ответов: либо «цепочка принадлежит языку», либо «цепочка не принадлежит языку». Этот алгоритм должен разделить все возможные входные цепочки на два класса: один класс

— принадлежащие языку цепочки, другой класс — не принадлежащие языку цепочки.

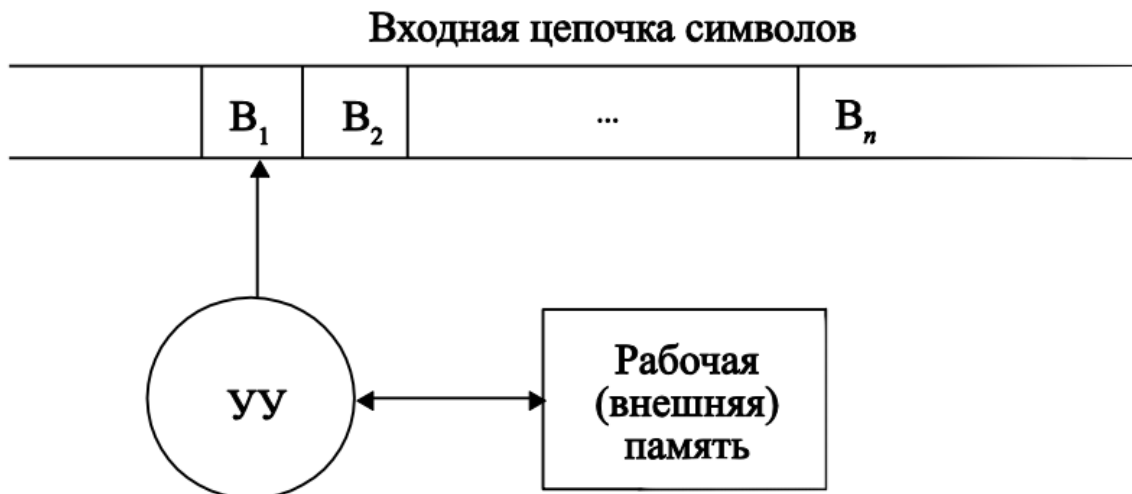


Задачу по определению принадлежности цепочки языку может решать, например, распознаватель.

8.5. Модель распознавателя

Распознаватель — это специальный автомат, который позволяет определить принадлежность цепочки символов некоторому языку. Задача распознавателя заключается в том, чтобы на основании исходной цепочки дать ответ на вопрос о принадлежности ее заданному языку.

В общем виде распознаватель можно представить в виде следующей схемы:



Распознаватель состоит из следующих основных компонентов:

- **лента**, содержащая входную цепочку символов, и считывающей головки, обозревающей очередной символ этой цепочки;
- **устройство управления (УУ)**, которое координирует работу распознавателя и имеет некоторый набор состояний и конечную память

для хранения своего состояния и некоторой промежуточной информации;

- **внешняя (рабочая) память**, которая может хранить некоторую информацию в процессе работы распознавателя и, в отличие от памяти УУ, имеет неограниченный объем.

В процессе своей работы распознаватель может выполнять некоторые элементарные операции:

- чтение очередного символа из входной цепочки;
- сдвиг входной цепочки на заданное количество символов (вправо или влево);
- доступ к рабочей памяти для чтения или записи информации;
- преобразование информации в памяти УУ, изменение состояния УУ.

Проектируемый распознаватель является конечным автоматом, т.е. количество состояний, в которых он может находиться, известно заранее.

Распознаватель будет называться **детерминированным** в том случае, если для каждой допустимой конфигурации распознавателя, которая возникла на некотором шаге его работы, существует единственно возможная конфигурация, в которую распознаватель перейдет на следующем шаге работы.

Для того, чтобы написать распознаватель, требуется составить таблицу переходов на основе порождающей грамматики. Процесс составления таблицы переходов разберем на примере задачи:

8.5.1. Построение таблицы переходов распознавателя на примере

Напишите программу конечного автомата, который распознает следующие цепочки как верные:

- предложения состоят из слов, разделенных точкой с запятой;
- первая буква слова может состоять только из букв {a,b,c,d,e};
- остальная часть слова может состоять из букв a-f, а также цифр 0-4.

Для решения такой задачи ваш первый шаг – составление порождающей грамматики языка:

```
<предложение> -> <слово> <разделитель> <предложение> |  
<слово>  
<слово> -> <первая буква> <остальные буквы>  
<первая буква> -> a-e  
<остальные буквы> -> (a-f | 0-4) <остальные буквы> |  $\emptyset$   
<разделитель> -> ;
```

У распознавателя будет шесть состояний (пять из них – возьмем правила порождающей грамматики), а именно:

- предложение;
- слово;
- первая буква;
- остальные буквы;
- разделитель;
- НЕТ – состояние, в которое переходит распознаватель, если встречается неизвестный символ или известный символ, который стоит на неверном месте.

При построении таблицы переходов состояний конечного детерминированного автомата основное внимание уделяется терминальным символам и тому, можно ли из некоторого правила перейти по нему в другое или нет.

[illegible]

Сокращение	Расшифровка
предл.	предложение
слово	слово
п.б.	первая буква слова
о.б.	остальные буквы слова
разд.	разделитель
НЕТ	ложное состояние

правило – предложение, также его можно бы
стоянием», что в дальнейшем рекомендую дел
лиц переходов, т.е. сокращенно можно его наз
ного состояния можно перейти в а, это будет
к в буквы b-e:

[illegible][illegible]

Затем, из первой буквы можно перейти в любой из символов a-f и 0-4, это будет оставшая часть слова. Также можно перейти и в точку с запятой, тогда слово закончится и должно начаться другое, т.е. автомат перейдет в состояние «разделитель»:

Сост.	a	b	c	d	e	f	0	1	2	3	4	;
Н.С.	п.б.	п.б.	п.б.	п.б.	п.б.							
слово												
п.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	разд.
о.б.												
разд.												
НЕТ												

Затем, из состояния «остальные буквы слова» можно сколько угодно далее переходить по символам a-f 0-4, или же перейти в точку с запятой и перейти в начальное состояние.

Сост.	a	b	c	d	e	f	0	1	2	3	4	;
Н.С.	п.б.	п.б.	п.б.	п.б.	п.б.							
слово												
п.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	разд.
о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	разд.
разд.												
НЕТ												

Из состояния «разделитель» можно переходить по тем же правилам, что и у начального состояния:

Сост.	a	b	c	d	e	f	0	1	2	3	4	;
Н.С.	п.б.	п.б.	п.б.	п.б.	п.б.							
слово												
п.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	разд.
о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	разд.
разд.	п.б.	п.б.	п.б.	п.б.	п.б.							
НЕТ												

Состояние «слово» конечному автомату оказалось не нужно, два состояния «п.б.» и «о.б.» уже подробно описывают как собрать слово, так что уберем его из таблицы. Также заметим, что состояние «разделитель» просто дубликат начального состояния, а потому сразу сделаем, что при чтении точки запятой из состояний «п.б.» и «о.б.» автомат переходит в начальное состояние.

Сост.	a	b	c	d	e	f	0	1	2	3	4	;
Н.С.	п.б.	п.б.	п.б.	п.б.	п.б.							
п.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	Н.С.
о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	Н.С.
НЕТ												

И, как завершающий штрих, все пустые клетки, т.е. при чтении таких символов порождающей грамматикой не предусмотрен переход куда-либо, должны означать, что входная цепочка не принадлежит порождающей грамматике, т.е. автомат перейдет в ложное состояние – «НЕТ»:

Сост.	a	b	c	d	e	f	0	1	2	3	4	;
Н.С.	п.б.	п.б.	п.б.	п.б.	п.б.	НЕТ	НЕТ	НЕТ	НЕТ	НЕТ	НЕТ	НЕТ
п.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	Н.С.
о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	о.б.	Н.С.
НЕТ	НЕТ	НЕТ	НЕТ	НЕТ	НЕТ	НЕТ	НЕТ	НЕТ	НЕТ	НЕТ	НЕТ	НЕТ

Таким образом, таблица переходов состояний конечного детерминированного автомата успешно построена. Она будет «мозгом» для моделируемого распознавателя, который будет на каждом этапе сверяться с ней и переходить в соответствующее состояние.

8.5.2. Создание распознавателя на языке Python

Для упрощения работы с таблицей переходов её необходимо немного видоизменить, а именно заменить все наименования состояний на числа, начиная от нуля. Также сделать у всех переходов:

```
eda203;f5;da22
```

Сост.	a	b	c	d	e	f	0	1	2	3	4	;
0	1	1	1	1	1	3	3	3	3	3	3	3
1	2	2	2	2	2	2	2	2	2	2	2	0
2	2	2	2	2	2	2	2	2	2	2	2	0
3	3	3	3	3	3	3	3	3	3	3	3	3

Теперь нужно написать код на Python, реализуем класс конечного автомата (Finite State Automata) – он будет иметь в качестве полей алфавит (alphabet) языка, таблицу переходов (jumping_table), текущее состояние автомата (state), будет помнить какое состояние неверное (wrong_state), а также будет объявлен список верных состояний для подведения итогов

(right_state_list), ведь верных состояний может быть два – автомат может остановиться после распознавания первой буквы (состояние п.б., или же 1), а может после распознавания остальных букв слова (о.б. или же 2), определим все эти поля при инициализации класса, или же в его конструкторе:

```
class FSA:
    def __init__(self):
        self.alphabet = 'abcdef01234;'
        self.jumping_table = [
            [1, 1, 1, 1, 1, 3, 3, 3, 3, 3, 3, 3],
            [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0],
            [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0],
            [3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
        ]

        self.state = 0
        self.wrong_state = 3

        # Имеется несколько состояний, когда цепочка верная
        # Это "п.б." и "о.б.", т.е. 1 и 2
        self.right_state_list = [1, 2]
```

Затем создадим метод, который будет определять принадлежность входной цепочки к языку, определяемому таблицей переходов. В цикле будет считываться новый символ, затем будет происходить проверка принадлежит ли символ алфавиту, если он принадлежит, то в соответствии с текущим состоянием автомата и входным символом будет определено новое состояние. По окончании цикла, если состояние автомата находится в состоянии, определяющим верную цепочку возвращаем истину, иначе – ложь.

```
def is_right(self, input_text: str) -> bool:
    """
    Определения принадлежности входной цепочки
    input_text: строка на проверку
    Возвращает истину или ложь
    """
    self.state = 0

    k = 0
    while k < len(input_text):
        letter = input_text[k]
        if letter in self.alphabet:
            letter_index = self.alphabet.index(letter)
            self.state = self.jumping_table[self.state]
        else:
```

```

        self.state = self.wrong_state
        k += 1

    if self.state in self.right_state_list:
        return True
    else:
        return False

```

Также определим метод вывода информации на экран, который будет сообщать пользователю верна ли входная цепочка на основе метода `is_right`, который был описан выше:

```

def give_answer(self, input_text:str) -> None:
    if self.is_right(input_text) is True:
        print(f'Входная цепочка верна')
    else:
        print(f'Входная цепочка неверна')

```

И затем, создадим точку входа для исполнения программы – считаем строку с клавиатуры, создадим объект класса `FSA`, у которого вызовем метод `give_answer` (т.е. «выведи ответ»).

```

if __name__ == '__main__':
    input_text = input('Введите строку для проверки: ')
    Machine = FSA()
    Machine.give_answer(input_text)

```

Программирование распознавателя на этом закончено. Протестируем его работу:

Ввод пользователя	Результат работы программы
Введите строку для проверки: ada11;efc023a;deda22	Входная цепочка верна
Введите строку для проверки: ee	Входная цепочка верна
Введите строку для проверки: eda203;f5;da22	Входная цепочка неверна
Введите строку для проверки: asd11	Входная цепочка неверна
Введите строку для проверки: 0;11	Входная цепочка неверна

Удостоверьтесь, что распознаватель верно отвечает на то, принадлежит ли некая строка заданному языку или нет. Попробуйте несколько своих тестов, помимо предложенных.

Попрактикуемся:

Доработайте распознаватель выше так, чтобы он не только определял верна строчка или неверна, но также в случае подачи неверной строки вывести символ и индекс первой ошибки.

Ввод пользователя	Результат работы программы
Введите строку для проверки: eda203;f5;da22	Входная цепочка неверна Ошибка на месте 7 - символ f
Введите строку для проверки: ada11;efc023a;	Входная цепочка неверна Ошибка на месте 13 - символ ;

9. Проект: Моделирование распознавателя

Требуется построить детерминированный конечный автомат, который распознаёт входную цепочку в соответствии с вариантом задания. Для решения задачи требуется вывести формальную грамматику языка, построить таблицу переходов, и на основе неё написать программу на Python, которая моделирует работу распознавателя.

9.1.1. Вариант 1. «Двоичная запись»

Распознаватель должен верно определять последовательность из двоичных чисел, имеющих префикс «0b», разделенных точкой с запятой. Само число после префикса не может начинаться с нуля, если присутствует целая ненулевая часть. Допускается вещественная запись числа.

Вещественная запись числа, в которой дробная часть отсутствует, не допускается.

Ввод пользователя	Результат работы программы
Введите строку для проверки: 0b10110;0b11;0b111010	Входная цепочка верна
Введите строку для проверки: 0b10110.101;0b11.0110;0b0.011	Входная цепочка верна
Введите строку для проверки: 0b101;0b011	Входная цепочка неверна
Введите строку для проверки: 0b1101/0b.01	Входная цепочка неверна

9.1.2. Вариант 2. «Восьмеричная запись»

Распознаватель должен верно определять последовательность из восьмеричных чисел, имеющих префикс «0o», разделенных косой чертой. Само число после префикса не может начинаться с нуля, если присутствует целая ненулевая часть. Допускается вещественная запись числа.

Вещественная запись числа, в которой дробная часть отсутствует, не допускается.

Ввод пользователя	Результат работы программы
Введите строку для проверки: 0o556/0o11/0o2	Входная цепочка верна
Введите строку для проверки: 0o31.22/0o761.3/0o5.0/0o0.25	Входная цепочка верна
Введите строку для проверки: 0o77/0o218	Входная цепочка неверна
Введите строку для проверки: 0o.2	Входная цепочка неверна

9.1.3. Вариант 3. «Шестнадцатеричная запись»

Распознаватель должен верно определять последовательность из шестнадцатеричных чисел, имеющих префикс «0x», разделенных решеткой. Само число после префикса не может начинаться с нуля, если присутствует целая ненулевая часть. Допускается вещественная запись числа.

Вещественная запись числа, в которой дробная часть отсутствует, не допускается.

Ввод пользователя	Результат работы программы
Введите строку для проверки: 0xAE23D#0x23C#0x11	Входная цепочка верна
Введите строку для проверки: 0xEA.F#0x23.A#0x0.AE33EA	Входная цепочка верна
Введите строку для проверки: AE21.33#AA#0x22	Входная цепочка неверна
Введите строку для проверки: 0x54#0x02#0xАН	Входная цепочка неверна

9.1.4. Вариант 4. «Путь к файлу»

Распознаватель должен верно определять последовательность, описывающую путь к файлу в файловой системе, сначала идет буква жесткого диска, на котором хранится файл, затем перечисляются папки, разделенные символом обратной косой черты. В конце написано имя файла и его расширение через точку.

Ввод пользователя	Результат работы программы
Введите строку для проверки: Q:\notes\Data\fl.py	Входная цепочка верна
Введите строку для проверки: Z:\oil_research.pas	Входная цепочка верна
Введите строку для проверки: \\lab\new_files\i.txt	Входная цепочка неверна
Введите строку для проверки: Oil_research.pas	Входная цепочка неверна

9.1.5. Вариант 5. «Язык гласных и согласных»

Распознаватель должен верно определять последовательность цепочек языка, слова которого могут начинаться с:

- гласной в верхнем регистре, тогда остальные буквы слова будут состоять только из согласных в нижнем регистре и цифр;
- согласной в верхнем регистре, тогда остальные буквы слова будут состоять только из гласных.

Слова, начинающиеся с гласной, разделяются со словом, которое началось с согласной точкой с запятой. Слова, начинающиеся с согласной, разделяются со словом, которое началось с гласной запятой. Слова же, начинающиеся с одной буквы, друг с другом разделяются символом нижнего подчеркивания.

Ввод пользователя	Результат работы программы
Введите строку для проверки: Oгpц22;Ноое_Вея,Ястл	Входная цепочка верна
Введите строку для проверки: Маое	Входная цепочка верна
Введите строку для проверки: Няа_Ояф	Входная цепочка неверна
Введите строку для проверки:	Входная цепочка неверна

Пооие_Поауа,Поу	
-----------------	--

9.1.6. Вариант 6. «Цепочки из латиницы»

Распознаватель должен верно определять последовательность цепочек языка, слова которого могут состоять букв латинского алфавита, части слова могут отделяться точкой (не более двух на слово, подряд быть не могут), разделенных косой чертой. Последовательность должна заканчиваться символом решетки.

Ввод пользователя	Результат работы программы
Введите строку для проверки: AISD.gumrf.ru/arts.SPBU.ru#	Входная цепочка верна
Введите строку для проверки: a.sd/hello.world/grapejuice#	Входная цепочка верна
Введите строку для проверки: p..s/h.o.a.r/	Входная цепочка неверна
Введите строку для проверки: as...#	Входная цепочка неверна

9.1.7. Вариант 7. «Десятеричная запись»

Распознаватель должен верно определять последовательность из десятиричных чисел, числа могут быть отрицательными, т.е. иметь префикс «-», разделенных точкой с запятой. Само число после префикса не может начинаться с нуля, если присутствует целая ненулевая часть. Допускается вещественная запись числа.

Вещественная запись числа, в которой дробная часть отсутствует, не допускается.

Ввод пользователя	Результат работы программы
Введите строку для проверки: 54;-11;895	Входная цепочка верна
Введите строку для проверки: -0.4543;254.88;123	Входная цепочка верна
Введите строку для проверки: 55;0xAE24;22	Входная цепочка неверна
Введите строку для проверки: 0.55;	Входная цепочка неверна

9.1.8. Вариант 8. «Запись дат, вер. 1»

Распознаватель должен верно определять последовательность из дат в формате «ДД.ММ.ГГГГ», разделенных вертикальной чертой «|». День может быть числом от 1 до 31, месяц от 1 до 12, год не может быть нулевым.

Ввод пользователя	Результат работы программы
Введите строку для проверки: 31.12.2000 22.01.1995 11.02.1765	Входная цепочка верна
Введите строку для проверки: 22.01.0053	Входная цепочка верна
Введите строку для проверки: 22.11.2000/14.01.2023	Входная цепочка неверна
Введите строку для проверки: 83.99.0000	Входная цепочка неверна

9.1.9. Вариант 9. «Запись дат, вер. 2»

Распознаватель должен верно определять последовательность из дат в формате «ГГ.ММ.ДД.», каждая дата заключается в квадратные скобки «[]», разделенных точкой. День может быть числом от 1 до 31, месяц от 1 до 12, год не может быть нулевым.

Ввод пользователя	Результат работы программы
Введите строку для проверки: [66.01.31].[55.03.01]	Входная цепочка верна
Введите строку для проверки: [01.12.31]	Входная цепочка верна
Введите строку для проверки: 89.01.23.[77.53.22]	Входная цепочка неверна
Введите строку для проверки: [55.23.34]	Входная цепочка неверна

9.1.10. *Вариант 10. «Язык гласных и согласных, вер. 2»*

Распознаватель должен верно определять последовательность из цепочек языка, где слово может состоять только из чередующихся друг за другом гласных и согласных, при этом слово не может заканчиваться на гласной. Буквы «ь» и «Ъ» - нейтральные, допускаются вместо использования как гласных, так и согласных. Слова разделяются друг от друга запятой.

Ввод пользователя	Результат работы программы
Введите строку для проверки: фаворит, виногра́д, апельсин	Входная цепочка верна
Введите строку для проверки: закон	Входная цепочка верна
Введите строку для проверки: о	Входная цепочка неверна
Введите строку для проверки: пони, виноград, цепочка	Входная цепочка неверна
Введите строку для проверки: о, а, и, в,	Входная цепочка неверна

9.1.11. *Вариант 11. «Цепочки из латиницы, вер. 2»*

Распознаватель должен верно определять последовательность слов, состоящих из цифр и латинских букв, разделенных тире «-». Первые три буквы слова могут состоять только из букв {a,d,z}, остальная часть слова может быть любыми остальными буквами или цифрами.

Ввод пользователя	Результат работы программы
Введите строку для проверки: aaa12-zzd22-dadf	Входная цепочка верна
Введите строку для проверки: d-a-z-zzafff	Входная цепочка верна
Введите строку для проверки: a, d, z	Входная цепочка неверна
Введите строку для проверки: 1123-zza-aaa	Входная цепочка неверна

9.1.12. *Вариант 12. «Русская нотация шахматных фигур»*

Распознаватель должен верно определять шахматную нотацию, начала идёт обозначение фигуры (см. таблицу сокращений), затем идёт клетка, с которой ход был сделан, а после через тире клетка, на которую сходил фигура. Ходы различных фигур разделены точкой с запятой.

Допустимые номера клеток, на которые может совершить ход та или иная фигура:1-8

Таблица. Русская сокращенная запись шахматных фигур для нотации

Наименование фигуры	Сокращение
Король	Кр
Ферзь	Ф
Ладья	Л
Конь	К
Слон	С
Пешка	П

Ввод пользователя	Результат работы программы
Введите строку для проверки: Kh1-h8;Фe2-b1	Входная цепочка верна
Введите строку для проверки: Ph8-a1	Входная цепочка верна
Введите строку для проверки: h2-e8	Входная цепочка неверна
Введите строку для проверки: Kpb4-h7;Le8-e9;Pq1-h2	Входная цепочка неверна

9.1.13. *Вариант 13. «Английская нотация шахматных фигур»*

Распознаватель должен верно определять шахматную нотацию, начала идёт обозначение фигуры (см. таблицу сокращений), затем идёт клетка, на которую сходила фигура. Ходы различных фигур разделены знаком решетки.

Допустимые номера клеток, на которые может совершить ход та или иная фигура: 1-8

Таблица. Английская сокращенная запись шахматных фигур для нотации

Наименование фигуры	Сокращение
Король	K
Ферзь	Q
Ладья	R
Конь	N
Слон	B
Пешка	Не записывается

Ввод пользователя	Результат работы программы
Введите строку для проверки: Kh7#Qa3#Nb2	Входная цепочка верна
Введите строку для проверки: b7	Входная цепочка верна
Введите строку для проверки: Qb7#Ka0#	Входная цепочка неверна
Введите строку для проверки: NB88	Входная цепочка неверна

9.1.14. *Вариант 14. «Оператор присваивания»*

Распознаватель должен верно определять последовательность из операций присваивания и вычислений в них, разделенных точкой с запятой. В левой части оператора присваивания может быть только имя переменной, в правой части может быть как имя переменной, так и число, также числа или имена переменных могут быть объединены математическими операциями («+», «-», «*», «/»).

Ввод пользователя	Результат работы программы
Введите строку для проверки: a22=b2+22-44	Входная цепочка верна
Введите строку для проверки: b=44+100;d55=a22	Входная цепочка верна
Введите строку для проверки: 2a=33	Входная цепочка неверна
Введите строку для проверки: B=A+C-D;	Входная цепочка неверна

9.1.15. *Вариант 15. «Пара ключ-значение»*

Распознаватель должен верно определять последовательность пар словаря ключ-значение. Ключом и значением может быть строка или число. Строка может состоять из символов кириллицы, цифр и символа нижнего подчеркивания. Несколько пар ключ-значение должны разделяться запятой.

Ввод пользователя	Результат работы программы
Введите строку для проверки: 'Ключ1': 22, 'Ключ2': 'слово_33'	Входная цепочка верна
Введите строку для проверки: 22: 'Значение1'	Входная цепочка верна
Введите строку для проверки: asd2: 22;	Входная цепочка неверна
Введите строку для проверки: 500: '»@-'	Входная цепочка неверна

10. Работа с pip

Установка библиотек, изменение версий, обзор установленных библиотек

Отображение всех установленных библиотек

`pip list`

11. Создание графического интерфейса с помощью PyQt5

Создание графического пользовательского интерфейса (Graphical User Interface - GUI) является одним из ключевых способов улучшить опыт пользователя (User Experience - UX) при взаимодействии с вашим приложением.

UX/UI дизайн – это процесс проектирования таких пользовательских интерфейсов, которые будут не просто приятно выглядеть, но и удобны в использовании.

11.1. Используемые инструменты

Создавать интерфейсы позволяют различные фреймворки, в рамках данного пособия выбор выпал на Qt. Qt (читается как «cute» или же «ку-тэ») является совокупностью инструментов для создания и разработки кроссплатформенных приложений, а также проектирования пользовательского интерфейса. Изначально создан для C++, но имеет реализации на различных языках: Python – PyQt и PySide, Java – QtJambi, Ruby – QtRuby и др..

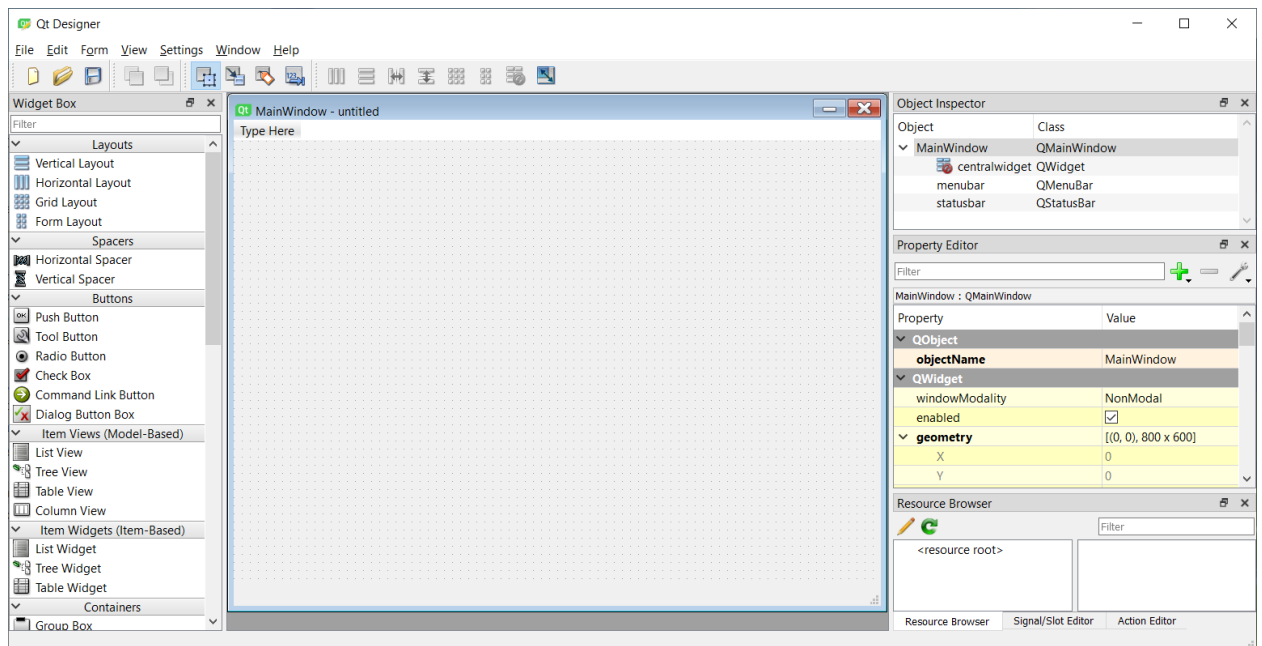
В качестве реализации Qt на Python будем использовать PyQt пятой версии (PyQt5). Для его установки требуется в `pip` прописать:

```
pip install pyqt5
```

Если у вас система не может найти `pip` сама, он должен располагаться в папке по пути:

```
«C:\<имя пользователя>\AppData\Local\Programs\Python\<версия Python>\Scripts\»
```

Для удобного проектирования интерфейса можно воспользоваться инструментарием Qt Designer. Qt Designer позволяет быстро создавать макеты окон приложения просто перетаскивая различные объекты Qt на поле проектируемого окна.



Все файлы из Qt Designer рекомендуется сохранять в формате «.ui», так при открытии файла будет видно, что данные сохраняются в виде xml-разметки, например:

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <ui version="4.0">
3    <class>MainWindow</class>
4    <widget class="QMainWindow" name="MainWindow">
5      <property name="geometry">
6        <rect>
7          <x>0</x>
8          <y>0</y>
9          <width>1163</width>
10         <height>800</height>
11        </rect>
12      </property>
13      <property name="sizePolicy">
14        <sizepolicy hsizeType="Fixed" vsizeType="Fixed">
15          <horstretch>0</horstretch>
16          <verstretch>0</verstretch>
17        </sizepolicy>
18      </property>
19      <property name="windowTitle">
20        <string>ОПОП Обработчик</string>
21      </property>
22      <property name="autoFillBackground">
23        <bool>false</bool>
24      </property>
25      <widget class="QWidget" name="centralwidget">
26        <widget class="QTabWidget" name="tabWidget">
```

Для того, чтобы обеспечить связь между работой PyQt и файлами Qt Designer нужно перевести xml-разметку в классы на Python, такую функцию

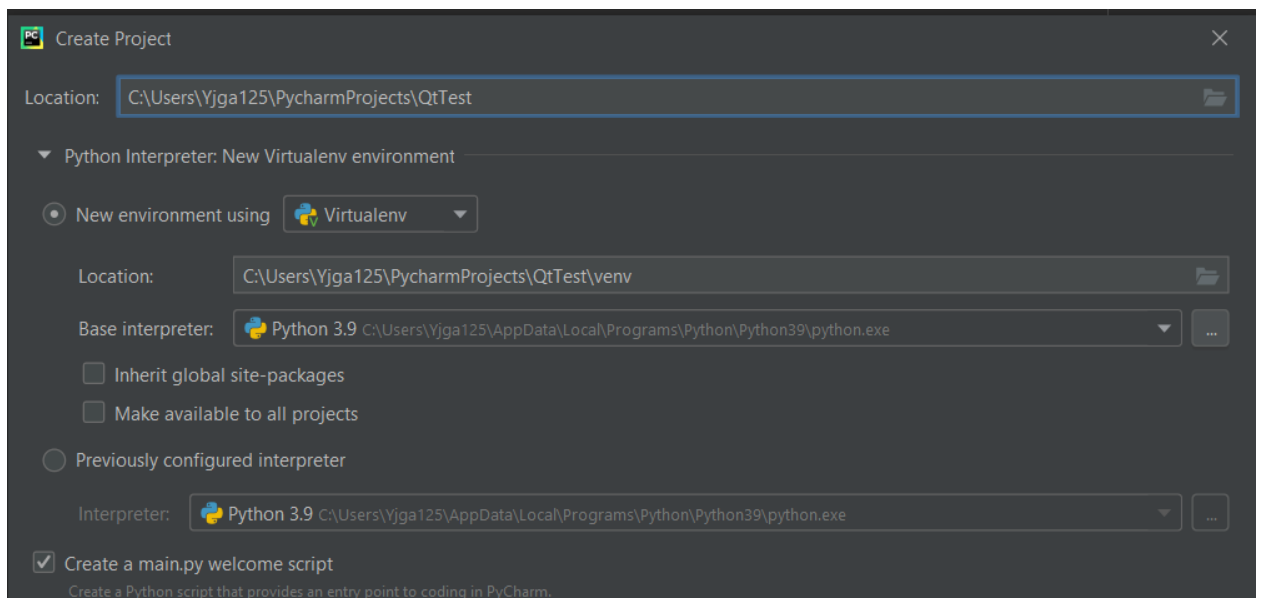
позволяет выполнять библиотека `pyuic5-tool`, для её установки в `pip` пропишем:

```
pip install pyuic5-tool
```

11.2. Создание первого проекта с графическим интерфейсом

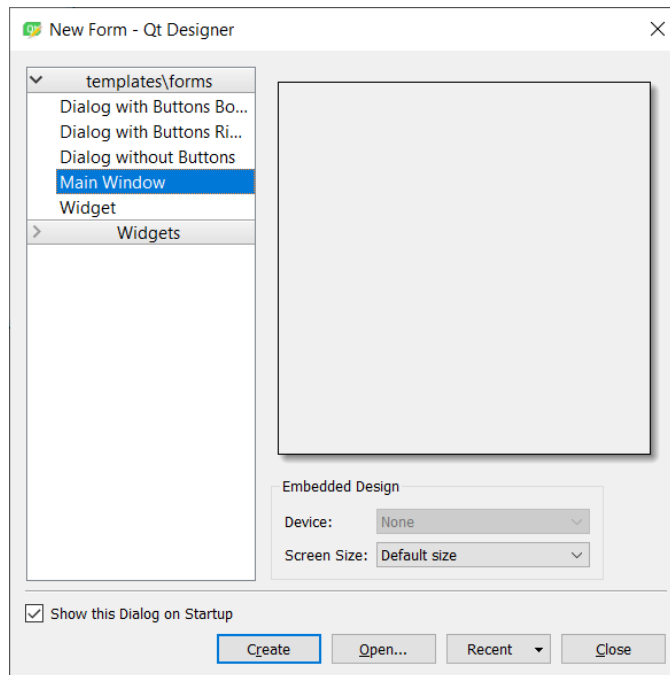
11.2.1. Создание проекта и подготовка макета окна

Создайте новый проект в PyCharm, можете его назвать, например, «QtTest»:

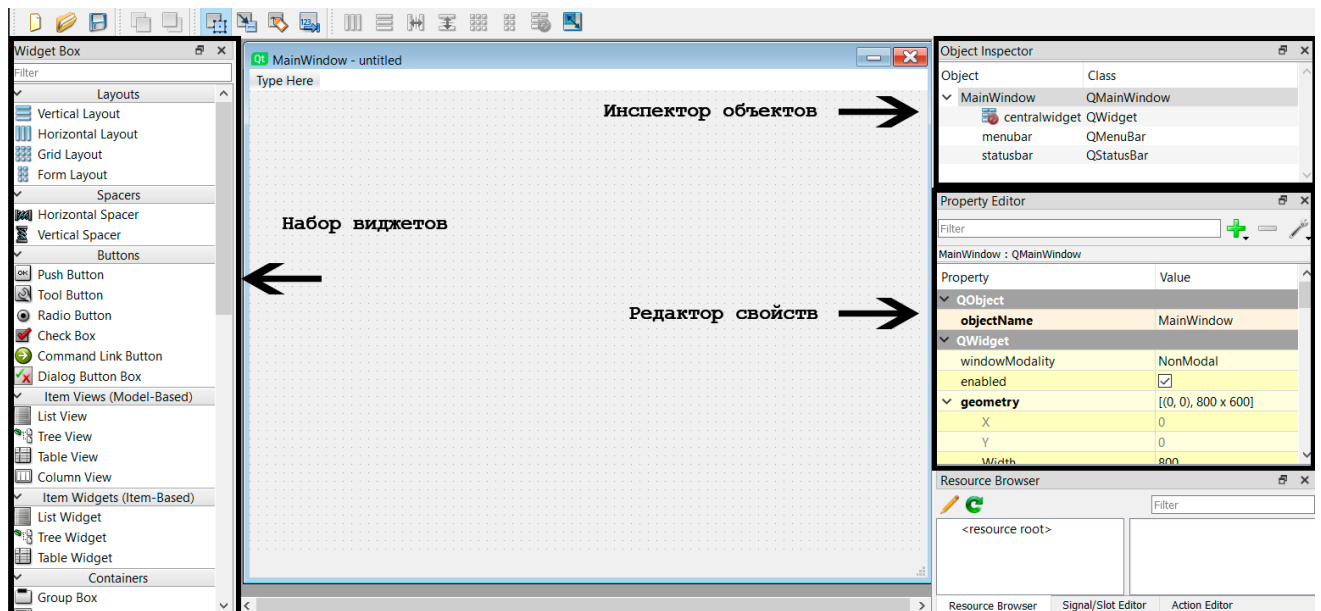


Запомните путь, по которому хранится ваш проект в PyCharm.

Спроектируем главное окно, которое будет использоваться в программе, для этого в Qt Designer создайте «Main Window»:



После нажатия «Create» будет отображено создаваемое окно, а также по бокам будут расположены различные инструменты дизайнера:



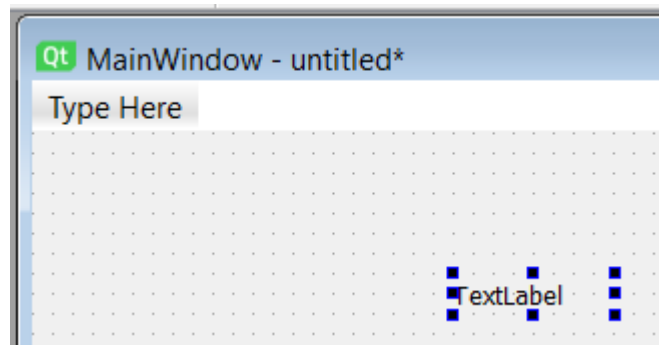
Рассмотрим более подробно инструментарий Qt Designer:

- слева располагается «Набор виджетов» - оттуда можно перетаскивать различные виджеты на проектируемое окно;
- справа сверху располагается «Инспектор объектов», который позволяет обращаться к созданным виджетам напрямую, на данный момент он не

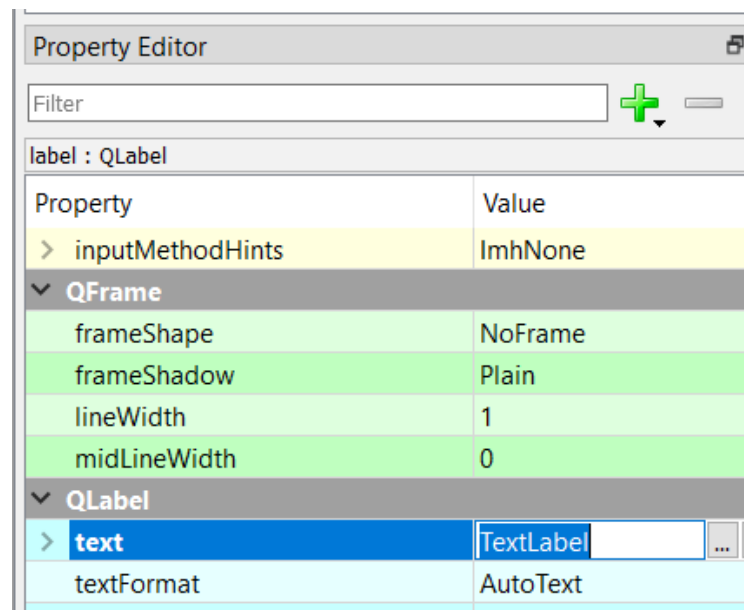
особо для нас важен, но при большом количестве объектов он определенно выручает;

- справа снизу располагается «Редактор свойств», который позволяет изменять различные свойства объектов, например, название объекта в коде или же его отображение при запуске программы.

Перенесите из набора виджетов объект «Label» - этот объект позволяет отобразить текст в определенном месте окна пользователя.



Теперь при выбранном объекте Label перейдите в редактор свойств, найдите строчку «text» и задайте вместо «TextLabel» «Hello world!»:



Также обратите внимание, как этот объект будет называться в формируемом коде – за это отвечает поле «objectName»:

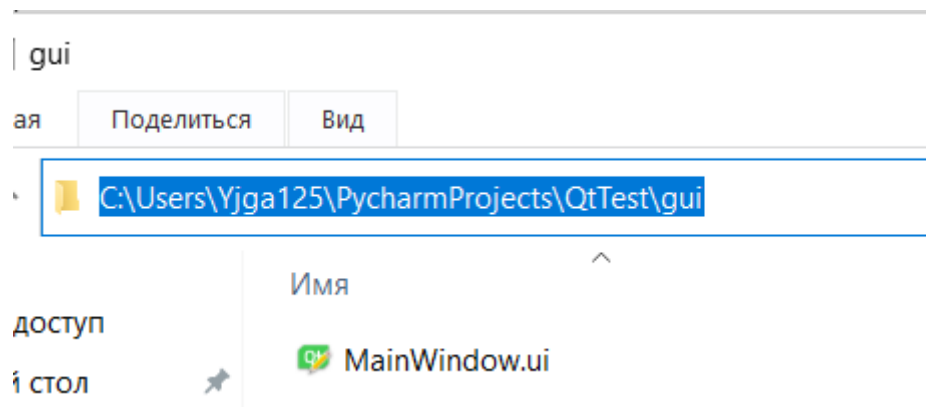
Property	Value
QObject	
objectName	label

11.2.2. Перевод файла макета окна в код на Python

Осталось только сохранить созданный проект главного окна. В папке проекта создайте папку «gui», в которой будут храниться все созданные окна в Qt Designer:

Компьютер > Локальный диск (C:) > Пользователи > Yjga125 > PycharmProjects > QtTest >		
Имя	Дата изменения	Тип
.idea	26.04.2023 13:55	Папка с файлами
gui	26.04.2023 14:38	Папка с файлами
venv	26.04.2023 13:38	Папка с файлами
main.py	26.04.2023 13:55	Python File

Затем сохраните файл Qt Designer в эту папку с названием «MainWindow.ui». Для использования спроектированного окна в проекте его надо перевести в код PyQt, поэтому воспользуемся библиотекой pyuic5. В папке gui вместо пути к проекту напишите cmd:



Тогда вызовется командная консоль, в ней мы должны прописать инструкцию «`pyuic5 MainWindow.ui -o MainWindow_ui.py`»:

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19044.2846]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\Yjga125\PycharmProjects\QtTest\gui>pyuic5 MainWindow.ui -o MainWindow_ui.py
```

При возникновении проблемы на этом шаге, когда pyuic5 не может быть обнаружен, то требуется указать полный путь к pyuic5 – он также лежит в папке Scripts, как и pip, тогда команда, которую вам следует написать будет выглядеть так:

```
C:\Users\<Имя пользователя>\AppData\Local\Programs\<Папка с  
версией Python>\Scripts\pyuic5.exe MainWindow.ui -o  
MainWindow_ui.py
```

В результате в папке gui вы сможете увидеть сконвертированный файл главного окна на языке Python «MainWindow_ui.py»:

мпьютер > Локальный диск (C:) > Пользователи > Yjga125 > PycharmProjects > QtTest > gui				
Имя	Дата изменения	Тип	Размер	
 MainWindow.ui	26.04.2023 14:47	Файл "UI"	2 КБ	
 MainWindow_ui.py	26.04.2023 14:45	Python File	2 КБ	

Теперь осталось использовать данный файл в проекте.

11.2.3. Подключение файла GUI к проекту на Python

Для того, чтобы в дальнейшем менять поля и методы автоматически сформированного кода главного окна, создадим класс «RootApp» (Корень приложения), который будет наследовать свойства основного окна из стандартных виджетов, а также свойства созданного ранее окна в Qt Designer.

```
# Стандартные библиотеки
import sys

# Внешние модули
from PyQt5 import QtWidgets

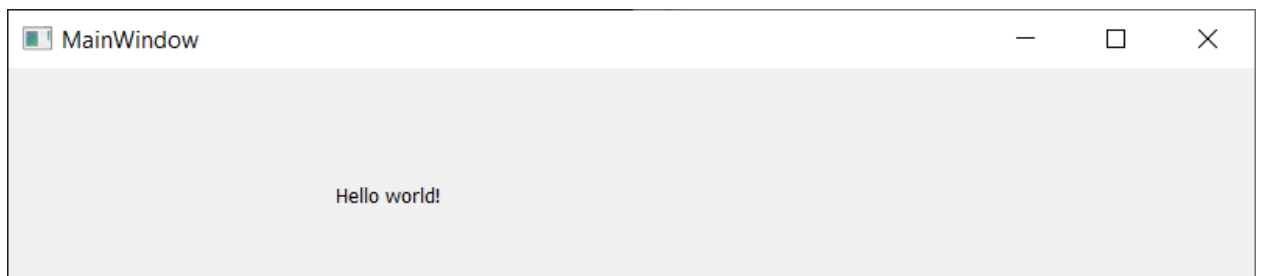
# Мои модули
from gui import MainWindow_ui

class RootApp(QtWidgets.QMainWindow,
MainWindow_ui.Ui_MainWindow):
    def __init__(self):
        super().__init__()
        self.setupUi(self)
```

Затем напомним код точки входа инициализируем приложение, а также отобразим главное окно:

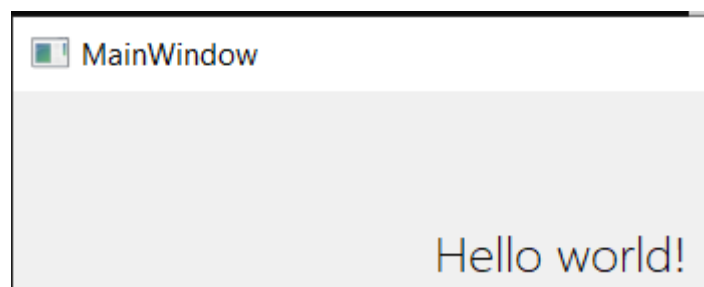
```
if __name__ == '__main__':
    app = QtWidgets.QApplication(sys.argv)
    window = RootApp()
    window.show()
    app.exec()
```

Запустив такой код, вы увидите окошко с маленьким текстом «Hello world!»:



Попрактикуемся:

С помощью редактора свойств измените шрифт (font) выводимого текста на YaHei Light, размер шрифта 14, а затем пересоздайте код главного окна на Python и запустите программу.



11.2.4. Добавление кнопки

Создадим кнопку, которая позволяет заменить текст с «Hello world!» на «Работа кнопки»

Создайте объект кнопки в Qt Designer. В редакторе свойств поменяйте название объекта на PB_change_text.

Затем доработаем код, добавьте в init следующую строчку:

```
self.PB_change_text.clicked.connect(self.change_text)
```

Т.е. этой строчкой мы связываем событие нажатия на кнопку с вызовом метода change_text. Теперь создадим сам метод change_text у RootApp:

```
def change_text(self):
    self.label.setText('Смена текста')
```

Заметьте, что мы не обращаемся напрямую к какому-либо полю «text», а вызываем метод «задай текст», т.е. в классах Qt выполняется принцип инкапсуляции.

Проверьте работу программы. Не забудьте, что после изменения макета код главного окна нужно перекомпилировать.

12. Создание приложения-библиотеки

Для дальнейшего изучения создания графического интерфейса зададимся целью – нужно создать программу, которая позволит вести учет книг – их можно добавлять, изменять и удалять из библиотеки. Данные о книгах должны выноситься в отдельный файл.

12.1. Настройка общего шрифта

Вместо того, чтобы настраивать каждый элемент интерфейса отдельно, можно настроить общий стиль шрифта для окна. Для этого выберите главное окно и в редакторе свойств поменяйте шрифт в подходящий (в примерах будет использоваться «YaHei, 12»).

12.2. Планирование пользовательского интерфейса

Для начала распланируем как будет выглядеть данное приложение – пусть сверху будет поисковая строка, чтобы исключать отображение лишних книг, ниже будет таблица, отображающая книги, а справа – действия, которые можно выполнить как с таблицей (Добавить книгу), так и с отдельными книгами (Изменить или удалить книгу).

Перетащите на окно два элемента – «LineEdit» для ввода поиска и «PushButton» для выполнения поиска. Для того, чтобы влезал текст вам может потребоваться поменять высоту этих объектов.



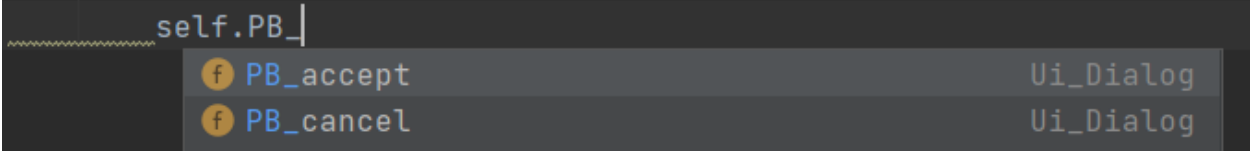
При наименовании объектов будем придерживаться некоторых договоренностей, чтобы было легче определить к какому элементу интерфейса принадлежит тот или иной объект:

Виджет	Префикс
LineEdit	LE_
TextEdit	TE_
PushButton	PB_
TableWidget	T_

Во время написания кода отметьте важный момент: так как мы договорились об однообразном наименовании объектов одного класса (например, кнопки), то IDE удобно подскажет все возможные кнопки этого окна выпадающим меню:

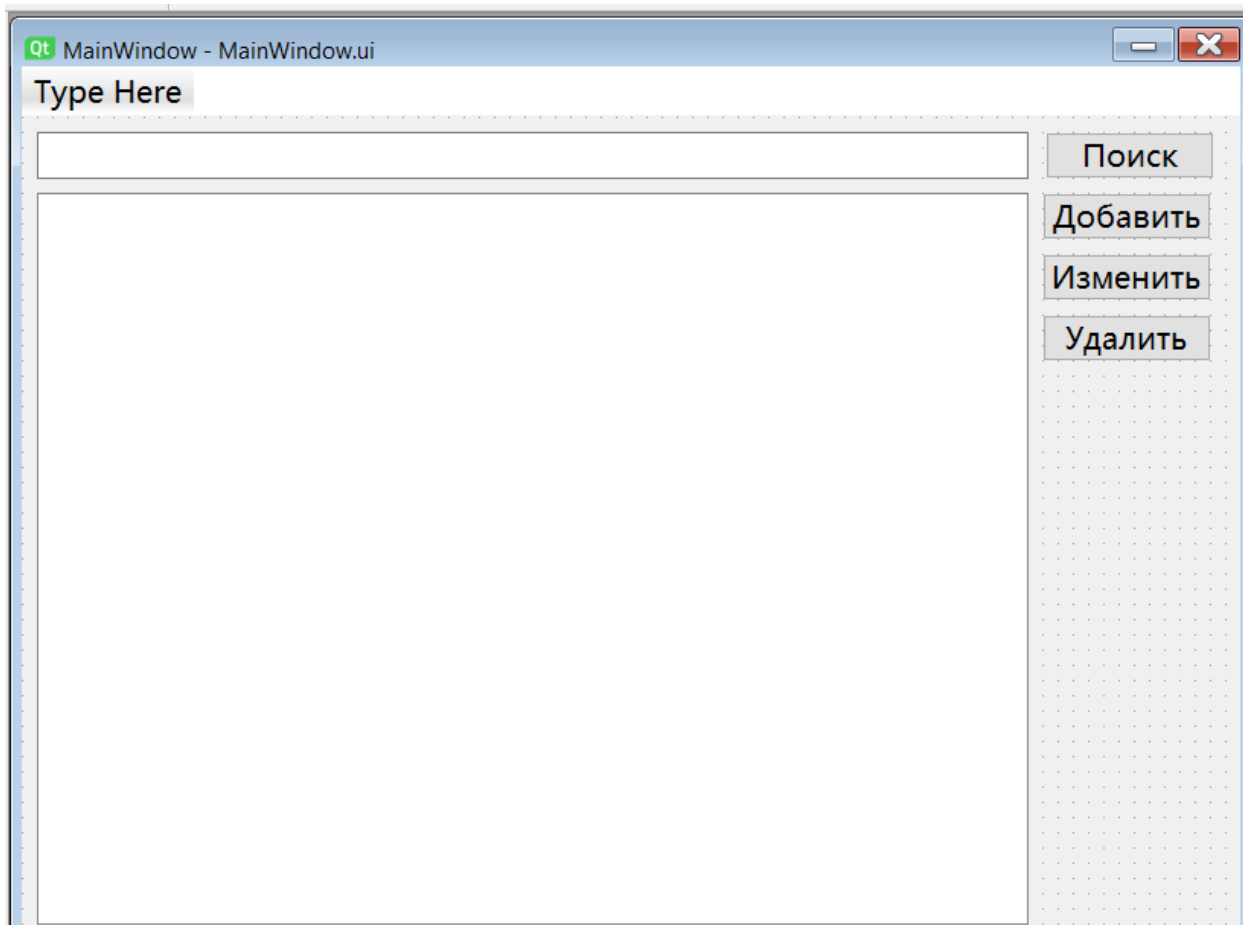
```
class AddEditBookDialog(QDialog, AddEditBook_ui.Ui_Dialog):
    def __init__(self):
        super().__init__()
        self.setupUi(self)

        # Зададим действия у кнопок
        self.PB_
```

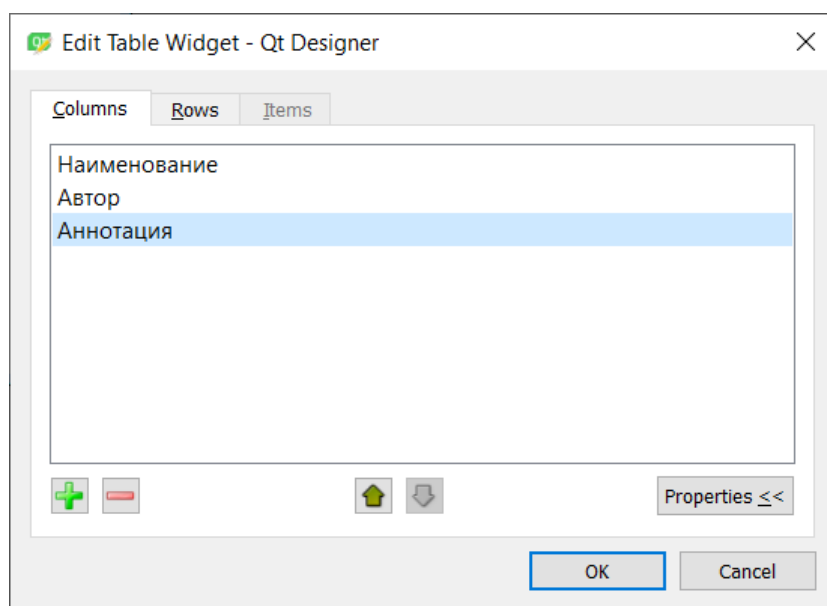


В соответствии с этим обозначим строку поиска за «LE_search_request», а кнопку «PB_to_search».

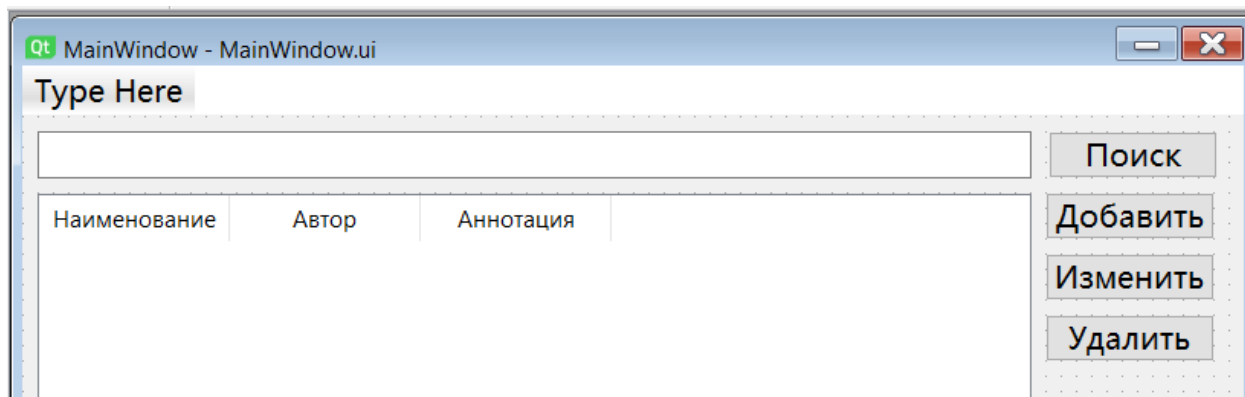
Затем добавим таблицу TableWidget с названием объекта T_Books и 3 кнопки – «Добавить» (PB_to_add), «Изменить» (PB_to_edit) и «Удалить» (PB_to_delete):



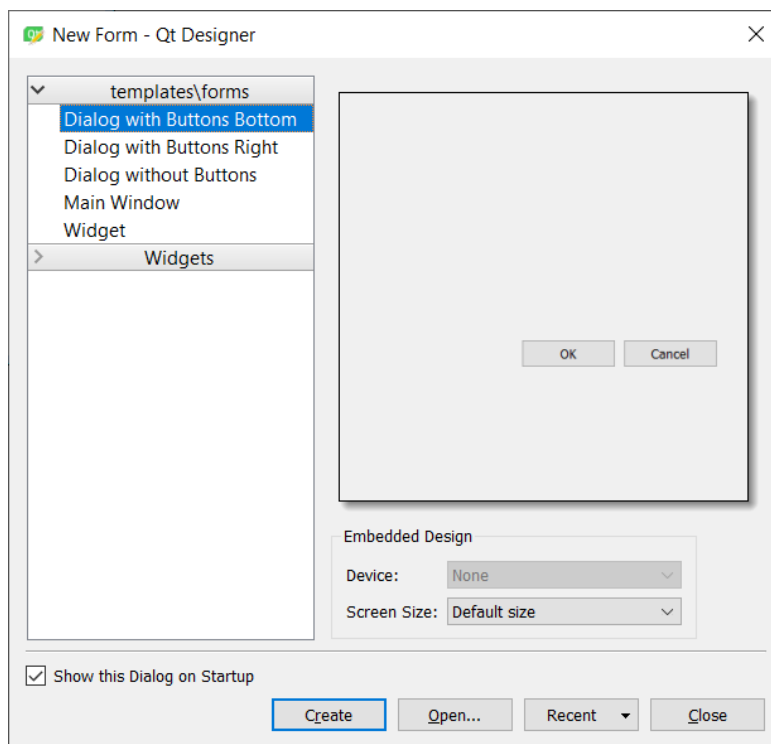
Добавим в таблицы столбцы – ПКМ по таблице – «Edit Items...» и добавьте три позиции – «Наименование», «Автор» и «Аннотация»:



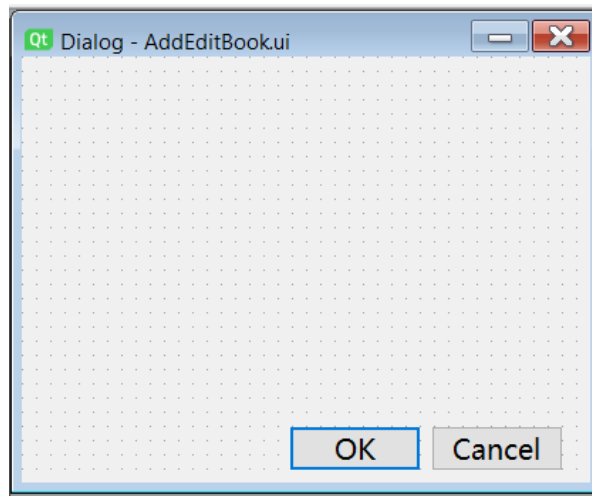
Проектирование главного окна завершено:



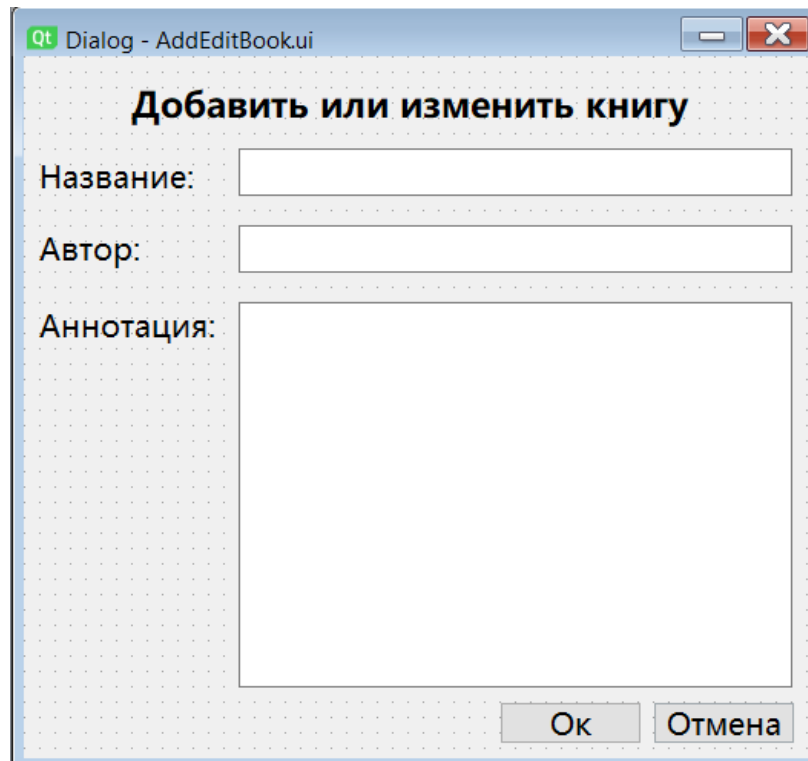
Для того, чтобы добавить или изменить книгу нужно ещё одно окно – создадим его. Это будет диалоговое окно – итогом будет либо подтверждение изменений пользователем, либо отказ. Для создания нового окна нужно выбрать «File» - «New» - «Dialog with Buttons Bottom»:



Будет создано следующее диалоговое окно, которое было сохранено под названием «AddEditBook», а также был задан общий шрифт, как у главного окна:



Для того, чтобы вводить данные можно воспользоваться двумя инструментами – Line Edit и Text Edit. Применим каждый из них: наименование книги и автора – объекты Line Edit (LE_bookname и LE_author соответственно), аннотация к книге будет объектом Text Edit (TE_annotation). Также удалите кнопки «Ok» и «Cancel» - вместо этого создадим свои собственные «Ok» (PB_ассепт) и «Отмена» (PB_cancel):



Скомпилируем код данного макета – для этого пропишем в cmd команду «*pyuic5 AddEditBook.ui -o AddEditBook_ui.py*».

Вместо того, чтобы каждый раз прописывать команды компиляции окон можете создать bat-файл в папке gui, который будет содержать все команды по рекомпиляции. Его можно называть, например, «recompile.bat»:

Имя	Дата изменения	Тип	Размер
__pycache__	27.04.2023 15:13	Папка с файлами	
AddEditBook.ui	02.05.2023 20:33	Файл "UI"	4 КБ
AddEditBook_ui.py	02.05.2023 21:13	Python File	3 КБ
MainWindow.ui	01.05.2023 21:07	Файл "UI"	3 КБ
MainWindow_ui.py	02.05.2023 21:13	Python File	4 КБ
recompile.bat	02.05.2023 21:13	Пакетный файл ...	1 КБ

```
recompile.bat x
1 pyuic5 MainWindow.ui -o MainWindow_ui.py
2 pyuic5 AddEditBook.ui -o AddEditBook_ui.py
3 pause
```

При двойном клике данный исполняемый файл будет заново перекомпилировать макеты в Python-код. Директива «pause» нужна для того, чтобы после выполнения компиляции окно сразу не закрывалось. При какой-либо ошибке компиляции будет полезно об этом узнать заранее.

12.3. Определение классов-наследников макетов окон

Теперь требуется создать класс, который будет наследовать признаки двух классов – QDialog и только что созданного AddEditBook_ui.Ui_Dialog. Вынесем этот класс в отдельный файл AddEditBook_window.py:

```
# Внешние библиотеки
from PyQt5 import QtWidgets

# Мои модули
from gui import AddEditBook_ui

class AddEditBookDial(QtWidgets.QDialog, AddEditBook_ui.Ui_Dialog):
    def __init__(self):
        super().__init__()
        self.setupUi(self)

        # Зададим действия у кнопок
        self.PB_accept.clicked.connect(self.accept)
        self.PB_cancel.clicked.connect(self.close)

    def accept(self):
        """
        Метод выполняет сохранение данных в отдельный файл
        :return:
        """
        bookname = self.LE_bookname.text()
        author = self.LE_author.text()
        annotation = self.TE_annotation.toPlainText()

        f = open('books.txt', 'a')
        f.write(f'Наименование:{bookname}\nАвтор:{author}\nАннотация:{annotation}\n---\n')
        f.close()
```

Теперь зададим код главного окна и точки входа, в нем зададим действие для кнопки добавить — должно быть вызвано окно класса **AddEditBookDial** с помощью метода **exec**:

```
# Стандартные библиотеки
import sys

# Внешние модули
from PyQt5 import QtWidgets

# Мои модули
from gui import MainWindow_ui
import AddEditBook_window

class RootApp(QtWidgets.QMainWindow, MainWindow_ui.Ui_MainWindow):
    def __init__(self):
        super().__init__()
        self.setupUi(self)

        self.PB_to_add.clicked.connect(self.to_add)

    def to_add(self):
        AddEditWindow = AddEditBook_window.AddEditBookDial()
        AddEditWindow.exec()

if __name__ == '__main__':
    app = QtWidgets.QApplication(sys.argv)
    window = RootApp()
    window.show()
    app.exec()
```

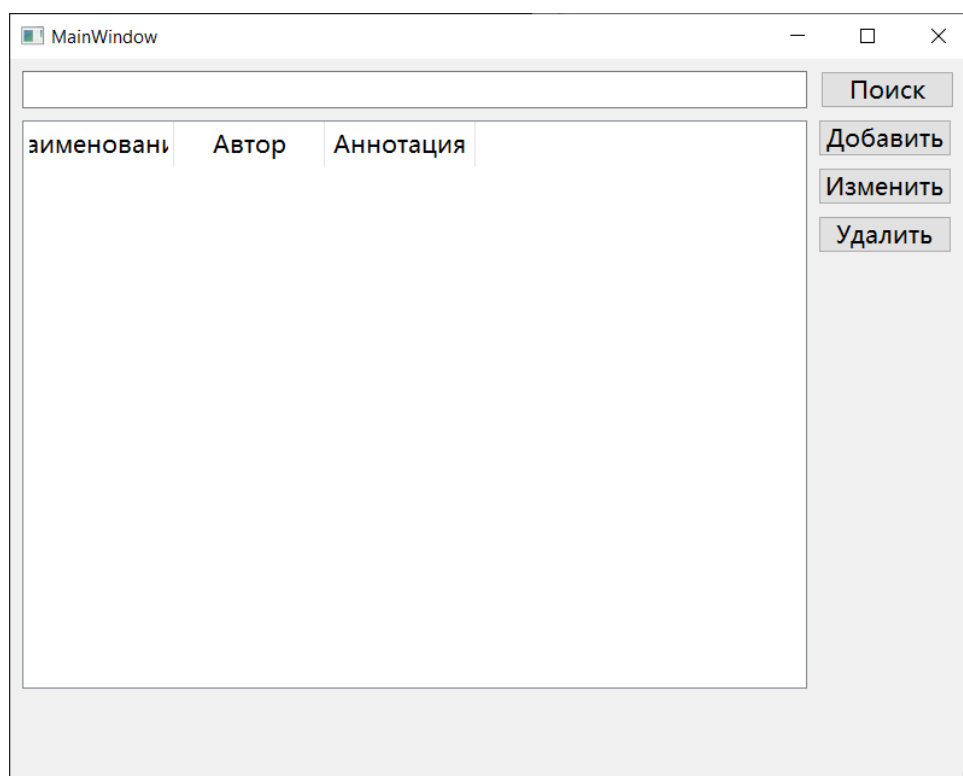
12.4. Запуск и тестирование работы проекта

Так как PyCharm не всегда верно определяет ошибки, связанные с работой Qt, то предлагаю вызывать работу приложения с помощью основного интерпретатора Python, а не виртуальной оболочки, для этого можем создать bat-файл «start.bat» в корне проекта:

Имя	Дата изменения	Тип	Размер
.idea	02.05.2023 21:16	Папка с файлами	
gui	02.05.2023 21:31	Папка с файлами	
venv	26.04.2023 13:38	Папка с файлами	
AddEditBook_window.py	02.05.2023 21:39	Python File	1 КБ
main.py	02.05.2023 21:50	Python File	1 КБ
start.bat	02.05.2023 21:53	Пакетный файл ...	1 КБ

```
start.bat x
1 main.py
2 pause
```

Запустите проект и проверьте работоспособность окон, при нажатии на кнопку добавить должно появляться соответствующее окно:



Dialog

Добавить или изменить книгу

Название:

Автор:

Аннотация:

Ок Отмена

При вводе некоторой информации и нажатии на «Ок» должен был создаться файл «books.txt»:

Dialog

Добавить или изменить книгу

Название:

Автор:

Аннотация:

Ок Отмена

Имя	Дата изменения
 .idea	02.05.2023 21:16
 __pycache__	02.05.2023 21:55
 gui	02.05.2023 21:31
 venv	26.04.2023 13:38
 AddEditBook_window.py	02.05.2023 21:39
 books.txt	02.05.2023 21:56
 main.py	02.05.2023 21:50
 start.bat	02.05.2023 21:53

 books.txt – Блокнот

Файл Правка Формат Вид Справка

Наименование: Тестовая книга
 Автор: Тестовый автор
 Аннотация: Тестовая аннотация

12.5. Доработка проекта – очистка текстовых полей

Заметим один недостаток – при нажатии на кнопку «Ок» поля остаются заполненными – исправим это в классе «AddEditBookDialog»:

```
def accept(self):
    """
    Метод выполняет сохранение данных в отдельный файл
    :return:
    """
    bookname = self.LE_bookname.text()
    author = self.LE_author.text()
    annotation = self.TE_annotation.toPlainText()

    f = open('books.txt', 'a')
    f.write(f'Наименование:{bookname}\nАвтор:{author}\nАннотация: {annotation}\n---\n')
    f.close()

    # Очистка полей для ввода:
    self.LE_bookname.setText('')
    self.LE_author.setText('')
    self.TE_annotation.setText('')
```

12.6. Работа с таблицей

Теперь следует разобраться с таблицей, которая должна отображать имеющиеся в «books.txt» книги. Зададим автоматический подбор ширины в таблице с помощью метода `setSectionResizeMode`, добавив следующие строки в конструктор главного окна:

```
class RootApp(QtWidgets.QMainWindow, MainWindow_ui.Ui_MainWindow):
    def __init__(self):
        super().__init__()
        self.setupUi(self)

        header = self.T_Books.horizontalHeader()
        header.setSectionResizeMode(0, QtWidgets.QHeaderView.ResizeToContents)
        header.setSectionResizeMode(1, QtWidgets.QHeaderView.ResizeToContents)
        header.setSectionResizeMode(2, QtWidgets.QHeaderView.ResizeToContents)

        self.PB_to_add.clicked.connect(self.to_add)
```

Добавим в RootApp новый метод – refresh_table, который будем вызывать в конструкторе:

```
class RootApp(QtWidgets.QMainWindow, MainWindow_ui.Ui_MainWindow):
    def __init__(self):
        super().__init__()
        self.setupUi(self)

        header = self.T_Books.horizontalHeader()
        header.setSectionResizeMode(0, QtWidgets.QHeaderView.ResizeToContents)
        header.setSectionResizeMode(1, QtWidgets.QHeaderView.ResizeToContents)
        header.setSectionResizeMode(2, QtWidgets.QHeaderView.ResizeToContents)

        self.refresh_table()

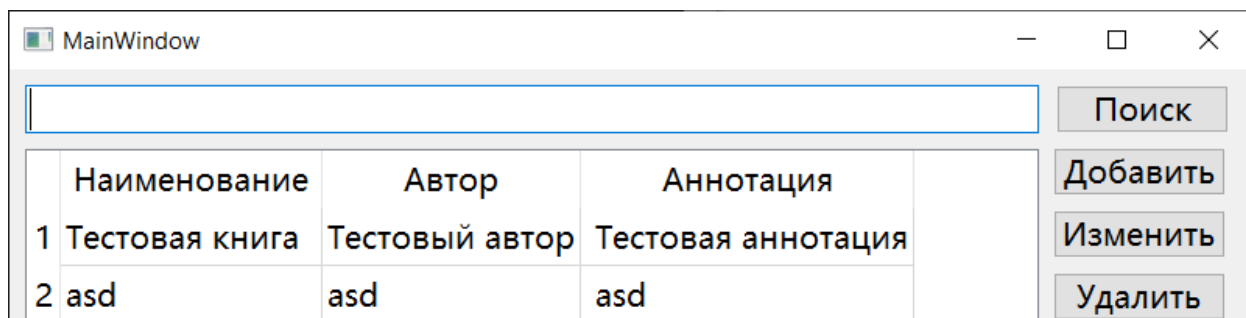
        self.PB_to_add.clicked.connect(self.to_add)
```

```
def refresh_table(self):
    f = open('books.txt', 'r')
    books_data = f.read().split('---\n')[:-1]
    f.close()

    # Зададим количество строчек в таблице
    self.T_Books.setRowCount(len(books_data))
    print(books_data)
    for index, data in enumerate(books_data):
        data_list = data.split('\n')
        bookname = data_list[0][len('Наименование:'):]
        author = data_list[1][len('Автор:'):]
        annotation = data_list[2][len('Аннотация:'):]

        # Структура setItem - № строки, № столбца, объект класса QTableWidgetItem
        self.T_Books.setItem(index, 0, QtWidgets.QTableWidgetItem(bookname))
        self.T_Books.setItem(index, 1, QtWidgets.QTableWidgetItem(author))
        self.T_Books.setItem(index, 2, QtWidgets.QTableWidgetItem(annotation))
```

При запуске программы таблица отобразит все данные из файла:



12.7. Поиск книг по ключевому слову

Теперь определим работу поиска – пусть при вводе некоторого слова он оставит записи книг, в которых присутствует данное слово в наименовании, авторе или же аннотации.

Алгоритм работы поиска таков – пусть программа читает файл, формирует на его основе список книг, в одном из полей которых присутствует ключевое слово и на основе этого списка строит таблицу. Код будет схож с методом **refresh_table**.

Доработаем конструктор – свяжем кнопку поиска **PB_to_search** с соответствующим методом **to_search**:

```
def __init__(self):
    super().__init__()
    self.setupUi(self)

    header = self.T_Books.horizontalHeader()
    header.setSectionResizeMode(0, QtWidgets.QHeaderView.ResizeToContents)
    header.setSectionResizeMode(1, QtWidgets.QHeaderView.ResizeToContents)
    header.setSectionResizeMode(2, QtWidgets.QHeaderView.ResizeToContents)

    self.refresh_table()

    self.PB_to_add.clicked.connect(self.to_add)
    self.PB_to_search.clicked.connect(self.to_search)
```

Создадим метод **to_search**, для более удобной фильтрации книг определим функцию **is_text_in_words**:

```
def to_search(self):
    f = open('books.txt', 'r')
    books_data = f.read().split('---\n')[:-1]
    f.close()

    search_word = self.LE_search_request.text()

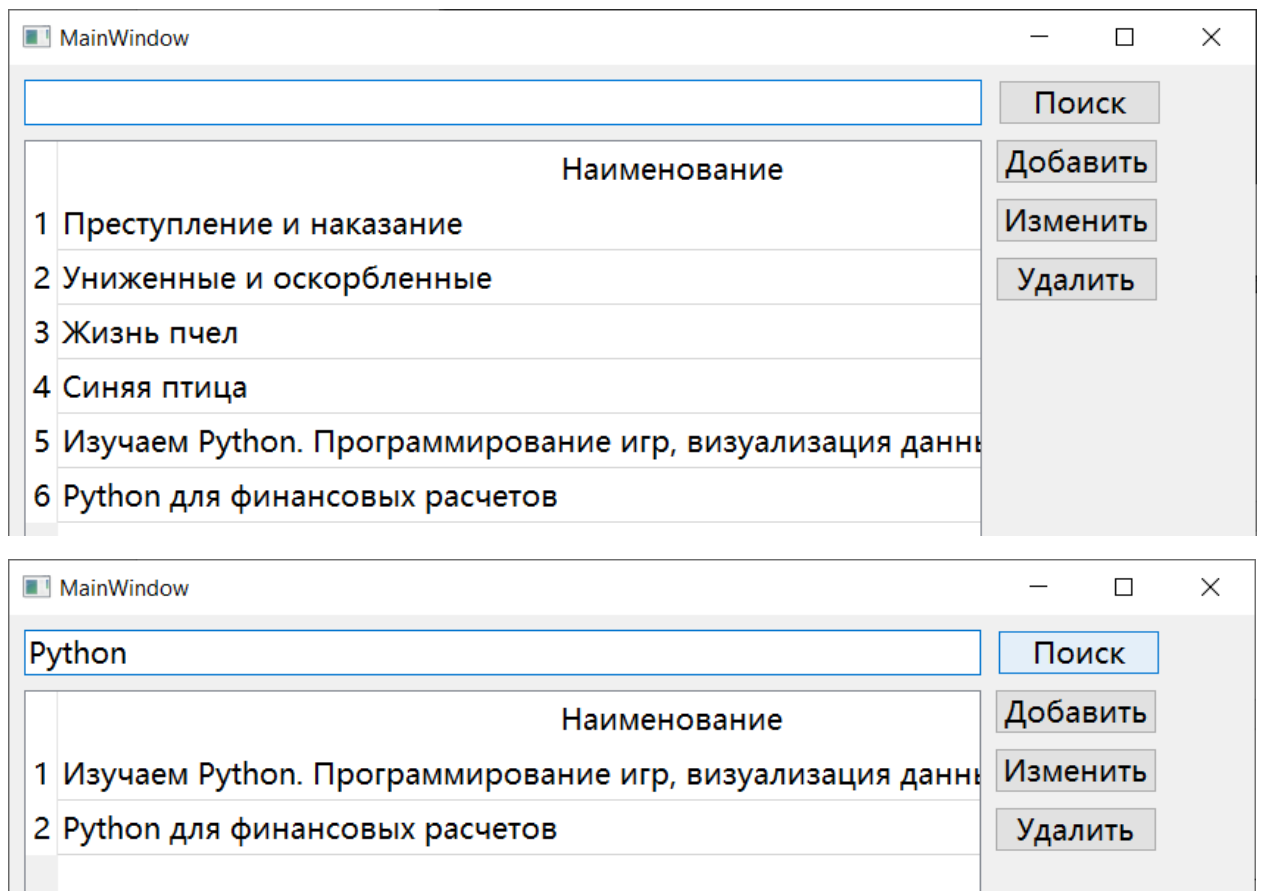
    books_list = []
    for data in books_data:
        data_list = data.split('\n')
        bookname = data_list[0][len('Наименование:'):]
        author = data_list[1][len('Автор:'):]
        annotation = data_list[2][len('Аннотация:'):]
        data_list = [bookname, author, annotation]

        # Определим функцию чтобы узнать есть ли слово в полях книги
        if is_text_in_words(search_word, data_list):
            books_list.append(data_list)

    # Сформируем таблицу на основе отфильтрованных записей
    self.T_Books.setRowCount(len(books_list))
    for index, data in enumerate(books_list):
        self.T_Books.setItem(index, 0, QTableWidgetItem(data[0]))
        self.T_Books.setItem(index, 1, QTableWidgetItem(data[1]))
        self.T_Books.setItem(index, 2, QTableWidgetItem(data[2]))
```

```
def is_text_in_words(text: str, words: list) -> bool:
    """
    Функция для определения слова в списке слов
    :param text:
    :param words:
    :return:
    """
    for word in words:
        if text in word:
            return True
    return False
```

Проверьте работу программы. Например, зададим в поиск книги, в которых присутствует слово «Python» с некоторым количеством записей:



12.8. Удаление элемента из таблицы

В ходе работы удаления элемента по кнопке удалим строку таблицы, а также исключим данную запись из текстового файла.

Добавим действие кнопки в конструктор:

```
def __init__(self):
    super().__init__()
    self.setupUi(self)

    header = self.T_Books.horizontalHeader()
    header.setSectionResizeMode(0, QtWidgets.QHeaderView.ResizeToContents)
    header.setSectionResizeMode(1, QtWidgets.QHeaderView.ResizeToContents)
    header.setSectionResizeMode(2, QtWidgets.QHeaderView.ResizeToContents)

    self.refresh_table()

    self.PB_to_add.clicked.connect(self.to_add)
    self.PB_to_search.clicked.connect(self.to_search)
    self.PB_to_delete.clicked.connect(self.to_delete)
```

Определим действие метода to_delete:

```
def to_delete(self):
    f = open('books.txt', 'r')
    books_data = f.read().split('---\n')[:-1]
    f.close()

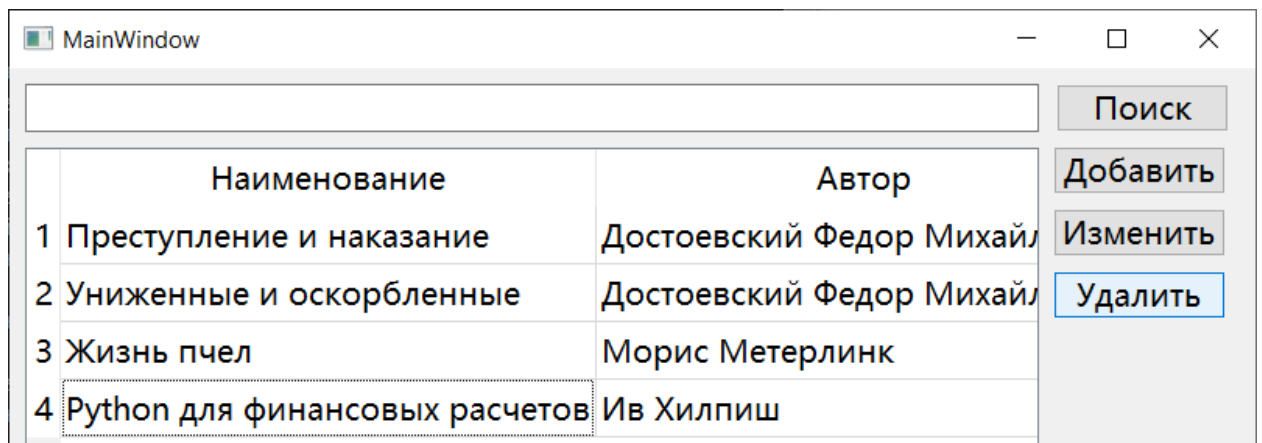
    books_list = []
    for data in books_data:
        data_list = data.split('\n')
        bookname = data_list[0][len('Наименование:'):]
        author = data_list[1][len('Автор:'):]
        annotation = data_list[2][len('Аннотация:'):]
        books_list.append([bookname, author, annotation])

    # Соберем номера всех строк, которые подлежат удалению
    rows_to_remove = []
    for child in self.T_Books.selectedIndexes():
        # child.row() - возвращает номер строки
        rows_to_remove.append(child.row())

    # При удалении номера строк будут меняться,
    # потому создадим переменную minus
    minus = 0
    for row in rows_to_remove:
        self.T_Books.removeRow(row-minus)
        books_list.pop(row-minus)
        minus += 1

    # Создание файла, в котором не будут удаленные книги
    f = open('books.txt', 'w')
    for book in books_list:
        f.write(f'Наименование:{book[0]}\nАвтор:{book[1]}\nАннотация:{book[2]}\n---\n')
    f.close()
```

Проверьте работоспособность кода – попробуйте удалить пару записей и удостоверьтесь, что эти изменения также отразились и на текстовом файле:



Изменения в текстовом файле должны быть:

```
books.txt – Блокнот
Файл  Правка  Формат  Вид  Справка
Наименование:Преступление и наказание
Автор:Достоевский Федор Михайлович
Аннотация:Роман Фёдора Михайловича Достоевского «Прес
---
Наименование:Униженные и оскорбленные
Автор:Достоевский Федор Михайлович
Аннотация:"Униженные и оскорбленные" - это не только (
---
Наименование:Жизнь пчел
Автор:Морис Метерлинк
Аннотация:Философское эссе Метерлинка «Жизнь пчел» – (
---
Наименование:Python для финансовых расчетов
Автор:Ив Хилпиш
Аннотация:Python стал языком выбора для разработки фи
```

Таким образом, используя фреймворк Qt мы добились удобного взаимодействия с базой данных книг посредством графического интерфейса.

12.9. Самостоятельная доработка приложения

В качестве самостоятельной работы требуется доработать приложение, а именно:

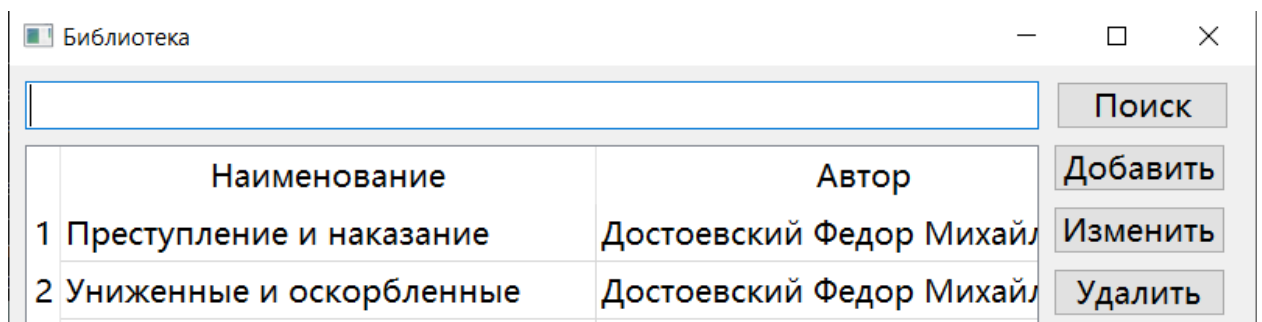
Задание 1. Программа избыточное количество раз читает файл «books.txt», было бы логичнее хранить книги в общей памяти.

Задание 2. При добавлении книги и закрытии окна таблица должна отражать новые добавленные книги.

Задание 3. Реализуйте работу кнопки «Изменить» - должно открываться окно «Добавить или изменить книгу» и пользователь сможет вносить изменения в существующую запись.

Если происходит добавление книги, должен отображаться только текст «Добавить книгу», если же происходит изменение существующей – «Изменить книгу». При изменении данных о книге и нажатии на «Ок» окно должно закрываться.

Задание 4. Измените наименование главного окна при работе программы с MainWindow на «Библиотека»:



Также у окна «AddEditBook» изменять название в зависимости от ситуации «Добавить книгу» или же «Изменить книгу».

Для динамической замены наименования применяется метод `setWindowTitle`.

13. Массивы и списки

Для начала, представим то, как хранятся данные программы в памяти. Например, вам нужно сдать два предмета в камеру хранения – зонт и шляпу. Для вас будут выделены две ячейки – оставляйте их там и идите по делам, что отображено на рисунке ниже (серыми клетками отображаются уже занятые ячейки другими предметами).



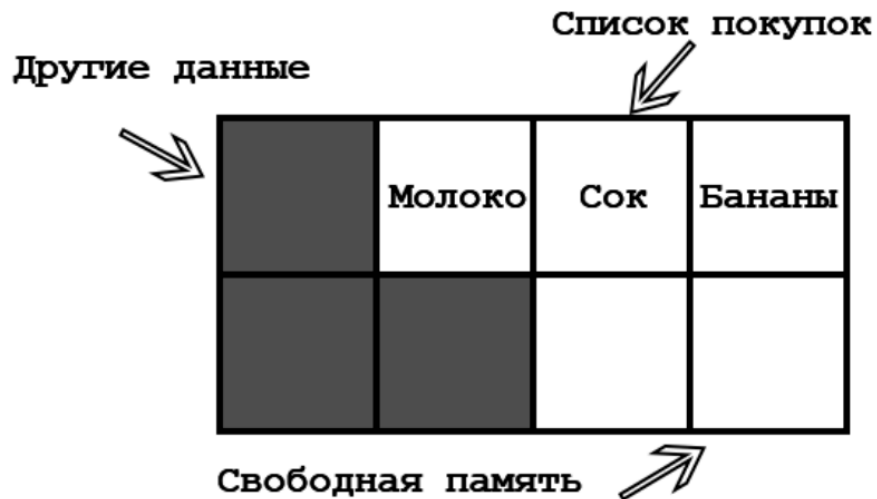
В общем смысле, именно так и работает память вашего компьютера. Она представляет собой нечто вроде огромной камеры с множеством ячеек, и при этом каждая ячейка имеет уникальный адрес.

Каждый раз, когда вы хотите сохранить в памяти отдельное значение, вы запрашиваете у компьютера место в памяти, а он выдает адрес для сохранения значения. Для хранения нескольких элементов, объединенных некоторым смыслом (последовательность чисел, список книг и т.п.), можно воспользоваться двумя основными способами: воспользоваться массивом или списком. Так как единственно верного способа сохранения данных на все случаи жизни не существует, следует знать, чем различаются эти структуры данных.

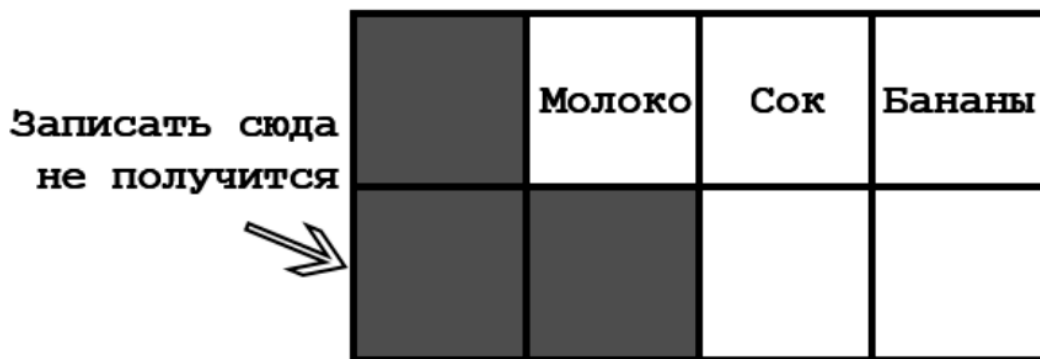
Пусть задача для разбора структур данных будет такова: требуется создать приложение для управления списком покупок. Для хранения данных для начала используем массив, т.е. все задачи будут храниться в памяти непрерывно (располагаться друг за другом).

13.1. Пример работы с массивом

Рассмотрим схематично как бы хранился в памяти список покупок из трех продуктов с помощью массива:



Оказалось, что мы забыли ещё один продукт, но ячейка следующая за бананами ячейка памяти уже занята, в неё нельзя уже ничего добавить.



В таком случае, нужно запросить другую область в памяти компьютера, где будут свободны четыре ячейки подряд и в них перезаписать все данные, включая новый продукт. При этом, если вы снова решите что-то добавить в список покупок – придется искать место для пяти подряд ячеек и так далее. Добавление в массив новых элементов может стать большой проблемой. Если свободного места нет и вам каждый раз приходится перемещаться в новую область в памяти, операция добавления нового элемента будет выполняться очень медленно.

Самым простым выходом из ситуации было бы заранее выделить несколько свободных ячеек для продуктов, о которых можно вспомнить позднее. Так, если список состоит из 3 покупок, сразу можно было бы запросить место для 10. Тогда продуктов можно было бы добавлять по 10 и никаких дополнительных операций производить не нужно. У такого решения проблемы есть несколько недостатков:

- **неэффективное использование памяти** - не всегда выделенное лишнее место может быть нужно, таким образом память будет

расходиться неэффективно. Не только вы не используете эту память, но её не может использовать и любая другая программа;

- **непостоянное решение** - как только список покупок станет больше 10, пересоздавать массив все же придется.

Такой подход нельзя назвать идеальным, связанные списки решают проблему добавления новых элементов.

13.2. Пример работы со связанным списком

Связанные списки позволяют размещать свои элементы в любом удобном месте памяти. Таким образом, на схеме продукты теперь будут располагаться среди свободной и занятой памяти раздельно:

	Молоко		
		Сок	
Бананы			

Каждый элемент связанного списка хранит в себе информацию о том, по какому адресу располагается следующий элемент. Таким образом, набор произвольных адресов памяти объединяется в цепочку.



Пример адреса, по которому может храниться элемент в памяти:
0x0000023A1F584700.

Добавить новый элемент в связанный список куда проще, чем в массив – достаточно сохранить элемент в любую незанятую ячейку и сообщить, чтобы последний на данный момент элемент списка стал указывать на новый.

Выпишем интересующие нас данные, а также их адреса:

Продукт	Адрес
Молоко	2
Сок	7
Бананы	9

Теперь несложно структуру связанного списка записать с помощью цепочки из адресов: «2 → 7 → 9». При добавлении нового элемента запись может стать такой «2 → 7 → 9 → 12».

Но что, если потребуется вставить элемент не в конец, а куда-нибудь посередине? Пусть новый элемент стоит после первого, тогда запись станет: «2 → 12 → 7 → 9». В связанных списках это делается легко, достаточно было сказать ячейке 2 указывать теперь на 12, вместо 7, а также сообщить 12 ячейке указывать на 7. В массиве бы пришлось выполнять сдвиг всех последующих элементов для освобождения места под вставляемые данные, при условии наличия свободных в массиве ячеек.

Удаление происходит также просто – удалим из связанного списка Сок (находится по адресу 7), тогда из исходной цепочки «2 → 7 → 9» нужно

просто сделать так, чтобы 2 указывала теперь на 9, т.е. «2 → 9». В массиве бы пришлось снова выполнять сдвиг элементов.

Также решается ещё одна проблема – если при использовании массива в памяти невозможно найти подряд свободных, например, 1000 ячеек, при этом тысяча свободных ячеек имеется, разбросанных по всей памяти, то вы сможете сохранить данные с помощью связанного списка.

Но у связанного списка есть и недостатки – разберем операцию поиска элемента:

- в массиве, зная адрес начального элемента, например, 15 несложно узнать по какому адресу находится четвертый – на 18;
- в связанном списке придется пройти от самого начала (головы) по адресам до искомого элемента.

13.3. Оценка работы операций над структурами данных

Сложность работы алгоритма можно выразить с помощью O («о» большое), что говорит о количестве выполняемых операций для получения результата в самом худшем (максимально трудозатратном) случае.

Структура данных	Массив	Связанный список
Чтение	$O(1)$	$O(n)$
Вставка	$O(n)$	$O(1)$
Удаление	$O(n)$	$O(1)$

Таким образом, массивы и списки имеют свои недостатки и преимущества. Массивы обеспечивают произвольный доступ к данным, в то время как списки – последовательный. В зависимости от того, что в задаче важнее программист и станет выбирать определенную структуру данных.

13.4. Сущность списков в Python

Разобравшись получше в вопросах работы таких структур данных возникает вопрос – «А что же такое список в Python?». Можно выделить следующие свойства списка:

- произвольный доступ к элементам;

- при добавлении/вставке/удалении элементов не нужно задумываться про выделение памяти и сдвиги элементов.

В отличие от языка LISP, Python при реализации списка не использует связанные списки, на самом деле это непрерывный массив ссылок на другие объекты. Таким образом, индексирование в списке стоит $O(1)$.

Рассмотрим реализацию работы списка на примере CPython – самой распространенной реализации языка Python, написанный на языке C, он представляет из себя следующую структуру:

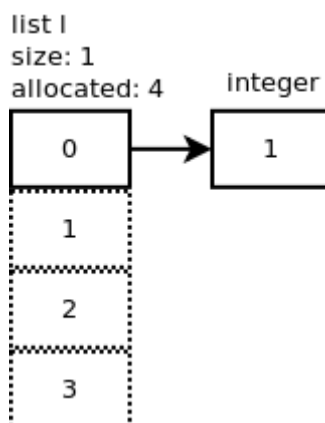
```
typedef struct {
    PyObject_VAR_HEAD
    PyObject **ob_item;
    Py_ssize_t allocated;
} PyListObject;
```

Поле `ob_item` является массивом указателей на элементы списка. Поле `allocated` говорит о том, сколько зарезервировано места под список.

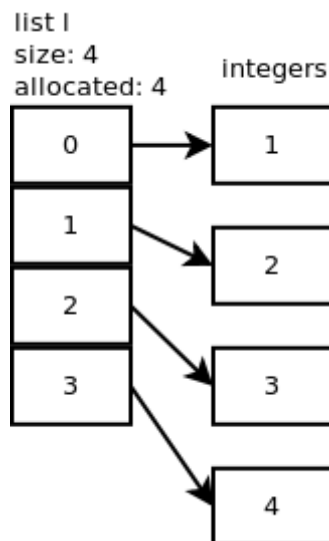
13.4.1. Добавление в список (append)

По мере заполнения списка будет выделяться дополнительная память в следующем порядке: 0, 4, 8, 16, 25, 35, 46, 58, 72, 88 и т.д..

Создадим список, на данный момент в нём ноль элементов, а значит и памяти выделено 0. При добавлении одного элемента, например, числа 1, будет сразу выделено 4 ячейки. Сложность добавления нового элемента в такую структуру данных всего $O(1)$.

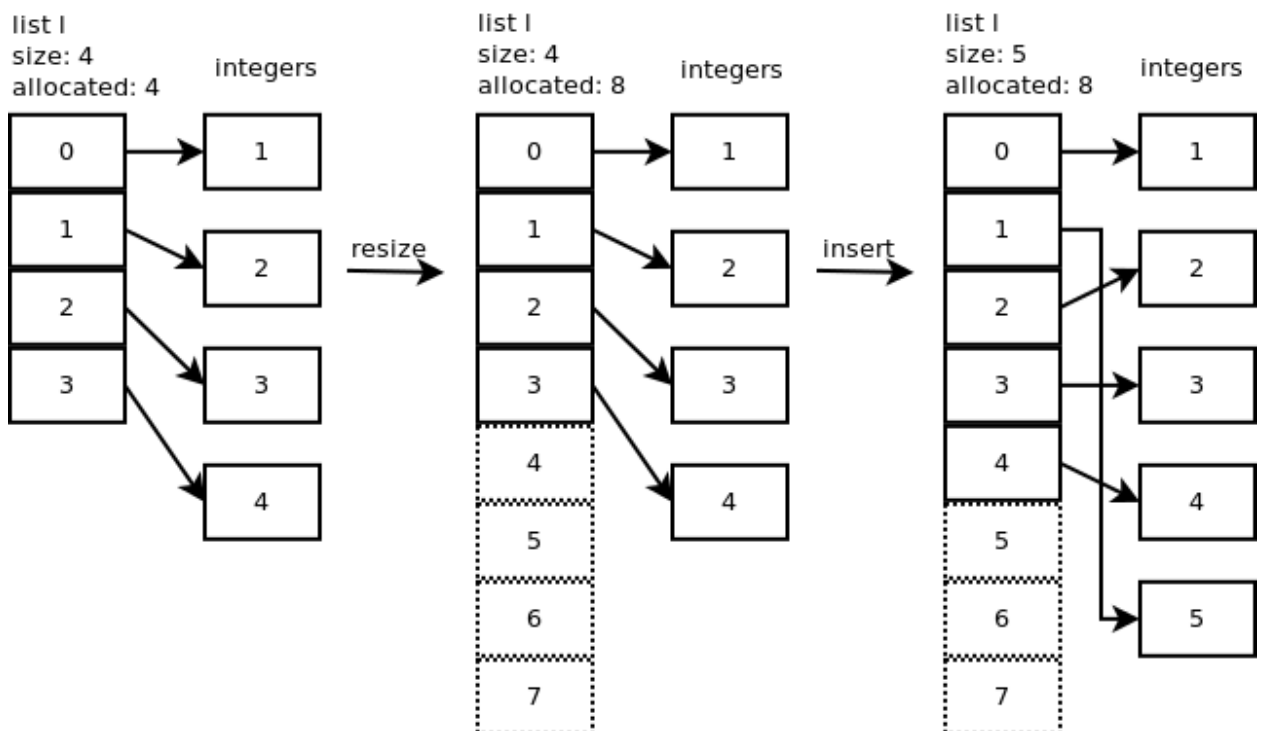


Добавим ещё три элемента и убедимся – нет нужды в выделении новой памяти, хватает выделенной ранее:



13.4.2. Вставка в список (insert)

Добавим новое число – 5 на место по индексу 1 и посмотрим что произойдет:

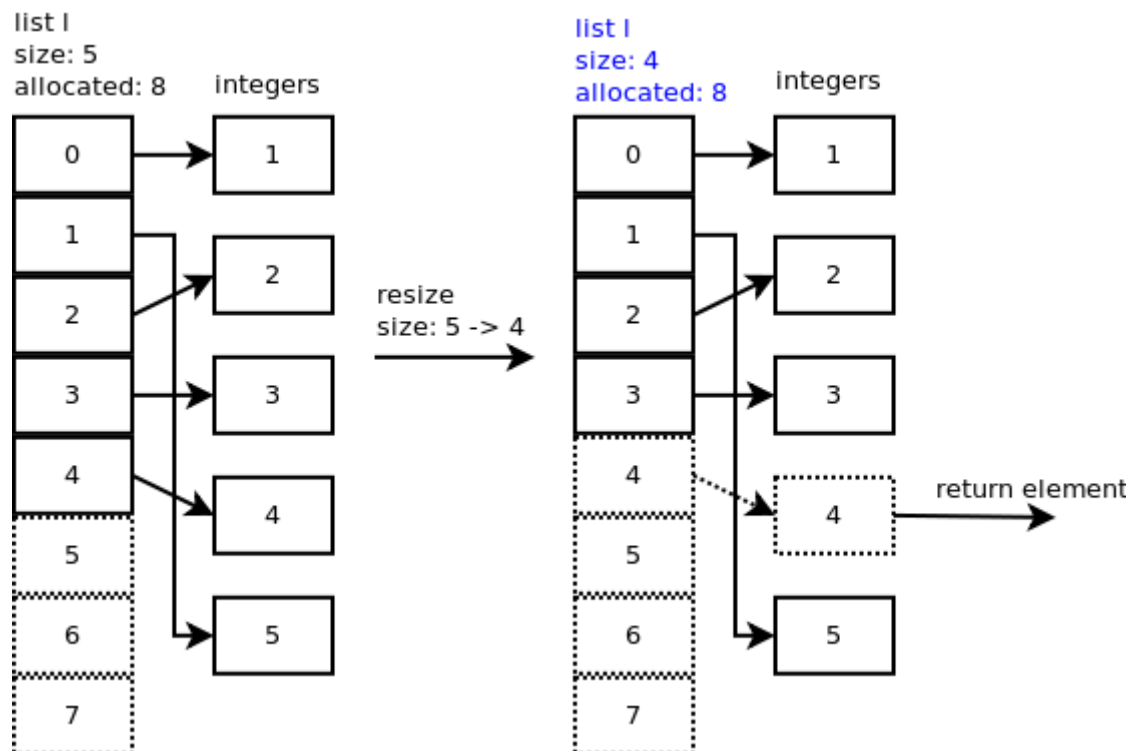


Таким образом, во время добавления нового элемента для начала выделяется новая память – теперь массив состоит из 8 ячеек для записи, а затем произойдет сдвиг – теперь адрес по индексу 1 указывает на значение 5, а остальные сдвинулись.

Сложность вставки – $O(n)$.

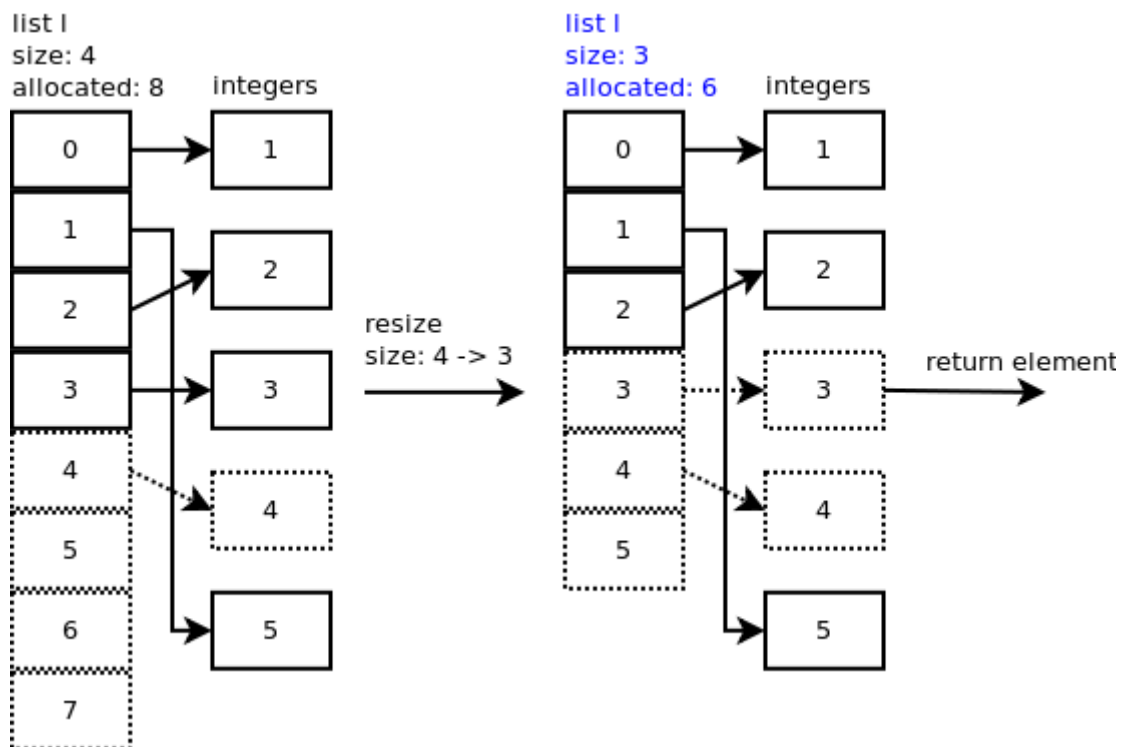
13.4.3. Удаление из конца списка

При использовании метода pop() без аргументов произойдет удаление последнего элемента списка – рассмотрим что произойдет при этом:



Размер списка стал 4, выделенная память все равно на 8 ячеек. На самом деле, по индексу 4 всё равно хранится указатель на число 4, но теперь это недостижимые для программиста данные.

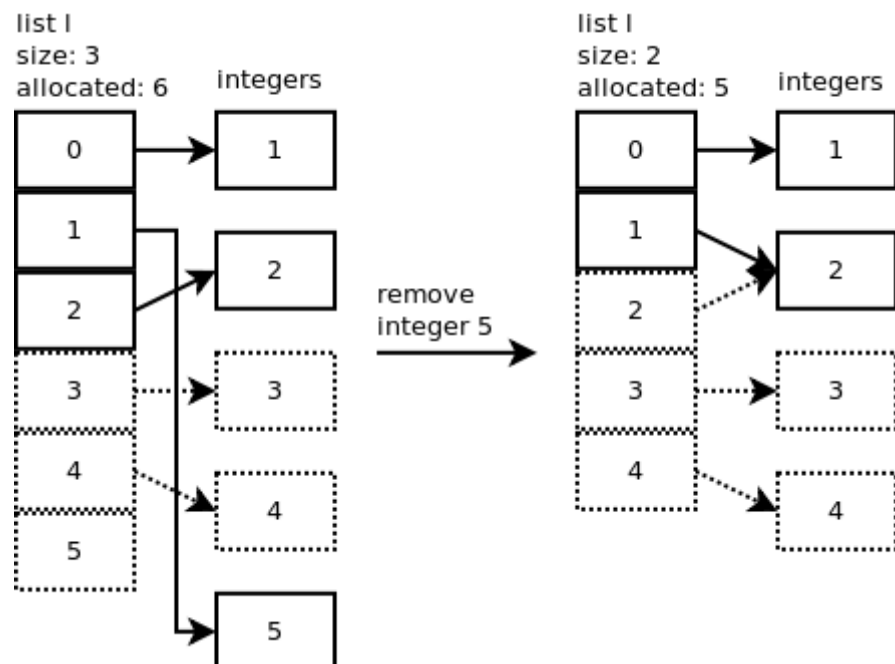
При удалении ещё одного элемента размер списка станет 3, что меньше половины выделенной памяти – поэтому массив уменьшится и будет размером в 6 ячеек:



Сложность удаления из конца списка – $O(1)$.

13.4.4. Удаление по индексу из списка

Рассмотрим также работу при удалении из любого другого места списка, кроме как его конца – удалим элемент по индексу 1 – число 5:



Так как ячейка 1 теперь ссылается на число 2, то на 5 ссылаться ничего не может – оно удалено из памяти, размер выделенной на массив памяти также уменьшился.

Если же добавить ещё один элемент, то будет выделена память под 4 ячейки и всего их будет 8.

13.5. Резюме

В данной главе мы познакомились с различными структурами данных – массивами и связанными списками, а также разобрались во внутренней структуре работы списков в Python, которые, по своей сути, совместили в себе логику работы как массивов, так и связанных списков.

Ниже представлены характеристики работы списков в Python:

Операция	Сложность
Поиск	$O(1)$
Добавление в конец	$O(1)$
Вставка	$O(n)$
Удаление с конца	$O(1)$
Удаление по индексу	$O(n)$

14. Научное программирование. Библиотека Matplotlib

Под **научным программированием** будем понимать применение языков программирования для научных расчетов, построения графиков и т.п..

В рамках ознакомления с научным программированием в Python будут использованы библиотеки Matplotlib и NumPy, так как они не входят в стандартный пакет библиотек, то их следует установить с помощью pip, пропишите в консоли следующие команды:

```
pip install matplotlib
pip install numpy
```

Библиотека Matplotlib позволяет отображать различную информацию в виде графиков и схем. Получаемые изображения можно выводить на экран, использовать в графических интерфейсах, а также сохранять в отдельные файлы для использования в публикациях.

В данной главе будет обзор создания простых схем в интерактивном сеансе с помощью модуля Matplotlib – pyplot. Для использования в Python модуля pyplot обычно пишут следующую директиву:

```
import matplotlib.pyplot as plt
```

14.1. Точечные диаграммы и линейные графики

Для того, чтобы отобразить набор точек (создать точечную диаграмму), определяемых координатами x и y можно воспользоваться функцией `plt.scatter`.

Опробуем его – создадим два списка – координаты по x и y точек на плоскости, которые генерируются с помощью библиотеки `random` и затем передаются `plt.scatter`:

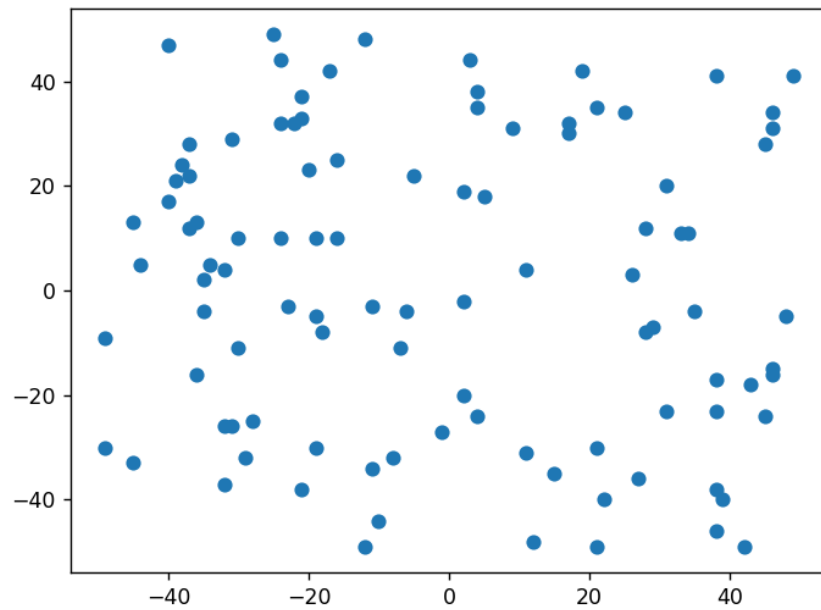
```
import random
import matplotlib.pyplot as plt

ax = []
ay = []

for i in range(100):
    ax.append(random.randint(-50, 50))
    ay.append(random.randint(-50, 50))

# Создание объекта диаграммы
plt.scatter(ax, ay)
# Вывод на экран
plt.show()
```

Результатом работы программы будет следующая точечная диаграмма (т.н. диаграмма рассеяния):



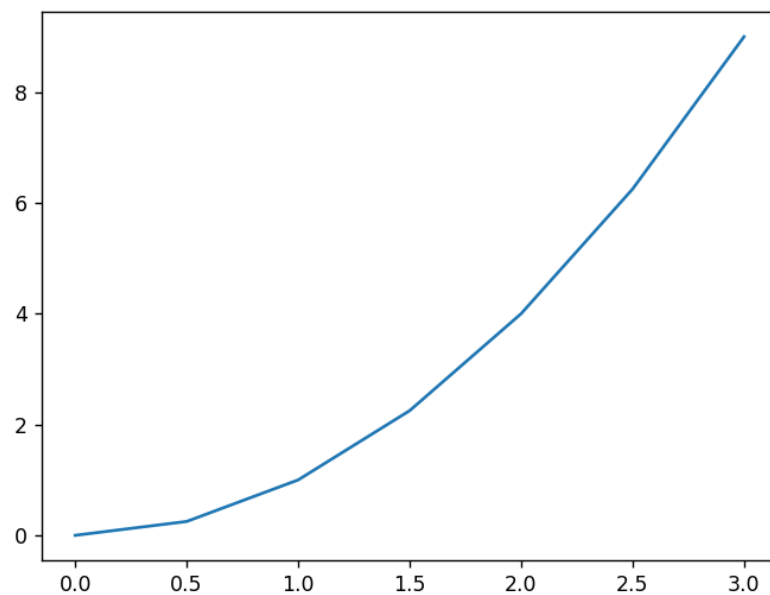
Для создания линейного графика (x,y) можно воспользоваться функцией `plt.plot`, аргументами к которой могут быть два списка – значения по x и значения по y .

Построим линейный график:

```
ax = [0, 0.5, 1, 1.5, 2, 2.5, 3]
ay = [0, 0.25, 1, 2.25, 4, 6.25, 9]
```

```
plt.plot(ax, ay)
plt.show()
```

Результат работы программы:



Для того, чтобы сохранить диаграмму можно воспользоваться функцией `plt.savefig(<наименование файла>)`.

Сохраните точечную диаграмму или же линейный график в различных форматах с помощью команд:

```
plt.savefig('plot.png')
plt.savefig('plot.jpg')
plt.savefig('plot.pdf')
```

Важно: при вызове `plt.show` фигуры будут отображены на экране и затем стерты, потому сохранять изображение нужно **перед** выводом графика/диаграммы на экран.

Попрактикуемся:

Рассмотрим построение графика функции $y = \sin^2 x$ при $-2\pi \leq x \leq 2\pi$.

Вычисляется и отображается 1000 точек (x, y) , координаты которых сохраняются в списках `ax` и `ay`. Для того, чтобы определить изменение x нужно вычислить шаг Δx :

$$\Delta x = (x_{\max} - x_{\min}) / (n - 1)$$

Зная шаг, точки абсцисс можно рассчитать по формуле:

$$x_i = x_{\min} + i \cdot \Delta x \text{ при } i = 0, 1, 2, \dots, n - 1.$$

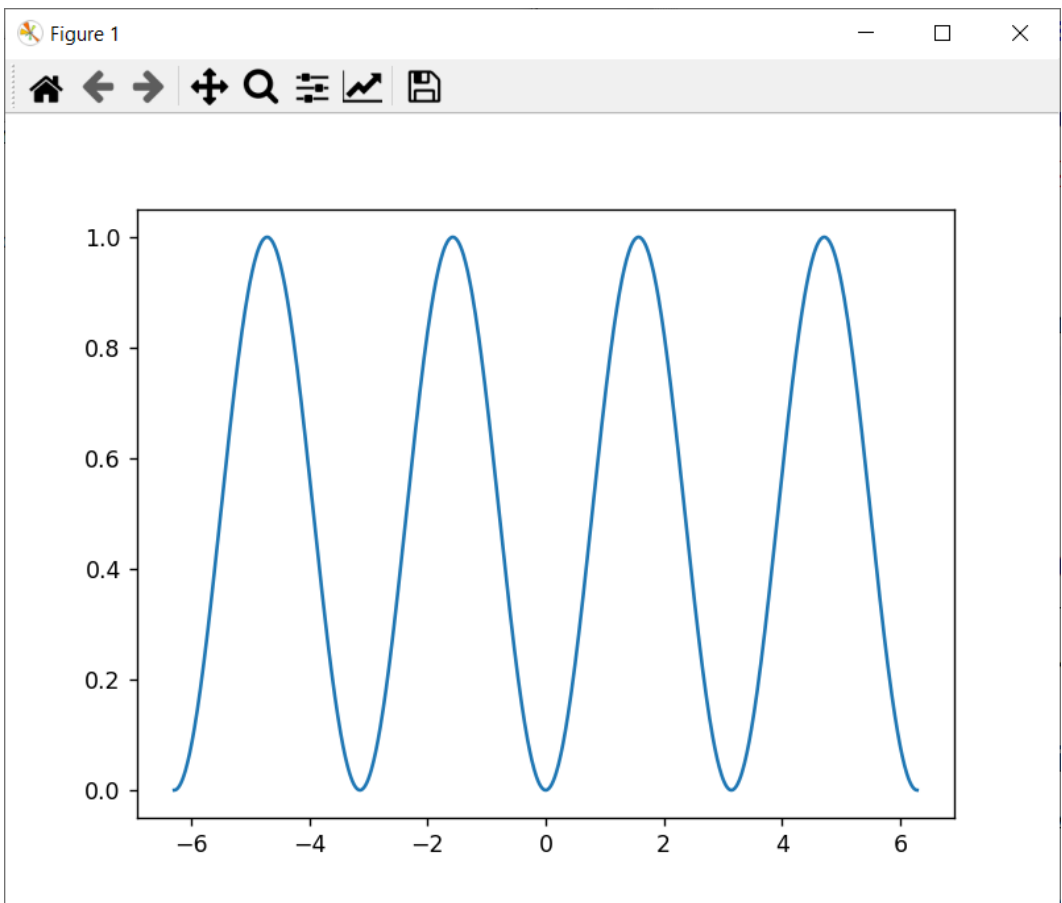
Соответствующие координаты y вычисляются по формуле

$$y_i = \sin^2(x_i).$$

```
import math
math.pi
math.sin
```

```
import numpy as np
np.pi
np.sin
```

Должно получиться следующее изображение:



14.2. Настройка параметров графика

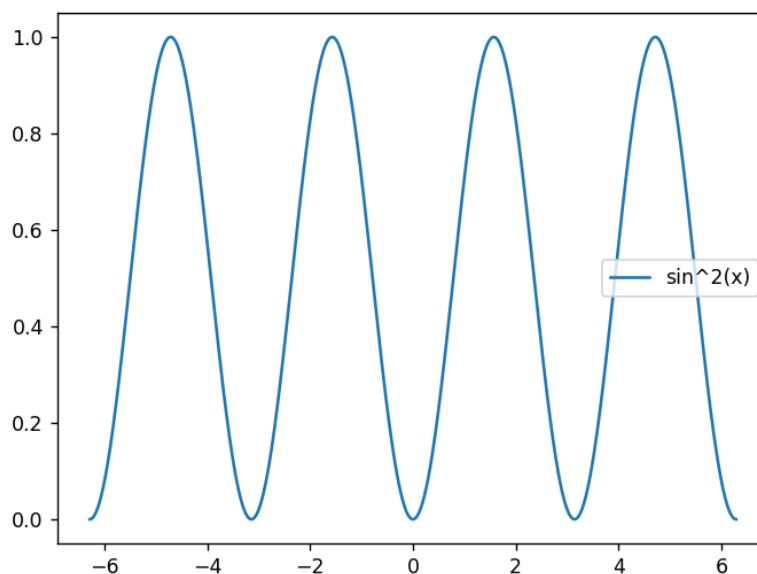
14.2.1. Описание графика функции

Для каждой линии на графике можно присвоить описание графику функции с помощью параметра **label**. Для того, чтобы отобразить описания на графике, требуется перед функцией `plt.show` вызвать функцию **plt.legend**.

Доработаем код, который был решен в практике выше – для этого в функции `plt.plot` передадим аргумент параметру `label` и вызовем функцию `plt.legend`:

```
plt.plot(ax, ay, label='sin^2(x)')  
plt.legend()  
plt.show()
```

Результатом работы программы будет:



i. Расположение описания графика функции

По умолчанию описание графика будет располагаться посередине справа. Для изменения положения описания нужно воспользоваться параметром `loc` функции `legend`, передавая либо строковое, либо целочисленное значение. Таблица с возможными аргументами для параметра `loc` приведена ниже:

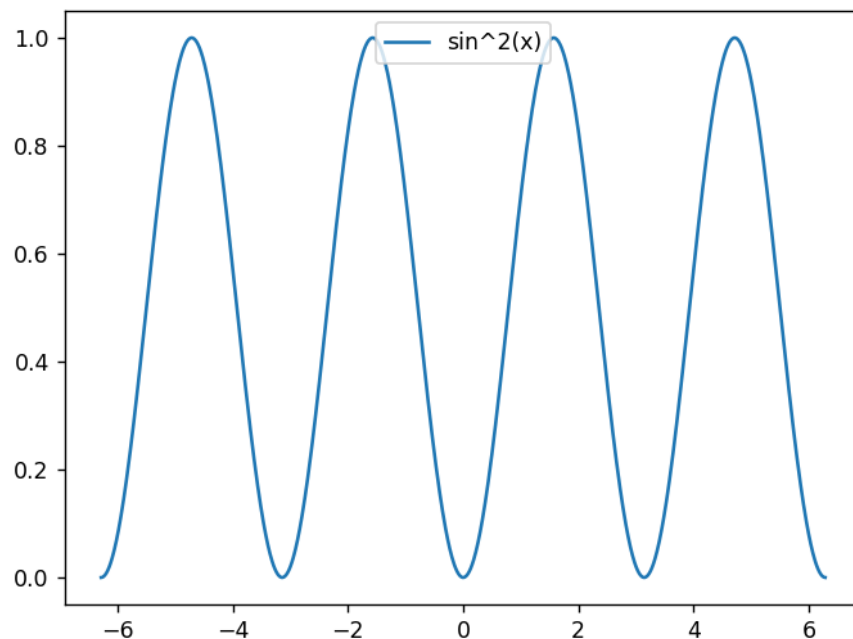
Расположение описания	Строка	Целое число
-----------------------	--------	-------------

Наилучшее расположение по мнению библиотеки	'best'	0
Посередине справа	'right', 'center right'	5, 7
Сверху справа	'upper right'	1
Сверху слева	'upper left'	2
Снизу слева	'lower left'	3
Снизу справа	'lower right'	4
Посередине слева	'center left'	6
Посередине снизу	'lower center'	8
Посередине сверху	'upper center'	9
Посередине	'center'	10

Доработаем программу выше, сменим расположение описания, чтобы оно было посередине сверху:

```
plt.plot(ax, ay, label='sin^2(x)')
plt.legend(loc=9)
plt.show()
```

Таким образом, получим:



14.2.2. Название графика и метки осей

Графику можно присвоить название, расположенное выше осей координат, с помощью вызова функции `plt.title` и передачи в него строки названия. Методы `plt.xlabel` и `plt.ylabel` управляют метками осей x и y : нужно

просто передать метку как строку в эти методы. Необязательный дополнительный параметр `fontsize` устанавливает размер шрифта в пунктах. Например, приведенный ниже код генерирует график, показанный на рис. 3.5.

```
import matplotlib.pyplot as plt
import numpy as np

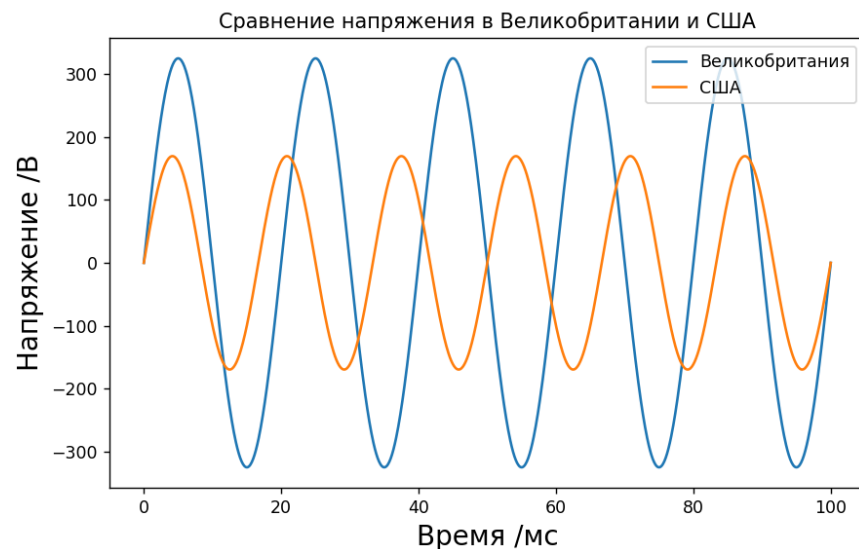
t = np.linspace(0, 0.1, 1000)

# Максимальное напряжение
Vp_uk = 230 * np.sqrt(2)
Vp_us = 120 * np.sqrt(2)

f_uk = 50
f_us = 60

V_uk = Vp_uk * np.sin(2 * np.pi * f_uk * t)
V_us = Vp_us * np.sin(2 * np.pi * f_us * t)

plt.plot(t*1000, V_uk, label='Великобритания')
plt.plot(t*1000, V_us, label='США')
plt.title('Сравнение напряжения в Великобритании и США')
plt.xlabel('Время /мс', fontsize=16)
plt.ylabel('Напряжение /В', fontsize=16)
plt.legend()
plt.show()
```



14.2.3. Расширенная настройка графиков

При отрисовке графиков можно настроить практически все графические элементы matplotlib – форму и цвет точек диаграммы рассеивания, а также различные параметры линий графиков функций.

Integer numbers from 0 to 11 create lines and triangles. Those are equally accessible via capitalized variables, like CARETDOWNBASE. Hence the following are equivalent:

```
plt.plot([1, 2, 3], marker=11)
plt.plot([1, 2, 3], marker=matplotlib.markers.CARETDOWNBASE)
```

https://matplotlib.org/stable/api/markers_api.html

Цвета

Цвет выводимой линии и/или ее маркеров можно определить с помощью аргумента color. Поддерживается несколько форматов определения цвета. Во-первых, однобуквенные коды для некоторых основных цветов приведены



в табл. 3.3. Например, color='r' определяет красную линию и маркеры. Эти цвета слишком яркие, поэтому (с версии Matplotlib 2.0) для последовательности линий на одном графике по умолчанию установлены более мягкие группы цветов Tableau (живой цвет), строки идентификаторов которых также приведены в табл. 3.3.

Таблица 3.3. Буквенные и строковые обозначения цветов в Matplotlib

Коды основных цветов	Живые (Tableau) цвета
b = синий	tab:blue
g = зеленый	tab:orange
r = красный	tab:green
c = бирюзовый	tab:red
m = фиолетовый	tab:purple
y = желтый	tab:brown
k = черный	tab:pink
w = белый	tab:gray
	tab:olive
	tab:cyan

Кроме того, можно определять оттенки серого цвета как строку, представляющую число с плавающей точкой `float` в диапазоне 0–1 (0. соответствует черному цвету, 1. – белому). Шестнадцатеричные строки HTML, определяющие красную, зеленую и синюю (RGB) компоненты цвета в диапазоне 00-ff, также могут передаваться в аргументе `color` (например, `color='#ff00ff'` определяет фиолетовый (magenta) цвет). Наконец, компоненты RGB можно передавать как кортеж (tuple) из трех значений в диапазоне 0–1 (например, `color=(0.5, 0., 0.)` – это темно-красный цвет).

#2ab735

Стили и ширина линий

По умолчанию принят стиль для графика: сплошная линия шириной 1.5 пункта. Для настройки этого параметра определяется аргумент `linestyle` (это тоже строка). Некоторые возможные значения, определяющие стиль линии, приведены в табл. 3.4.

Таблица 3.4. Стили линий Matplotlib

Код	Стиль линии
-	Сплошная
--	Штриховая
:	Пунктирная
-.	Штрихпунктирная

Чтобы линии вообще не отображались, устанавливается значение `linestyle=' '` (пустая строка). Толщину линии можно задать в пунктах, передавая значение типа `float` в атрибуте `linewidth`. Например:

```

x = np.linspace(0.1, 1., 100)
yi = 1. / x
ye = 10. * np.exp(-2 * x)
plt.plot(x, yi, color='r', linestyle=':', linewidth=4.)
plt.plot(x, ye, color='m', linestyle='--', linewidth=2.)
plt.show()

```

Результат выполнения этого кода показан на рис. 3.7.

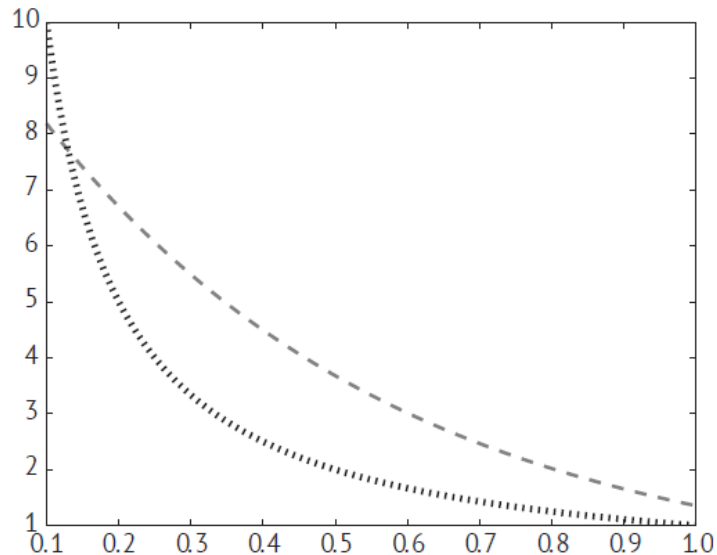


Рис. 3.7. Два различных стиля линий на одном графике

Кроме того, допустимыми являются следующие сокращенные обозначения свойств линий графика:

- `c` – `color` – цвет;
- `ls` – `linestyle` – стиль линии;
- `lw` – `linewidth` – ширина линии.

Например:

```
plt.plot(x, y, c='g', ls='--', lw=2)          # Утолщенная зеленая штриховая линия.
```

Также возможно определение цвета, стиля линии и маркера в одной строке:

```
plt.plot(x, y, 'r:^')                        # Красная пунктирная линия с треугольными маркерами.
```

Наконец, можно изобразить несколько линий, используя последовательность аргументов `x`, `y`, `format`:

```
plt.plot(x, y1, 'r--', x, y2, 'k-.')
```

Выводится красная штриховая линия для $(x, y1)$ и черная штрихпунктирная линия для $(x, y2)$.

Границы графика

Методы `plt.xlim` и `plt.ylim` определяют границы графика по осям x и y соответственно. Они должны вызываться только после всех инструкций `plt.plot`, но перед выводом или сохранением изображения. Например, следующий код создает график по предоставленным последовательностям данных между выбранными границами (см. рис. 3.8):

```
t = np.linspace(0, 2, 1000)
f = t * np.exp(t + np.sin(20*t))
plt.plot(t, f)
plt.xlim(1.5, 1.8)
plt.ylim(0, 30)
plt.show()
```

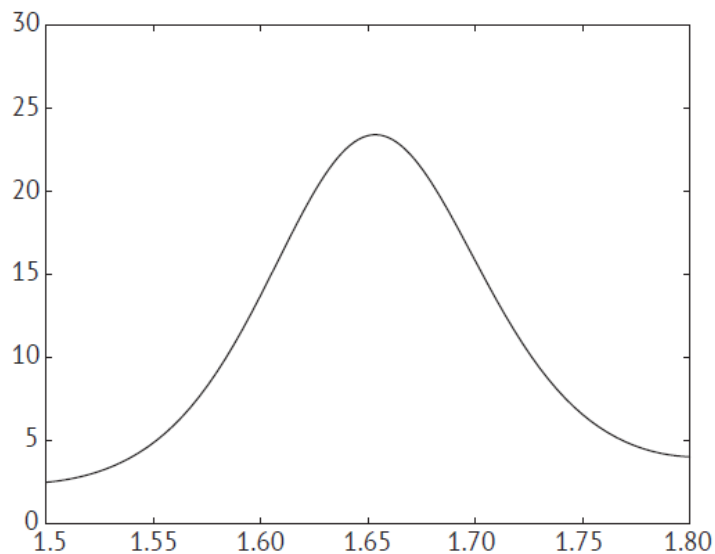


Рис. 3.8. График, созданный с явно определенными границами данных

15. Библиотека NumPy

Библиотека NumPy (от англ. Numerical Python – «численные методы на Python») является одной из стандартных библиотек, используемых в научном программировании на Python. NumPy добавляет поддержку больших многомерных массивов и матриц, а также предоставляет большую библиотеку высокоуровневых математических функций для операций над этими массивами.

Основой NumPy является объект класса `ndarray` – многомерный массив данных одного типа, у которого можно менять форму, производить сортировку, а также массив может быть операндом в математических операциях и статистическом анализе.

Реализации NumPy таких математических операций и алгоритмов обладают двумя главными преимуществами над основными объектами языка Python, которые мы использовали до сих пор. Во-первых, они реализованы как предварительно скомпилированный код на языке C, поэтому обеспечивают скорость выполнения, сравнимую со скоростью программы, написанной на C. Во-вторых, библиотека NumPy поддерживает векторизацию: единственная операция может быть выполнена над всем массивом в целом вместо применения явного цикла для последовательной обработки элементов массива.

Например, сравним операцию умножения двух одномерных списков `x` и `y` из `n` чисел в обычном языке Python:

```
z=[]
for k in range(n):
    z.append(x[k] * y[k])
```

Используя NumPy, эта же операция займет всего одну строчку:

```
z = x * y
```

Математический подход к синтаксису работы с матрицами, а именно отсутствие явной конструкции цикла и индексирования делает код более ясным, в меньшей степени подверженным ошибкам и приближает его к соответствующему стандартному математическому формату записи.

Для работы с библиотекой NumPy её следует подключить с помощью директивы `import`, также принято назначать библиотеке псевдоним `np` для более удобного обращения к ней:

```
import numpy as np
```

15.1. Массивы в NumPy

Классом массива NumPy является `ndarray`, который состоит из многомерной таблицы элементов, индексируемых кортежем целых чисел. В отличие от списков и кортежей Python, элементы не могут принадлежать к различным типам: каждый элемент в массиве NumPy имеет один и тот же тип, который определяется соответствующим объектом типа данных (`dtype`). Тип `dtype` массива определяет как общий класс элемента (`int`, `float`, `complex` и т.д.), так и его представление в памяти (сколько байт в памяти выделяется на элемент).

Измерения массива NumPy называются осями (axes), а количество измерений - рангом (rank). Не следует путать ранг массива в библиотеке NumPy с концепцией ранга матрицы в линейной алгебре.

15.1.1. Создание простого массива

Создадим массив, используя средства библиотеки NumPy, для этого вызовем функцию `array`:

```
# Создание массива целых чисел
arr_1 = np.array([23, 56, 223, -11, 0])
print(arr_1)

# Создание многомерного массива вещественных чисел
arr_2 = np.array([[-1.5, 2.33, -5.], [-1.5, -4, -6.2]])
print(arr_2)
```

Результат работы программы:

```
[ 23  56 223 -11   0]
[[-1.5  2.33 -5.  ]
 [-1.5 -4.   -6.2 ]]
```

Важно отметить, что так как это матрица $N \times M$, то у каждой строчки должно быть равное количество элементов. Попробуем создать следующий массив:

```
arr_3 = np.array([[1, 2, 3], [1, 2]])
print(arr_3)
```

Результат работы программы:

```
ValueError: setting an array element with a sequence. The
requested array has an inhomogeneous shape after 1 dimensions.
The detected shape was (2,) + inhomogeneous part.
```

Переведем данное сообщение об ошибке:

Ошибка значения: задание элемента массива последовательностью. Создаваемый массив имеет неоднородную размерность после 1 измерения. Определяемый размер был 2 + неоднородная часть.

Что означает, что первое измерение не содержит ошибок – присутствует 2 строчки, неоднородность наблюдается у второго измерения (В первом случае была задана 3, а во втором - 2).

Для того, чтобы обратиться к элементу двумерного списка, требуется использовать последовательно два оператора индексирования:

```
list_1 = [[1, 2, 3], [0, 25]]  
print(list_1[1][0])
```

Результат работы программы:

0

Библиотека NumPy при работе со структурой ndarray позволяет обращаться к элементам многомерного массива куда удобнее. Обращение к нескольким измерениям производится через запятую:

```
arr_1 = np.array([[1, 2, 3], [0, 25, 12]])  
print(arr_1[1, 0])
```

Такой способ обращения к элементам массива был достигнут путем перегрузки оператора индексирования – на самом деле туда передается кортеж целых чисел, т.е. можно было написать и так:

```
print(arr_1[(1, 0)])
```

Также его можно применять и для присваивания новых значений элементам массива:

```
arr_1[0,2] = 125  
print(arr_1)
```

15.1.2. Типизация массивов

Тип данных логически выводится из типа элементов последовательности и «принудительно приводится» к наиболее общему типу, если заданы элементы различных, но совместимых типов:

```
# Создадим массив со значениями int и float  
arr = np.array([22, 3, 4.])  
print(arr)
```

Результатом работы программы будет массив, в котором все значения находятся во float:

[22. 3. 4.]

Помимо неявного преобразования типов, можно явно указать какого типа должны быть элементы массива с помощью параметра `dtype`. Например, пусть массив содержит комплексные числа:

```
compl_arr = np.array([-5, 1, 3], dtype=complex)
print(compl_arr)
```

Результат работы программы:

```
[-5.+0.j  1.+0.j  3.+0.j]
```

15.1.3. Создание формы массива

Для создания формы массива, у которого изначально могут быть неизвестные значения или их часть можно применить функции `np.empty`, `np.zeros` и `np.ones`.

Функция `np.empty` принимает кортеж, описывающий форму массива, заполняя все элементы «мусором», оставшихся в памяти. Создадим массив, у которого 3 измерения – в первом 2 значения, во втором 5, в третьем - 3:

```
e_array = np.empty((2, 5, 3))
print(e_array)
```

Результат работы программы:

```
[[[9.67190439e-312  9.67185546e-312  6.96770841e+252]
  [1.72369955e-047  5.53588267e-048  6.01347002e-154]
  [3.92573072e-027  9.09495933e+223  3.93379755e-085]
  [6.56726362e+299  6.01119508e+175  8.59283573e-047]
  [3.02024221e-086  2.60339101e+209  1.05835773e-153]]

 [[5.83014740e+252  9.35082713e+252  4.88941582e-085]
  [4.87884920e+199  1.10919388e+200  9.28403227e+242]
  [2.22298114e+271  1.01444753e-091  1.18600496e-259]
  [1.28091380e+213  2.81325517e+179  3.85589708e-086]
  [4.83245960e+276  1.05089882e-153  6.99439131e+274]]]
```

Вместо того, чтобы заполнять массив мусором, можно воспользоваться функциями `np.zeros` и `np.ones` – каждая из них создает массив, который заполнен нулями или единицами соответственно. При создании массива с применением каждой из этих функций по умолчанию массив будет обладать типом `float`, это можно изменить с помощью параметра `dtype`. Попробуем на практике:

```
# Создание массива нулей типа float
```

```

z_array = np.zeros((2,3))
print(f'z_array:\n{z_array}\n')

# Создание массива единиц типа int
o_array = np.ones((3,3), dtype=int)
print(f'o_array:\n{o_array}')

```

Результат работы программы:

```

z_array:
[[0. 0. 0.]
 [0. 0. 0.]]

```

```

o_array:
[[1 1 1]
 [1 1 1]
 [1 1 1]]

```

Создавать форму массива не обязательно с нуля, можно его создать по образу уже существующего массива с помощью функций `np.empty_like`, `np.zeros_like` и `np.ones_like`. Создаваемый массив по умолчанию наследует тип данных `dtype` исходного массива. Проверим работу функций на практике:

```

# Создадим массив, по образу которого
# будут создаваться остальные
arr = np.array([[22.33, 5, 3.75], [1, 4.2, -5.5]])

# Массив с мусором, имеющий тип float
# (наследует тип от создаваемого)
a = np.empty_like(arr)
print(f'a:\n{a}')

# Массив из единиц, изменим тип на int
b = np.ones_like(arr, dtype=int)
print(f'b:\n{b}')

# Массив из нулей, изменим тип на complex
c = np.zeros_like(arr, dtype=complex)
print(f'c:\n{c}')

```

Результат работы программы:

```

a:
[[3.98746202e+252 2.44985911e-312 0.00000000e+000]
 [0.00000000e+000 0.00000000e+000 0.00000000e+000]]
b:
[[1 1 1]
 [1 1 1]]

```

```
c:
[[0.+0.j 0.+0.j 0.+0.j]
 [0.+0.j 0.+0.j 0.+0.j]]
```

15.1.4. Создание массива из последовательности

Одним из способов создания массива, содержащего последовательность чисел, является `np.arange`, который отчасти похож на функцию `range` в Python. Главным отличием будет являться то, что `np.arange` действительно создает массив, в то время как `range` не создает список.

Функция `np.arange` принимает следующие параметры (`start`, `stop`, `step`):

- `start` – начало последовательности (по умолчанию 0);
- `stop` – конец последовательности (обязательный аргумент);
- `step` – шаг (по умолчанию 1).

Создадим пару массивов с помощью `np.arange`:

```
# Массив от 0 до 6
a = np.arange(6)
print(f'a:\n{a}')

# Массив от 2.5 до 5 шагом 0.5
b = np.arange(2.5, 5., 0.5)
print(f'b:\n{b}')
```

Результат работы программы таков:

```
a:
[0 1 2 3 4 5]
b:
[2.5 3.  3.5 4.  4.5]
```

Заметьте, что также как и со срезами, и с `range` стандартного Python, «до» означает, что указанное в `stop` значение не включается в последовательность.

Другим способом создания массива из последовательности чисел является `np.linspace`, который имеет следующие основные параметры (`start`, `stop`, `num`):

- `start` – начало последовательности (обязательный параметр);
- `stop` – конец последовательности (обязательный параметр);
- `num` – количество элементов в промежутке последовательности (по умолчанию 50).

Опробуем np.linspace на практике:

```
# Создать массив от 0 по 10 на 50 элементов
a = np.linspace(0, 10)
print(f'a:\n{a}')
```

```
# Создать массив от 1 по 40 на 8 элементов
b = np.linspace(1, 40, 8)
print(f'b:\n{b}')
```

Результат работы программы:

```
a:
[0. 0.20408163 0.40816327 0.6122449 0.81632653 1.02040816
 1.2244898 1.42857143 1.63265306 1.83673469 2.04081633 2.24489796
 2.44897959 2.65306122 2.85714286 3.06122449 3.26530612
 3.46938776 3.67346939 3.87755102 4.08163265 4.28571429
 4.48979592 4.69387755 4.89795918 5.10204082 5.30612245
 5.51020408 5.71428571 5.91836735 6.12244898 6.32653061
 6.53061224 6.73469388 6.93877551 7.14285714 7.34693878
 7.55102041 7.75510204 7.95918367 8.16326531 8.36734694
 8.57142857 8.7755102 8.97959184 9.18367347 9.3877551
 9.59183673 9.79591837 10.]
```

```
b:
[1. 6.57142857 12.14285714 17.71428571 23.28571429 28.85714286
 34.42857143 40.]
```

Функция `np.linspace` позволяет получить размер шага в генерируемой последовательности чисел с помощью параметра `retstep` (сокращенно «return step»). Передадим параметру `retstep` аргумент `True` в следующей задаче:

Требуется определить каким шагом разделены 100 точек в диапазоне значений от 0 по 2π .

```
x, dx = np.linspace(0., 2*np.pi, 100, retstep=True)
print(dx)
```

Результат работы программы:

```
0.06346651825433926
```

Действительно, включая граничные значения получится $2\pi/99 = 0.0634665....$

С помощью передачи параметру `endpoint` аргумента `False` можно исключить конечную точку из последовательности, чтобы читалось как «с `start` до `stop`» как у `np.arange`:

```
# Создать массив от 0 до 5 на 5 элементов
x = np.linspace(0, 5, 5, endpoint=False)
print(x)
```

Результат работы программы:

```
[0.  1.  2.  3.  4.]
```

15.1.5. Создание массива на основе функции

Массивы можно также создавать на основе работы функций, для этого используется функция `np.fromfunction`, которая принимает в качестве аргументов сущность функции и кортеж из значений, которые представляют из себя форму создаваемого массива.

Назовем функцию, на основе которой создается массив **функцией-конструктором**, она должна принимать такое же количество элементов, сколько и измерений у создаваемого массива. Массив будет заполнен вычислением функции-создателя для каждого из элементов массива на основе его индексов в измерениях.

Формально работу `np.fromfunction` можно описать как:

`<a> = np.fromfunction(<b> , (<c>))`, где:

- `a` – переменная массива;
- `b` – имя функции-конструктора;
- `c` – форма создаваемого массива по каждому измерению через запятую.

Создадим массив на основе функции, каждый элемент которого будет равняться $2 * i * j$, где `i` и `j` – позиции элемента по измерениям:

```
# Определим функцию-конструктор
def func(i,j):
    return 2 * i * j

# Точка входа
if __name__ == '__main__':
    arr = np.fromfunction(func, (5,4))
    print(arr)
```

Результат работы программы:

```
[[ 0.  0.  0.  0.]
 [ 0.  2.  4.  6.]
 [ 0.  4.  8. 12.]
 [ 0.  6. 12. 18.]
 [ 0.  8. 16. 24.]]
```

Попробуем:

```
print(i)
print(j)
```

```
[[0. 0. 0. 0.]
 [1. 1. 1. 1.]
 [2. 2. 2. 2.]
 [3. 3. 3. 3.]
 [4. 4. 4. 4.]]
```

```
[[0. 1. 2. 3.]
 [0. 1. 2. 3.]
 [0. 1. 2. 3.]
 [0. 1. 2. 3.]
 [0. 1. 2. 3.]]
```

Попрактикуемся:

Создайте массив на основе функции, который будет содержать 20 элементов, в котором каждый шестой – единица, остальные – нуль.

Результатом работы программы должно быть значение массива:

```
[1 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0]
```

```
def func_1(i):
    print(f'i%6:\n{i%6}')
    print(f'i%6==0: \n{i%6==0}')
    return (i%6==0)*1

arr = np.fromfunction(func_1, (20, ))
print(arr)
```

15.1.6. Полезные поля класса ndarray

У объектов класса ndarray можно обратиться к различным полям для получения полезной информации, некоторые из них представлены в таблице ниже:

Поле	Содержание
shape	Измерения массива, возвращается в виде кортежа целых чисел
ndim	Количество измерений. <code>ndim == len(shape)</code>
size	Общее количество элементов в массиве
dtype	Тип данных массива
itemsize	Размер в байтах каждого элемента массива

Проверим данные полей на практике:

```
arr = np.array([[23, 100, 25], [-10, 0, 78]])
print(f'Измерения массива: {arr.shape}')
print(f'Количество измерений: {arr.ndim}')
print(f'Общее количество элементов: {arr.size}')
print(f'Тип данных массива: {arr.dtype}')
print(f'Размер в байтах элемента: {arr.itemsize}')
```

Результат работы программы:

```
Измерения массива: (2, 3)
Количество измерений: 2
Общее количество элементов: 6
Тип данных массива: int32
Размер в байтах элемента: 4
```

15.2. Основные типы данных dtype

До сих пор создаваемые в наших примерах массивы NumPy содержали целые числа или числа с плавающей точкой, и мы позволяли Python взять на себя ответственность о подробностях их представления. Но библиотека NumPy предоставляет мощный инструмент определения этих подробностей в явной форме с помощью объектов типа данных (data type). Это необходимо, потому что для организации взаимодействия (интерфейса) со скомпилированным кодом C более низкого уровня элементы массива NumPy обязательно должны сохраняться в совместимом формате: т. е. каждый элемент

представлен в виде фиксированного количества байтов, которые интерпретируются особым способом.

Например, рассмотрим беззнаковое (unsigned) целое число, хранящееся в 2 байтах (16 бит) памяти (тип языка C `uint_16t`). Такое число может принимать значение между 0 и $2^{16} - 1 = 65\,535$. В Python нет собственного равнозначного типа, в точности соответствующего этому представлению: целые числа в Python являются знаковыми величинами, а память для них динамически выделяется по их конкретному размеру. Поэтому NumPy определяет объект типа данных `np.uint16` для описания данных, хранящихся в формате языка C.

Полный список числовых типов данных приведен в официальной документации библиотеки NumPy (<https://docs.scipy.org/doc/numpy/user/basics.types.html>), но наиболее часто используемые перечислены в табл. 6.2. Все эти типы существуют в пакете `numpy`, поэтому на них можно ссылаться, например, так: `np.uint16`. К типам данных, которые создаются по умолчанию при использовании встроенных числовых типов языка Python, добавляется конечный символ подчеркивания: `np.float_`, `np.complex_` и `np.bool_`.

Тип данных	Описание
<code>int_</code>	Тип целого числа по умолчанию, соответствующий типу <code>long</code> языка C: зависит от платформы
<code>int8</code>	Целое число в 1 байте: от -128 до 127
<code>int16</code>	Целое число в 2 байтах: от $-32\,768$ до $32\,767$
<code>int32</code>	Целое число в 4 байтах: от $-2\,147\,483\,648$ до $2\,147\,483\,647$
<code>int64</code>	Целое число в 8 байтах: от -2^{63} до $2^{63} - 1$
<code>uint8</code>	Беззнаковое целое число в 1 байте: от 0 до 255
<code>uint16</code>	Беззнаковое целое число в 2 байтах: от 0 до 65 535
<code>uint32</code>	Беззнаковое целое число в 4 байтах: от 0 до 4 294 967 295
<code>uint64</code>	Беззнаковое целое число в 8 байтах: от 0 до $2^{64} - 1$
<code>float_</code>	Тип числа с плавающей точкой по умолчанию, другое имя для типа <code>float64</code>
<code>float32</code>	Знаковое число с плавающей точкой обычной точности: $\sim 10^{-38}$ до $\sim 10^{38}$ с ~ 7 точными знаками после десятичной точки
<code>float64</code>	Знаковое число с плавающей точкой двойной точности: $\sim 10^{-308}$ до $\sim 10^{308}$ с ~ 15 точными знаками после десятичной точки

<code>complex_</code>	Тип комплексного числа по умолчанию, другое имя для типа <code>complex128</code>
<code>complex64</code>	Комплексное число обычной точности (представленное 32-битовыми компонентами с плавающей точкой для действительной и мнимой частей)
<code>complex128</code>	Комплексное число двойной точности (представленное 64-битовыми компонентами с плавающей точкой для действительной и мнимой частей)
<code>bool_</code>	Логический тип по умолчанию, представленный 1 байтом

15.3. Универсальные функции (`unifunc`)

В дополнение к основным арифметическим операциям сложения, деления и т. п. NumPy предоставляет многие из хорошо знакомых математических функций, которые содержатся в модуле `math` (см. раздел 2.2.2), реализованные в виде так называемых универсальных функций. Универсальные функции обрабатывают каждый элемент массива, возвращая в результате массив без необходимости явного применения цикла. Универсальные функции – это способ, которым библиотека NumPy обеспечивает векторизацию, позволяющую писать ясный, эффективный и удобный для сопровождения код.

Например:

```
a = np.array([[1,2,3], [1,2,3]])
b = np.array([[3,4,5], [3,4,5]])
# Поэлементное умножение матриц
c = a * b
print(c)

# Умножение матриц
# 3x2 @ 2x3
d = a.transpose() @ b
print(d)
```

Метод `linspace` и векторизация

Для построения и вывода графика функции $y = \sin^2 x$ в примере из предыдущего раздела потребовалось достаточно много работы, самая большая часть которой пришлась на формирование списков x и y . Библиотека NumPy, более подробно описанная в главе 6, может существенно упростить жизнь разработчика.

Во-первых, последовательность координат x с постоянным шагом (список x) можно создать с использованием метода `linspace`. Метод очень похож на версию встроенной функции `range` для чисел с плавающей точкой: он принимает начальное и конечное значения, а также количество значений в последовательности и генерирует массив значений, представляющий арифметическую прогрессию между двумя заданными числами (включая сами эти граничные числа). Например, `x = np.linspace(-5, 5, 1001)` создает последовательность: $-5.0, -4.99, -4.98, \dots, 4.99, 5.0$.

Во-вторых, аналоги методов из модуля `math` в библиотеке NumPy могут работать с итерируемыми объектами (такими как списки или массивы NumPy). Поэтому выражение `y = np.sin(x)` создает последовательность значений (в действительности это массив NumPy `ndarray`), равных $\sin(x_i)$ для каждого значения x_i из массива x :

```
import numpy as np
import matplotlib.pyplot as plt
n = 1000
xmin, xmax = -2*np.pi, 2*np.pi
x = np.linspace(xmin, xmax, n)
y = np.sin(x)**2
plt.plot(x, y)
plt.show()
```

Эта операция называется векторизацией (*vectorization*) и описывается более подробно в разделе 6.1.3. Списки и кортежи можно преобразовать в объекты массивы, поддерживающие операцию векторизации, с помощью метода-конструктора `array`:

```
>>> w = [1.0, 2.0, 3.0, 4.0]
>>> w = np.array(w)
>>> w * 100          # Каждый элемент умножается на 100.
array([ 100., 200., 300., 400.])
```

Для добавления второй линии в график нужно просто вызвать еще раз метод `plt.plot`:

```
...
x = np.linspace(xmin , xmax , n)
y1 = np.sin(x)**2
y2 = np.cos(x)**2
plt.plot(x,y1)
plt.plot(x,y2)
plt.show()
```

Следует отметить, что после завершения вывода графика с помощью метода `show` или после сохранения с помощью метода `savefig` график становится недоступным для повторного вывода – для этого необходимо еще раз вызвать метод `plt.plot`. Причина в процедурной сущности интерфейса `matplotlib`: каждый вызов метода `matplotlib.pyplot.plot` изменяет внутреннее состояние объекта графика. Объект графика создается последовательными вызовами таких методов (добавление линий, подписей и меток, установка граничных значений осей и т. д.), затем сформированный объект графика выводится на экран или сохраняется в файле.

Пример П3.2. Функция `sinc` описывается такой формулой:

$$f(x) = \sin x / x.$$

Для построения ее графика в пределах $-20 \leq x \leq 20$ выполняется следующий код:

```
>>> x = np.linspace(-20, 20, 1001)
>>> y = np.sin(x)/x

__main__:1: RuntimeWarning: invalid value encountered in true_divide
>>> plt.plot(x,y)
>>> plt.show()
```

Обратите внимание: несмотря на то что Python предупреждает о делении на нуль при $x = 0$, график функции создается и выводится правильно: для этой особой точки устанавливается специальное значение `nan` (сокращение выражения «Not a Number» – «не число»), и на графике эта точка не отображается (см. рис. 3.4).

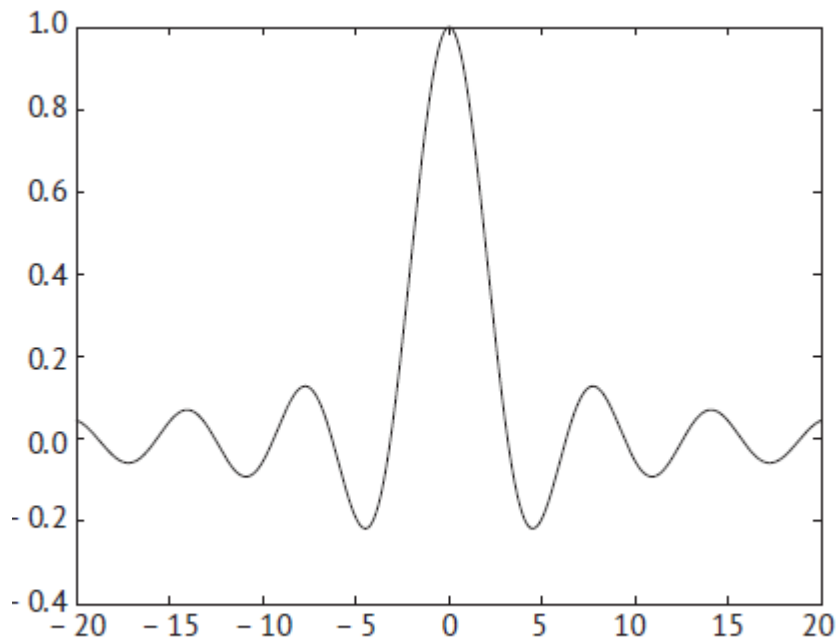


Рис. 3.4. График функции $y = \text{sinc}(x)$

15.4. Разница между вектором и матрицей

Вектор и матрица имеют принципиальную разницу - у одного размерность $(X,)$, а у другого - (X, Y) . Даже если у матрицы одна строка, то это всё равно матрица вида $(X, 1)$.

`layer_0 = streetlights[i]` - вектор из i -ой строки

`layer_0 = streetlights[i:i + 1]` - матрица, содержащую i -ую строку от оригинальной

15.5. Операции над матрицами

15.5.1. Транспонирование матрицы

Транспонирование матрицы:

```
result = np.array([[1], [1], [0], [0]])
[[1]
 [1]
 [0]
 [0]]
```

Если же использовать метод transpose или же просто «Т»:

```
result = np.array([[1], [1], [0], [0]]).T  
[[1 1 0 0]]
```

15.5.2. Умножение матриц

$a*b$ и `np.dot(a,b)`

`np.dot` - Есть разница что передаешь первое!

```
hidden = np.dot(weights[0], input)
```

и

```
hidden = np.dot(input, weights[0]) - разные ответы!!!
```

Telegram:

telebot

Backend сайтов:

Django Дронов

Парсеры

urllib

Работа с файлами:

os

shutil

Python и Blender

bpy

Excel

openpyxl

Потоки

threading

Потоки в Qt

COM-интерфейсы для конвертирования из docx в pdf

16. Регулярные выражения

Регулярные выражения (РВ) – это формальный язык, который определяет шаблон для поиска и внесения изменений в тексте.

Синтаксис регулярных выражений задается с помощью различных символов и метасимволов, их значение представлено ниже:

Метасимвол	Значение
.	Соответствует любому только 1 символу
[]	Соответствует одному символу из содержащихся в квадратных скобках
–	Граница последовательности символов в квадратных скобках
[^]	Соответствует одному символу из НЕ содержащихся в квадратных скобках
^	Соответствует началу строки
\$	Соответствует концу строки
*	Соответствует 0 или более предыдущих элементов
?	Соответствует 0 или одному предыдущему элементу
+	Означает, что предшествующий символ присутствует и может повторяться несколько раз
	Операция ИЛИ. Соответствует либо выражению до операции, либо выражению после нее
\	Символ управляющей последовательности. Позволяет использовать метасимвол как литерал
()	Группирует символы в подстроки
{ }	Количество повторений предыдущего символа

Также стоит упомянуть специальные последовательности:

Последовательность	Значение
\d	Любая цифра
\D	Все, кроме цифр

\w	Любая цифра, буква или знак подчеркивания
\W	Все, кроме цифр, букв и знака подчеркивания
\s	Любой пробельный символ
\S	Все, кроме пробельных символов
\b	Определяет только пустую строку, обозначающую начало или конец слова. Слово состоит из последовательности букв. Важно, что \b определяется как разделитель между \w и \W символами, другими словами, между \w и началом/концом строки. Это означает, что под шаблон r'\bfoo\b' подходят 'foo', 'foo.', '(foo)', 'bar foo baz' но не 'foobar' или 'foo3'.

Для работы с регулярными выражениями требуется подключить библиотеку re, что сокращенно и означает regular expressions (регулярные выражения).

Основными функциями библиотеки являются:

- `match` – осуществляет поиск первого совпадения на основе заданного шаблона. Ищет только от начала строки и возвращает подходящий объект, если такой существует. Но если подходящее значение где-то в середине строки, возвратит `None`.
- `search` - осуществляет поиск первого совпадения на основе заданного шаблона. Ищет по всей строке, даже если в ней несколько строчек и старается найти подходящую подстроку по всем строчкам строки.
- `findall` – осуществляет поиск всех совпадений на основе заданного шаблона.
- `split` – позволяет разбивать текст на подстроки

Один из сайтов, на которых можно отрабатывать синтаксис регулярных выражений:

<https://regex101.com/r/aGn8QC/1163>

16.1. Поиск первого совпадения. Функции match и search

16.1.1. Функция match

Опробуем данную функцию – запишем в отдельную переменную шаблон РВ – назовем её pattern, также в отдельную запишем входной текст - text. Затем, вызывая функцию библиотеки re будем её передавать эти значения как аргумент и получать результат.

В качестве шаблона РВ передадим «.», что означает «значение соответствует любому **только 1** символу».

```
pattern = '.'
text = 'Hello everyone!\nToday I will...'
result = re.match(pattern, text)
print(result)
```

Результатом выполнения программы будет:

```
<re.Match object; span=(0, 1), match='H'>
```

Разберем – здесь написано, что подошла подстрока от 0 до 1 (как у срезов), а содержимое этой подстроки ‘Н’, т.е. первая буква предложения «Hello everyone!».

Для того чтобы вывести только результат ‘Н’ можно воспользоваться методом group:

```
pattern = '.'
text = 'Hello everyone!\nToday I will...'
result = re.match(pattern, text)
print(result.group())
```

Сделаем проверку на то, соответствует ли строка формату номера телефона. Примером номера телефона будет «8 (921) 333-22-11», шаблоном должен быть «8\s(\d{3})\s\d{3}\-\d{2}\-\d{2}», что означает: «сначала идет число 8, затем пробел «\s», после этого должна открываться круглая скобка(так как открывающая скобка является частью специальной последовательности, то чтобы распознать именно символ «(» нужно писать как «\((»), затем должно следовать три цифры, т.е. \d{3}, где в фигурных скобках указывается количество цифр, которые должны следовать друг за другом и т.д...».

```

pattern = '8\s\(\d{3}\)\s\d{3}\-\d{2}\-\d{2}'
text = '8 (921) 333-22-11'
result = re.match(pattern, text)
print(result)

```

Применим шаблон посложнее. Например, запросим вывести первое слово, которое начинается с буквы «m». Рассмотрим таблицы метасимволов и специальных последовательностей, нужно написать шаблон, в котором говорится «слово начинается с **буквы m**, затем может идти **любой другой символ до пробела**».

Для того чтобы получить любой другой символ до пробела – воспользуемся \w. В общем выглядеть шаблон будет как: «m\w+».

Зададим этот шаблон снова в pattern, а также возьмем куда больший текст для примера:

```

pattern = 'm\w+'
text = '''
Some Visual Novel with no name
As you can see, I'm nothing but a hopelessly average high school
student.
Average on the outside, average on the inside.

I really don't have much going for me when it comes to grades,
special abilities,
even personality. I mean yeah, I have hobbies and stuff I like
to do, but it's not like
I'm a psychic or mutant or whatever. Like, if you asked me what
my favorite song was, or my
favorite movie or TV show... They'd all just be whatever's most
popular at that particular
moment. Even among the average, I'm completely average. So I
can't even say I'm your
"everyday hero" type. That's just who I am.
'''
result = re.match(pattern, text)
print(result)

```

Результатом работы программы будет:

None

Как же так? Неужели РВ написано неверно? Возвратимся к разнице между match и search. Match осуществляет поверхностный поиск, в то время как search будет проверять каждое слово и каждую строчку.

16.1.2. Функция search

Используем в коде `search` и проверим результат:

```
pattern = 'm\w+'
text = '''
Some Visual Novel with no name
As you can see, I'm nothing but a hopelessly average high school
student.
Average on the outside, average on the inside.

I really don't have much going for me when it comes to grades,
special abilities,
even personality. I mean yeah, I have hobbies and stuff I like
to do, but it's not like
I'm a psychic or mutant or whatever. Like, if you asked me what
my favorite song was, or my
favorite movie or TV show... They'd all just be whatever's most
popular at that particular
moment. Even among the average, I'm completely average. So I
can't even say I'm your
"everyday hero" type. That's just who I am.
'''
result = re.search(pattern, text)
print(result)
```

Результатом программы будет ответ:

```
<re.Match object; span=(3, 5), match='me'>
```

Т.е. первым словом, которое содержит `m` будет «Some», но ведь оно не начинается с «`m`». Да, изначально мы допустили ошибку, чтобы обозначить, что перед буквой «`m`» не должно быть других букв, можно поступить несколькими способами, например, можно попробовать поставить `\s` в начало шаблона и получить «`\sm\w+`».

Результатом работы такого шаблона будет действительно первое слово, начинающееся с буквы `m`.

```
<re.Match object; span=(174, 179), match=' much'>
```

Но вот незадача – также он прихватил и пробельный символ, за который и отвечает `\s`, т.е. если бы первым словом у строки было то, которое начинается с «`m`», то тогда шаблон бы не сработал – пробела перед словом не было бы. Воспользуемся специальной последовательностью `\b` – шаблоном «`\bm\w+`».

Отметим, что когда Вы будете назначать шаблон, то лучше перед строкой ставить префикс «`г`», что обозначает «raw string», или же «сырую строку». Некоторые специальные последовательности РВ могут пересекаться

с управляющими последовательностями самого языка Python, так \b (backspace) обозначает перевод каретки на один символ назад.

Использование сырой строки выглядит так:

```
pattern = r'\bm\w+'
```

Результатом работы будет:

```
<re.Match object; span=(175, 179), match='much'>
```

16.2. Поиск всех совпадений. Функция findall

Воспользуемся функцией findall, которая позволяет по шаблону получить сразу все подходящие результаты.

Доработаем код, чтобы для удобства выводился на экран не только результат работы шаблона, но и он сам – создадим функцию «pattern_result», которая через f-строку выводит «<шаблон РВ>: <результат работы функции>».

Выведены все символы, кроме \n. Снова с помощью шаблона «.»

```
import re

def pattern_result(pattern, result):
    print(f'{pattern}: {result}\n')

pattern = '.'
text = 'Hello everyone!\nToday I will...'
result = re.findall(pattern, text)
pattern_result(pattern, result)
```

Результатом будет:

```
.: ['H', 'e', 'l', 'l', 'o', ' ', 'e', 'v', 'e', 'r', 'y', 'o',
'n', 'e', '!', 'T', 'o', 'd', 'a', 'y', ' ', 'I', ' ', 'w', 'i',
'l', 'l', '.', '.', '.']
```

Для того, чтобы в списке не было пробела, можно воспользоваться \w:

```
pattern = '\w'
text = 'Hello everyone!\nToday I will'
result = re.findall(pattern, text)
pattern_result(pattern, result)
```

Возьмем каждое слово отдельно – воспользуемся как шаблоном «\w*».

* - означает 0 или более вхождений шаблона слева.

```
pattern = '\w*'
text = 'Hello everyone!\nToday I will'
result = re.findall(pattern, text)
pattern_result(pattern, result)
```

Результатом будет:

```
\w*: ['Hello', '', 'everyone', '', '', 'Today', '', 'I', '', 'will', '']
```

Пустые строки здесь – результат того, что в пробелах и других специальных символах (вместо восклицательного знака также пустая строка), как отдельных словах, не было ничего, что подходило бы под заданный шаблон. Чтобы от них избавиться, используем «+», вместо «*».

```
pattern = '\w+'
text = 'Hello everyone!\nToday I will'
result = re.findall(pattern, text)
pattern_result(pattern, result)
```

Результатом будет:

```
\w+: ['Hello', 'everyone', 'Today', 'I', 'will']
```

Для того, чтобы получить первое или последнее слово в тексте, воспользуемся «^» и «\$» соответственно. Доработаем программу, которая будет поддерживать работу двух шаблонов – первого для получения первого слова – «^\w+» pattern_front и второго для получения последнего слова «\w+\$» - pattern_back.

```
pattern_front = '^\w+'
pattern_back = '\w+$'

text = 'Hello everyone!\nToday I will'
result_front = re.findall(pattern_front, text)
pattern_result(pattern_front, result_front)

result_back = re.findall(pattern_back, text)
pattern_result(pattern, result_back)
```

Результатом работы будет:

```
^\w+: ['Hello']
\w+: ['will']
```

Можно сделать шаблон, который позволяет получить все буквы текста в качестве подстрок по две буквы – «\w\w»:

```
pattern = '\w\w'
text = 'Hello everyone!\nToday I will'
```

```
result = re.findall(pattern, text)
pattern_result(pattern, result)
```

Результатом работы будет:

```
\w\w: ['He', 'll', 'ev', 'er', 'yo', 'ne', 'To', 'da', 'wi', 'll']
```

Решим следующую задачу:

Дана строка с 5 email-адресами, разделенными через запятую. Нужно получить домены адресов из текста

Рассуждая с помощью РВ - нужно получить любые символы, которые идут после «@». Предположим, что это будет шаблон «@\\w*».

```
pattern = '@\\w*'
text = 'green44@gmail.com, singasong@gumrf.ru,
we.will.meet1987@yahoo.com, seiso7700@test.in, hell.taker@bk.ru'
result = re.findall(pattern, text)
pattern_result(pattern, result)
```

Результатом работы будет:

```
@\\w*: ['@gmail', '@gumrf', '@yahoo', '@test', '@bk']
```

Частично, задача решена, но не до конца. Рассмотрим на примере первой почты «green44@gmail.com» - получилось взять только «@gmail» - домен второго уровня, нужно также и верхнего – «.com».

Чтение шаблона @\\w* идет от @ до символа, не являющегося буквой или цифрой, т.е. здесь прерывается на точке для того чтобы продолжить чтение нужно добавить "после точки прочитай любую набор букв или символов" - @\\w+[.]\\w+.

Важно: «[.]» и «.» имеют разный смысл. «[.]» - символ точки «.» - правило "Один любой символ".

Сравним два шаблона с использованием «[.]» и «.» на двух текстах – верный текст, в котором почты написаны без ошибок и текст, в котором у почт вместо точки в домене верхнего уровня стоит пробел:

```
pattern_1 = '@\\w+[.]\\w+'
pattern_2 = '@\\w+\\.\\w+'
```

```
text_1 = 'green44@gmail.com, singasong@gumrf.ru,
we.will.meet1987@yahoo.com, seiso7700@test.in, hell.taker@bk.ru'
```

```

text_2 = 'green44@gmail com, singasong@gumrf ru,
we.will.meet1987@yahoo com, seiso7700@test in, hell.taker@bk ru'

result_1 = re.findall(pattern_1, text_1)
result_2 = re.findall(pattern_1, text_2)
result_3 = re.findall(pattern_2, text_1)
result_4 = re.findall(pattern_2, text_2)

print(f'Для доменов с точкой:')
pattern_result(pattern_1, result_1)
pattern_result(pattern_2, result_3)

print(f'Для доменов с пробелом:')
pattern_result(pattern_1, result_2)
pattern_result(pattern_2, result_4)

```

Результатом работы будет 4 списка:

Для доменов с точкой:

```
@\w+[\.]\w+: ['@gmail.com', '@gumrf.ru', '@yahoo.com',
 '@test.in', '@bk.ru']
```

```
@\w+.\w+: ['@gmail.com', '@gumrf.ru', '@yahoo.com', '@test.in',
 '@bk.ru']
```

Для доменов с пробелом:

```
@\w+[\.]\w+: []
```

```
@\w+.\w+: ['@gmail com', '@gumrf ru', '@yahoo com', '@test in',
 '@bk ru']
```

Таким образом, использование `[.]` даёт корректный результат. Если в написании почты была допущена ошибка, то она не войдёт в результат работы РВ.

Работа с датами. Получим только даты. Дата в формате - число, месяц и год через, например «01-02-2023» тире. Соответствующим шаблоном будет «`\d{2}[-]\d{2}[-]\d{4}`».

```

pattern = r'\d{2}[-]\d{2}[-]\d{4}'
text = '11-11-2022 asd 05-02-2000 85-aa-3333 DATE 12-April-1981
09-09-1961'
result = re.findall(pattern, text)
pattern_result(pattern, result)

```

Результатом работы будет список:

```
\d{2}[-]\d{2}[-]\d{4}: ['11-11-2022', '05-02-2000', '09-09-1961']
```

Если нужно получить ТОЛЬКО год, то заключим эту часть выражения в круглые скобки «\d{2}[-]\d{2}[-](\d{4})»:

```
pattern = r'\d{2}[-]\d{2}[-](\d{4})'
text = '11-22-2022 asd 05-02-2000 85-aa-3333 DATE 12-April-1981 09-09-1961'
result = re.findall(pattern, text)
pattern_result(pattern, result)
```

Результатом будет:

```
\d{2}[-]\d{2}[-](\d{4}): ['2022', '2000', '1961']
```

16.3. Комбинация нескольких регулярных выражений в одно

Есть проблема – шаблону, который распознавал даты выше можно передать и что-то, что похоже на дату, но ей не является, например, «99-99-0000». Доработаем шаблон! Разберем подробнее первую часть – день. Есть несколько вариантов допустимой даты. Первый вариант – если первая цифра – ноль, то вторая может быть от 1 до 9, тоже нулём быть она не может. Для таких условий подойдет шаблон «0[1-9]». Проверим в Python:

```
pattern = r'0[1-9]'
text = '02'
result = re.match(pattern, text)
pattern_result(pattern, result)
```

Результатом будет:

```
0[1-9]: <re.Match object; span=(0, 2), match='02'>
```

Если же передать «00», тогда программа выведет:

```
0[1-9]: None
```

Второй вариант – если первая цифра единица или двойка, тогда вторая может быть от 0 до 9, т.е. подойдет шаблон «[1-2][0-9]»:

```
pattern = r'[1-2][0-9]'
text = '11'
result = re.match(pattern, text)
pattern_result(pattern, result)
```

Результатом будет:

```
[1-2][0-9]: <re.Match object; span=(0, 2), match='11'>
```

Третий вариант – первая цифра – тройка. Вторая может быть только нулем или единицей, это шаблон «3[0-1]».

Создадим шаблон, который объединяет все три варианта в один – для этого воспользуемся специальной последовательностью «()», с помощью которой можно сгруппировать правила между собой, а также «|», которая является операцией ИЛИ, т.е. должно выполняться правило слева или справа. Объединим три шаблона – «(0[1-9])|([1-2][0-9])|(3[0-1])» и выполним проверку:

```
pattern = r'(0[1-9])|([1-2][0-9])|(3[0-1])'
days = ['11', '00', '20', '99', '44', '32']
for text in days:
    result = re.match(pattern, text)
    pattern_result(pattern, result)
```

Результатом работы программы будет:

```
(0[1-9])|([1-2][0-9])|(3[0-1]): <re.Match object; span=(0, 2),
match='11'>
(0[1-9])|([1-2][0-9])|(3[0-1]): None
(0[1-9])|([1-2][0-9])|(3[0-1]): <re.Match object; span=(0, 2),
match='20'>
(0[1-9])|([1-2][0-9])|(3[0-1]): None
(0[1-9])|([1-2][0-9])|(3[0-1]): None
(0[1-9])|([1-2][0-9])|(3[0-1]): None
```

Таким образом можно составить полное РВ, которое описывает «День-Месяц-Год» корректно.

16.4. Получение подстрок по шаблону. Функция split

Функция split позволяет разбить строку на подстроки на основе заданного шаблона РВ.

Представим ситуацию, что нужно из строки получить отдельные слова, т.е. «разделителями будут ни буквы, ни цифры, ни знак подчеркивания», что означает специальную последовательность \W, а если представить, что таких символов может идти друг за другом несколько, то «\W+»:

```
pattern = r'\W+'
text = 'Words, words, words'
result = re.split(pattern, text)
pattern_result(pattern, result)
```

Запустив такой код, получим:

```
\W+: ['Words', 'words', 'words']
```

Другой пример – слова перемешаны с различными другими символами, «Hopes,&*!,/[|[2]]6what|2131|does./it4mean» - получим только слова, зададим шаблон, который опишет разделители «`[^ A-Z]`», т.е. всё то, что не входит в «A-z» - разделитель.

```
pattern = r'[^ A-Z a-z]'  
text = 'Hopes,&*!,/[|[2]]6what|2131|does./it4mean'  
result = re.split(pattern,text)  
pattern_result(pattern,result)
```

При выполнении такого кода будет видно много пустых строк:

```
[^ A-Z a-z]: ['Hopes', '', '', '', '', '', '', '', '', '', '',  
'', '', '', 'what', '', '', '', '', '', 'does', '', 'it',  
'mean']
```

Для того, чтобы избавиться от них, доработаем шаблон, также как и в прошлом примере скажем, что таких разделителей друг за другом может быть несколько – добавим «+» в конце. При выполнении программы с шаблоном «`[^ A-Z a-z]+`» будет выведен верный список:

```
[^ A-Z a-z]+: ['Hopes', 'what', 'does', 'it', 'mean']
```

16.5. Задачи для самостоятельного выполнения

16.5.1. Легкие задачи

- 1) Дан текстовый файл, требуется вывести на экран список всех слов, которые начинаются на гласную.
- 2) Номер телефона записывается в формате «+7 999 111-22-33», сделать программу, которая определяет верность переданных данных.
- 3) Разбить строку по нескольким разделителям – запятая, точка с запятой, пробел, восклицательный знак, цифры, буквы d-f.
- 4) На вход программе подается некий текст. Требуется посчитать сколько в нём слов. Слова могут содержать как кириллицу, так и латиницу. Слова, написанные через дефис – это одно слово.

16.5.2. Определение регистрационных знаков ТС

В РФ имеется несколько различных типов регистрационных знаков для транспортных средств, которые позволяют понять назначение данного ТС.

Общее правило построения номеров:

Номера состоят только из комбинаций букв и цифр. Допустимыми буквами кириллицы только являются те, которые имеют аналог в латинице, т.е. А, В, Е, К, М, Н, О, Р, С, Т, У, Х.

Код региона – последовательность из двух цифр, позволяющий определить где зарегистрировано ТС.

Таблица. Частные правила для типов ТС

Тип ТС	Обозначение в программе	Пример номера	Правило построения
Легковые, грузовые автомобили и автобусы	Private	С227НА69	Буква, 3 цифры, 2 буквы, код региона
Такси	Taxi	АО36578	2 буквы, 3 цифры, код региона

Прицепы и полуприцепы	Trailer	АН733147	2 буквы, 4 цифры, код региона
Тракторы и мотоциклы	Tractors and motorcycles	3733MM55	4 цифры, 2 буквы, код региона
Внедорожник	SUV	8776AE64	2 буквы, 4 цифры, код региона
Мопед	Moped	MM55AA23	2 буквы, 2 цифры, 2 буквы, код региона

Вам будет предоставлено 4 различных файла, которые содержат некие данные, в т.ч. номера ТС. Если текст подходит под один из шаблонов, то требуется вывести, что номер соответствует сокращению в программе, иначе – None.

Формат входных данных	Формат выходных данных
ТЕКСР308Р7Е42К54В	ТЕКСР308Р7Е42К54В - None
АТ376764	АТ376764 - Trailer
МХ476304	МХ476304 - SUV
УЕ505055	УЕ505055 - Trailer
РА6ВКМВ2К0АО3Т44С65	РА6ВКМВ2К0АО3Т44С65 - None
ОС0КН80ХТР5Х6С	ОС0КН80ХТР5Х6С - None
О25ОС7УВАОКНР61Т	О25ОС7УВАОКНР61Т - None
5688РХ16	5688РХ16 - Tractors and motorcycles
7352ТО9СВ9В8ЕННТТ	7352ТО9СВ9В8ЕННТТ - None
ТОЕ	ТОЕ – None
	Results: Private – 0 Taxi – 0 Trailer – 2 Tractors and motorcycles – 1 SUV – 1

	Moped – 0
	None - 6

Задание 1. Создайте массив, содержащий 20 элементов последовательности Фибоначчи. Каждый элемент последовательности можно рассчитать по формуле Бине:

$$F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}}, \text{ где:}$$

$$\varphi = \frac{1 + \sqrt{5}}{2}$$

$$\psi = -\frac{1}{\varphi}$$

Результатом

Также дополните программу возможностью выбора проверяемого файла с клавиатуры.

16.5.3. Проверка формата даты

Требуется составить программу, в которой применяется РВ, описывающее корректно дату в формате «День-Месяц-Год». Важно помнить, что разные месяцы могут содержать от 28 до 31 дней.

Тема 11. Объектно-ориентированное программирование. (5/5)

Классы, методы, конструкторы, объекты, статические методы

Тема 13. Обработка структурированных текстовых данных (5/5)

Введение в XML, HTML, JSON, способы разбора XML (DOM, SAX), Разбор HTML с помощью сторонних библиотек.

Тема 14. Методы сбора информации из открытых источников и текстовых коллекций (5/5)

Загрузка файлов по протоколу HTTP, использование API для доступа к данным (Twitter, Wikipedia, VK), методы обработки и очистки текстовых данных, работа со открытыми тезаурусами и текстовыми корпусами, основы локального хранения и обработки данных

Тема 12. Разработка десктопных приложений

Многопоточность, разработка интерфейсов с PyQt5

Тема 15. Работа с данными и с неструктурированными текстовыми данными. (5/5)

Библиотеки numpy, pandas, matplotlib, NLTK, pymystem3 и pymorphy 2. Обзор основных задач компьютерной лингвистики: токенизация и морфологический разбор текста, построение частотного словаря, построение векторного представления слов, тематическая классификация текста, анализ тональности текста, автоматическое генерирование текста.